

# Architektura Systemów Komputerowych

Dr inż. Krzysztof Kosmowski

# CELE WYKŁADU

- Różnica między organizacją komputera i architekturą komputera
- Zrozumieć jednostki miary wspólne dla systemów komputerowych
- Pokazać ewolucję komputerów
- Komputer jako system warstwowy
- Zapoznanie z architekturą von Neumanna i funkcjami podstawowych komponentów komputera

# Organizacja i architektura systemu komputerowego

- *Organizacja komputera* – Obejmuje wszystkie fizyczne aspekty systemów komputerowych (np. projektowanie obwodów, sygnały sterujące, typy pamięci).
  - Jak działa komputer?
- *Architektura komputerowa* – Logiczne aspekty implementacji systemu widziane przez programistę (np. zestawy instrukcji, formaty instrukcji, typy danych, tryby adresowania).
  - Jak zaprojektować komputer?

# <sup>1</sup>Systemy komputerowe (1)

- Nie ma wyraźnego rozróżnienia między zagadnieniami związanymi z organizacją komputerów i sprawami istotnymi dla architektury komputerów
- *Zasada równoważności Sprzętu i Oprogramowania:*
  - Każde zadanie wykonywane przez oprogramowanie można również wykonać za pomocą sprzętu i wszelkie operacje wykonywane bezpośrednio sprzętowo można wykonać za pomocą oprogramowania\*

\*Zakładając, że prędkość nie jest problemem

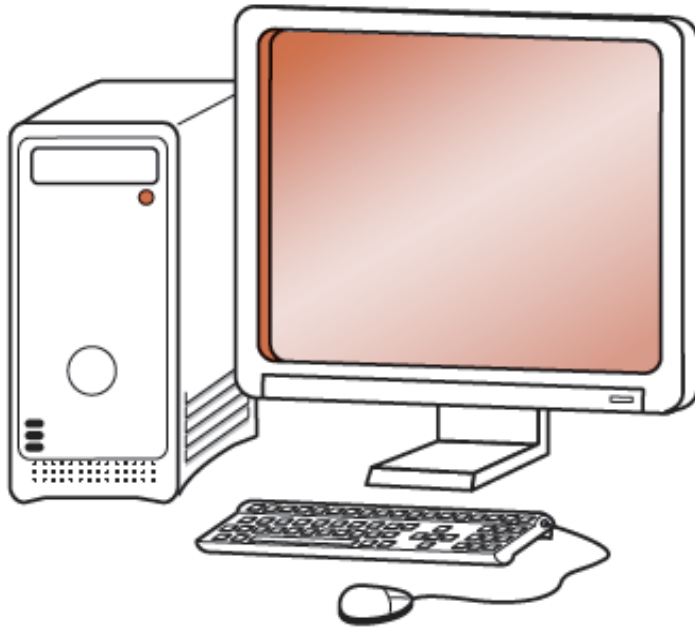
# Systemy komputerowe (2)

- Na najbardziej podstawowym poziomie komputer to urządzenie składające się z trzech części:
  - Procesor do interpretacji i wykonywania programów
  - Pamięć do przechowywania danych i programów
  - Mechanizm przesyłania danych do i ze świata zewnętrznego

# Przykładowy system (3)

L1 Cache??

## FOR SALE: OBSOLETE COMPUTER – CHEAP! CHEAP! CHEAP!



- Compact computer
- Intel i7 Quad Core, 4.20 GHz
- 2400MHz 32GB DDR4 SDRAM
- 128KB L1 cache, 2MB L2 cache
- Dual storage (7200RPM SATA 1TB HDD, 128GB SSD)
- Wireless 802.11 + Bluetooth 4.0
- 7-in-1 card reader
- 10 USB ports, 1 serial port, 4 PCI expansion slots (1 PCI, 1 PCI x 16, 2 PCI x 1), HDMI
- 24" widescreen LCD monitor, 16:10 aspect ratio, 1920 x 1200 WUXGA, 300 cd/m<sup>2</sup>, active matrix, 1000:1 (static), 8ms, 24-bit color (16.7 million colors), VGA/DVI input, 2 USB ports
- 16x CD/DVD +/- RW drive
- 1GB PCIe video card
- PCIe sound card
- Gigabit Ethernet

GHz??

GB??

PCI??

USB??

# Przykładowy system (4)

- Miary pojemności i prędkości:

- Kilo-(K) = 1 thousand / **tysiąc** =  $10^3$  ~  $2^{10}$
- Mega-(M) = 1 milion / **milion** =  $10^6$  ~  $2^{20}$
- Giga-(G) = 1 bilion / **miliard** =  $10^9$  ~  $2^{30}$
- Tera-(T) = 1 trillion / **bilion** =  $10^{12}$  ~  $2^{40}$
- Peta-(P) = 1 quadrillion / **biliard** =  $10^{15}$  ~  $2^{50}$

- To czy dana wartość odnosi się do potęgi dziesiątki, czy dwójki zazwyczaj zależy od tego, co jest mierzone

# Przykładowy system (5)

- **Herc** = cykle zegara na sekundę (częstotliwość)
    - 1MHz = 1,000,000Hz
    - Szybkości procesorów są mierzone w MHz lub GHz
  - **Bajt** = jednostka pamięci
    - 1KB =  $2^{10}$  = 1024 Bajty
    - 1MB =  $2^{20}$  = 1,048,576 Bajtów
    - 1GB =  $2^{30}$  = 1,099,511,627,776 Bajtów
    - Pamięć główna (RAM) jest mierzona w GB
    - Pamięć dyskowa jest mierzona w GB dla małych systemów, w TB ( $2^{40}$ ) dla dużych
- \*\* b = bit, B=Bajt (8bitów)

# Przykładowy system (6)

- Miary czasu i przestrzeni:

- |               |                    |              |                |
|---------------|--------------------|--------------|----------------|
| • mili - (m)  | = jedna tysięczna  | = $10^{-3}$  | $\sim 2^{-10}$ |
| • mikro(u)    | = jedna milionowa  | = $10^{-6}$  | $\sim 2^{-20}$ |
| • nano - (n)  | = jedna miliardowa | = $10^{-9}$  | $\sim 2^{-30}$ |
| • pico - (p)  | = jedna bilionowa  | = $10^{-12}$ | $\sim 2^{-40}$ |
| • femto - (f) | = jedna biliardowa | = $10^{-15}$ | $\sim 2^{-50}$ |

## 1.3 Przykładowy system (7)

- Milisekunda = 1 tysięczna sekundy
  - Czasy dostępu do dysku twardego często wynoszą od 10 do 20 milisekund
- Nanosekunda = 1 miliardowa część sekundy
  - Czasy dostępu do pamięci głównej często wynoszą od 50 do 70 nanosekund
- Mikron (mikrometr) = 1 milionowa część metra
  - Obwody w chipach komputerowych są mierzone w mikronach

## 1.3 Przykładowy system (8)

- **Zauważamy, że *czas cyklu* jest odwrotnością *częstotliwości zegara***
- Magistrala działająca z częstotliwością 133 MHz ma czas cyklu 7,52 nanosekund:
  - $1/(133\ 000\ 000\ \text{cykli/sekundę}) = 7,52\ \text{ns/cykl}$

Wróćmy do ogłoszenia

## 1.3 Przykładowy system (9)

**FOR SALE: OBSOLETE COMPUTER – CHEAP! CHEAP! CHEAP!**



- Compact computer
- Intel i7 Quad Core, 4.20 GHz
- 2400MHz 32GB DDR4 SDRAM
- 128KB L1 cache, 2MB L2 cache
- Dual storage (7200RPM SATA 1TB HDD, 128GB SSD)
- Wireless 802.11 + Bluetooth 4.0
- 7-in-1 card reader
- 10 USB ports, 1 serial port, 4 PCI expansion slots (1 PCI, 1 PCI x 16, 2 PCI x 1), HDMI
- 24" widescreen LCD monitor, 16:10 aspect ratio, 1920 x 1200 WUXGA, 300 cd/m<sup>2</sup>, active matrix, 1000:1 (static), 8ms, 24-bit color (16.7 million colors), VGA/DVI input, 2 USB ports
- 16x CD/DVD +/- RW drive
- 1GB PCIe video card
- PCIe sound card
- Gigabit Ethernet

Mikroprocesor jest „mózgiem” systemu. Wykonuje instrukcje zapisane w programach komputerowych. Ten to Intel i7 pracujący z szybkością 4.2 GHz

Czyli

4 200 000 000 cykli /s

Cykl =  $2,38 \times 10^{-10}$  s

czyli

0,238 ns/cykl

## 1.3 Przykładowy system (10)

- Komputery o dużej pojemności pamięci głównej mogą uruchamiać większe programy z większą prędkością niż komputery z małą pamięcią
- **RAM** (*random access memory*) to akronim oznaczający *dostęp losowy do pamięci*
  - Dostęp losowy oznacza, że zawartość pamięci jest dostępna bezpośrednio, jeśli znasz lokalizację zasobu
- **Cache** to rodzaj pamięci tymczasowej, która może być dostępna szybciej niż RAM

## 1.3 Przykładowy system (11)

**FOR SALE: OBSOLETE COMPUTER – CHEAP! CHEAP! CHEAP!**



- Compact computer
- Intel i7 Quad Core, 4.20 GHz
- 2400MHz 32GB DDR4 SDRAM
- 128KB L1 cache, 2MB L2 cache
- Dual storage (7200RPM SATA 1TB HDD, 128GB SSD)
- Wireless 802.11 + Bluetooth 4.0
- 7-in-1 card reader
- 10 USB ports, 1 serial port, 4 PCI expansion slots (1 PCI, 1 PCI x 16, 2 PCI x 1), HDMI
- 24" widescreen LCD monitor, 16:10 aspect ratio, 1920 x 1200 WUXGA, 300 cd/m<sup>2</sup>, active matrix, 1000:1 (static), 8ms, 24-bit color (16.7 million colors), VGA/DVI input, 2 USB ports
- 16x CD/DVD +/- RW drive
- 1GB PCIe video card
- PCIe sound card
- Gigabit Ethernet

Ten system ma 32 GB pamięci RAM (szybkien, synchronicznej z dynamicznym przydziałem, SDRAM (*synchronous dynamic RAM*))

... oraz dwa poziomy pamięci Cache, poziom 1 (L1) jest mniejszy i (prawdopodobnie) szybszy niż poziom L2.

*Zauważ. Że te pamięci Cache mają rozmiary mierzone w KB i MB*

## 1.3 Przykładowy system (12)

**FOR SALE: OBSOLETE COMPUTER – CHEAP! CHEAP! CHEAP!**



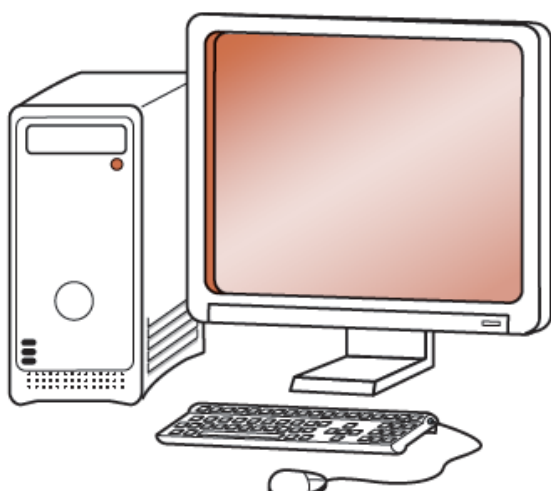
- Compact computer
- Intel i7 Quad Core, 4.20 GHz
- 2400MHz 32GB DDR4 SDRAM
- 128KB L1 cache, 2MB L2 cache
- **Dual storage (7200RPM SATA 1TB HDD, 128GB SSD)**
- Wireless 802.11 + Bluetooth 4.0
- 7-in-1 card reader
- 10 USB ports, 1 serial port, 4 PCI expansion slots (1 PCI, 1 PCI x 16, 2 PCI x 1), HDMI
- 24" widescreen LCD monitor, 16:10 aspect ratio, 1920 x 1200 WUXGA, 300 cd/m<sup>2</sup>, active matrix, 1000:1 (static), 8ms, 24-bit color (16.7 million colors), VGA/DVI input, 2 USB ports
- 16x CD/DVD +/- RW drive
- 1GB PCIe video card
- PCIe sound card
- Gigabit Ethernet

Ten dysk twardy może przechować 1TB.  
7200RPM jest to szybkość obracania się talerzy dysku. Generalnie im szybsza prędkość obrotowa tym szybciej dysk może dostarczyć dane do RAM (zależy to również od wielu innych aspektów)

Ten komputer ma również dysk półprzewodnikowy (trochę za mały jak na współczesne zastosowania)

## 1.3 Przykładowy system (13)

**FOR SALE: OBSOLETE COMPUTER – CHEAP! CHEAP! CHEAP!**



- Compact computer
- Intel i7 Quad Core, 4.20 GHz
- 2400MHz 32GB DDR4 SDRAM
- 128KB L1 cache, 2MB L2 cache
- Dual storage (7200RPM, SATA 1TB HDD, 128GB SSD)
- Wireless 802.11 + Bluetooth 4.0
- 7-in-1 card reader
- 10 USB ports, 1 serial port, 4 PCI expansion slots  
(1 PCI, 1 PCI x 16, 2 PCI x 1), HDMI
- 24" widescreen LCD monitor, 16:10 aspect ratio, 1920 x 1200 WUXGA, 300 cd/m<sup>2</sup>, active matrix, 1000:1 (static), 8ms, 24-bit color (16.7 million colors), VGA/DVI input, 2 USB ports
- 16x CD/DVD +/- RW drive
- 1GB PCIe video card
- PCIe sound card
- Gigabit Ethernet

SATA oznacza *serial advanced technology attachment – standard, który opisuje jak twardy dysk wymienia dane z innymi elementami systemu*

Ale w tym systemie występują również inne standardy wymiany informacji pomiędzy urządzeniami:

PCI

PCIe

USB

Serial port

Ethernet (Gigabitowy)

HDMI

## 1.3 Przykładowy system (14)

- W pozostałej części wykładów zobaczymy jak te komponenty działają i jak wchodzą w interakcję z oprogramowaniem w celu stworzenia kompletnego systemu komputerowego
- Powyższe stwierdzenie podnosi dwa ważne pytania:
  - Jaką pewność mamy, że komponenty tego komputera będą działać zgodnie z oczekiwaniami?
  - Jaką pewność mamy komponenty tego komputera będą działać razem?

## 1.4 Organizacje standaryzacyjne (1 z 4)

- Istnieje wiele organizacji, które ustawiają standardy dla sprzętu komputerowego, które uwzględniają interoperacyjność składników komputera
- Każdy z nas w swojej karierze ma szansę spotkania z wieloma z nich
- Do najważniejszych grup ustanawiających standardy należą:

## 1.4 Organizacje standaryzacyjne (2 z 4)

- Instytut Inżynierów Elektryków i Elektroników - The Institute of Electrical and Electronic Engineers (IEEE)
- Ustanawia (między innymi!) standardy dla
  - komponentów komputerów
  - reprezentacji danych
  - protokołów sygnalizacyjnych...

## 1.4 Organizacje standaryzacyjne (3 z 4)

- Międzynarodowy Związek Telekomunikacyjny (*The International Telecommunications Union, ITU*)
  - Zajmuje się interoperacyjnością systemów telekomunikacyjnych, w tym transmisji danych i telefonii
- Grupy krajowe ustalają standardy w ramach ich poszczególnych państw:
  - Amerykański Narodowy Instytut Norm (ANSI) - *The American National Standards Institute*
- Brytyjska Instytucja Normalizacyjna (BSI) - *The British Standards Institution*

## 1.4 Organizacje standaryzacyjne (4 z 4)

- Międzynarodowa Organizacja ds. Standaryzacji (ISO) - *The International Organization for Standardization*
  - ustanawia światowe standardy dla wszystkiego - od gwintów do klisz fotograficznych
  - ma wpływ na formułowanie standardów dla sprzętu i oprogramowania komputerowego, w tym na ich metody wytwarzania.

## 1.5 Rozwój historyczny (1 z 10)

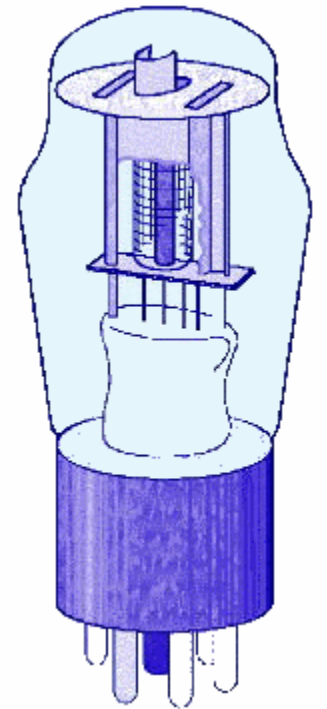
- Aby w pełni zrozumieć dzisiejsze komputery, warto znać drogę jaką przeszły maszyny obliczeniowe, w jaki sposób to wszystko się rozwinęło
- Ewolucja maszyn obliczeniowych miała miejsce na przestrzeni kilku stuleci.
- W dzisiejszych czasach ewolucja komputerów jest zwykle klasyfikowana jako cztery generacje według najważniejszej technologii epoki
- Zauważamy również, że wiele z dat ma charakter przybliżony przybliżony.

## 1.5 Rozwój historyczny (2 z 10)

- **Generacja Zero: mechaniczne maszyny liczące (1642 1945)**
  - Zegar obliczeniowy: Wilhelm Schickard (1592 1635) – pierwszy na świecie mechaniczny kalkulator
  - Koła Pascala: Blaise Pascal (1623 1662)
  - Maszyna różnicowa: Charles Babbage (1791 1871)
  - Maszyny do tabulacji kart perforowanych: Herman Hollerith (1860 1929)
- Karty perforowane były powszechnie używane w komputerach jeszcze w latach 70 XXw.

# 1.5 Rozwój historyczny (3 z 10)

- **Pierwsza generacja: Komputery na lampach próżniowych (1945 1953)**
  - komputer ABC - Atanasoff Berry Komputer (1937 1938)
    - do rozwiązywania układów równań liniowych
  - John Atanasoff i Clifford Berry z Uniwersytetu Stanowego Iowa



## 1.5 Rozwój historyczny (4 z 10)

- **Pierwsza generacja: Komputery na lampach próżniowych (1945-1953)**
  - **Electronic Numerical Integrator and Computer (ENIAC)**
  - John Mauchly and J. Presper Eckert
  - University of Pennsylvania, 1946
- ENIAC był pierwszym komputerem ogólnego zastosowania

## 1.5 Rozwój historyczny (5 z 10)

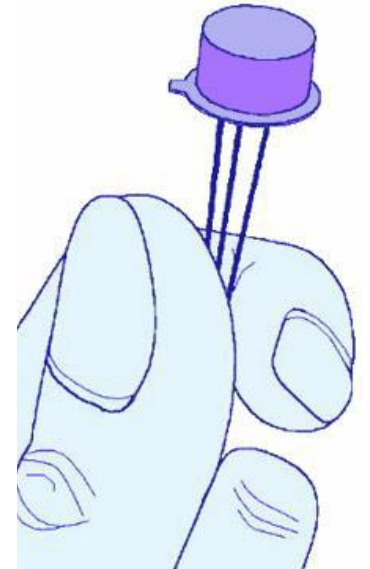
- **Pierwsza generacja: Komputery na lampach próżniowych (1945-1953)**
  - IBM 650 pierwszy masowo produkowany komputer (1955).
  - Wyszedł z użycia w 1969
- **Inne ważne firmy komputerowe tego okresu:**
  - UNIVAC
  - Engineering Research Associates (ERA)
  - Computer Research Corporation

# 1.5 Rozwój historyczny (6 z 10)

- **Druga generacja: komputery tranzystorowe (1954-1965)**

- IBM 7094 (scientific) oraz 1401 (business)
- Digital Equipment Corporation (DEC) PDP 1
- Univac 1100
- Control Data Corporation 1604
- . . . i wiele innych

- **Wszystkie te systemy miały kilka podobieństw jeżeli chodzi o ich architektury**



# 1.5 Rozwój historyczny (7 z 10)

- Trzecia generacja: Komputery zbudowane z układów scalonych (1965-1980)
  - IBM 360
  - DEC PDP 8 and PDP 11
  - Cray 1 supercomputer
  - ... i wiele innych
- W tym czasie IBM uzyskał dominującą pozycję w przemyśle komputerowym
  - W tamtym czasie rynek komputerowy charakteryzowano jako IBM i BUNCH - (Burroughs, Unisys, NCR, Control Data, and Honeywell)
  - BUNCH – ang. pęczek, garść ...

## 1.5 Rozwój historyczny (8 z 10)

- **Czwarta generacja: komputery oparte na układach o dużym stopniu scalenia (1980 - ????)**
  - *Very large scale integrated circuits (VLSI)* – mają więcej niż 10 000 komponentów w pojedynczym układzie
  - Umożliwiło stworzenie mikroprocesorów
  - Pierwszym mikroprocesorem był 4-bitowy Intel 4004
  - Późniejsze wersje takie jak 8080, 8086 i 8088 pozwoliły „narodzić się” idei komputerów osobistych (*personal computing*)

# 1.5 Rozwój historyczny (9 z 10)

- Pytanie o ostateczną granicę postępu
- Prawo Moore'a:
  - Gordon Moore, założyciel firmy Intel
  - *"The density of transistors in an integrated circuit will double every year"*
- Wersja współczesna:
  - – "The density of silicon chips doubles every 18 months"
- To „prawo” nie może być wieczne .....

# 1.5 Rozwój historyczny (10 z 10)

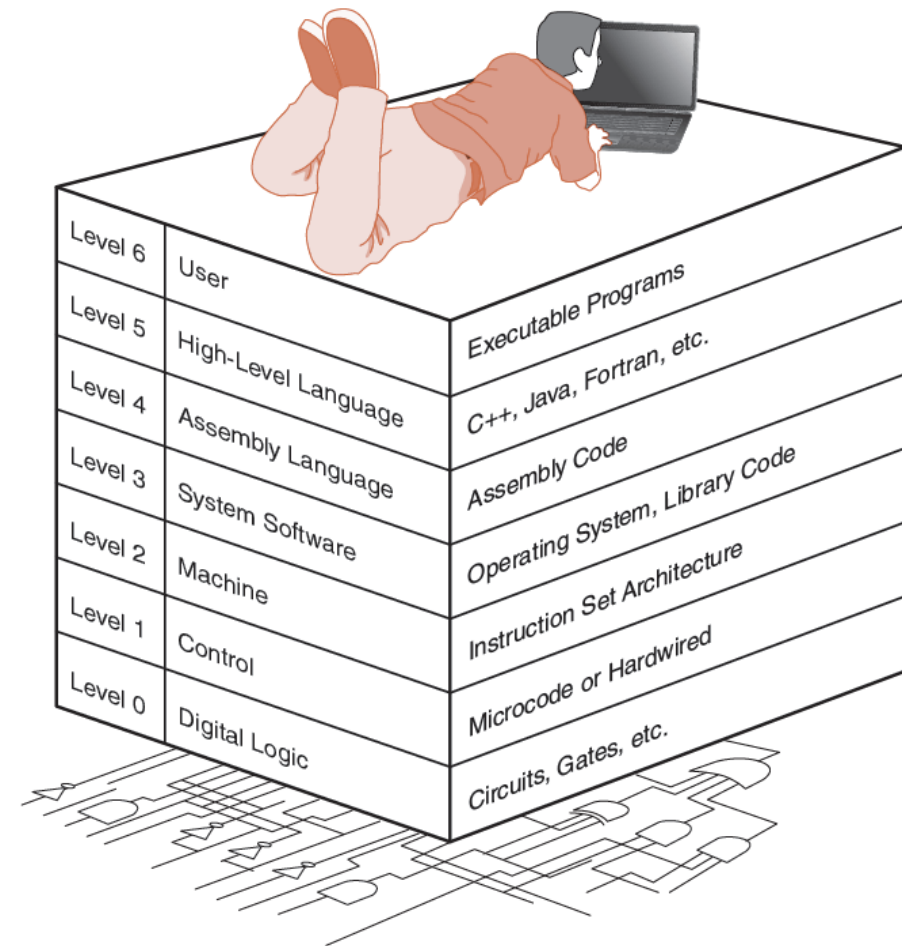
- Prawo Rock'a
  - Arthur Rock, finansista
  - “koszt sprzętu niezbędnego do produkcji półprzewodników będzie się co roku podwajał”
- W tym tempie do 2035 roku koszt pamięci byłaby mniejsza od atomu zaś koszt jej produkcji przewyższałby możliwości wszystkich państw świata razem wziętych
  - Oczywiście, aby prawo Moor'a mogło zostać w mocy, prawo Rock'a musi zostać obalone.
- Aby to się stało musimy zacząć wytwarzanie komputerów w zupełnie innej technologii

# Hierarchiczna struktura komputerów (1z 7)

- Komputery składają się jeszcze z wielu innych rzeczy oprócz chipów
- Zanim komputer będzie mógł zrobić cokolwiek wartościowego, musi także skorzystać z oprogramowania
- Pisanie skomplikowanych programów wymaga podejścia „dziel” i zwyciężaj”, gdzie każdy moduł programowy rozwiązuje jakiś mniejszy problem
- Organizacja systemów komputerowych wygląda podobnie. Na poziomie abstrakcji możemy wyobrazić sobie, że komputery składają się z pewnych hierarchicznie ułożonych warstw, które spełniają określone funkcje

# Hierarchiczna struktura komputerów (2 z 7)

- Każda warstwa maszyny wirtualnej jest abstrakcją dla poziomu niżej
- Maszyny na każdym poziomie wykonują swoje własne instrukcje, wywołując maszyny na niższych poziomach do wykonywania zadań zgodnie z wymaganiami.
- Fizyczne obwody komputerowe ostatecznie wykonują pracę.



# Hierarchiczna struktura komputerów (3 z 7)

- **Poziom 6: Poziom użytkownika**

- Aplikacje, uruchamiamy w nim programy, jak edytor tekstu
- wykonanie programu
- poziom interfejsu użytkownika
- Poziom, z którym jesteśmy najbardziej zaznajomieni

- **Poziom 5: poziom języków wysokiego poziomu**

- Poziom, z którym wchodzimy w interakcję, gdy piszemy programy w językach takich jak C, Pascal, Java....
- Języki te muszą być tłumaczone na język zrozumiały dla komputera za pomocą kompilatora lub interpretera

# Hierarchiczna struktura komputerów (4 z 7)

- **Poziom 4: Poziom asemblera**

- Działa na jednej z odmian języka asemblera „wyprodukowanym” z kodu wysokiego poziomu na Poziomie 5,
- a także wykonuje instrukcje zaprogramowane bezpośrednio na tym poziomie.
- Assembler jest tłumaczony „jeden-do-jednego” na kod maszynowy , czyli jednej instrukcji asemblera odpowiada dokładnie jedna instrukcja kodu maszynowego

- **Poziom 3: Poziom oprogramowania systemowego**

- Obejmuje instrukcje systemu komputerowego
- Kontroluje wykonywanie procesów w systemie, odpowiada za wieloprogramowość
- Chroni zasoby systemowe, synchronizacja procesów, itp.
- Instrukcje języka asemblera często przechodzą przez ten poziom bez modyfikacji.

# Hierarchiczna struktura komputerów (5 z 7)

- **Poziom 2: Poziom maszynowy**

- Znany również jako architektura zbioru rozkazów (ISA, Instruction Set Architecture)
- Składa się z instrukcji języka maszynowego, które są specyficzne dla danej architektury maszyny
- Programy napisane w języku maszynowym są wykonywane bezpośrednio przez obwody elektryczne, nie potrzebują kompilatorów, interpreterów lub translatorów

# Hierarchiczna struktura komputerów (6 z 7)

- **Poziom 1: Poziom sterowania**

- Jednostka sterująca dekoduje i wykonuje instrukcje i przenosi dane przez system.
- Jednostki sterujące mogą być mikroprogramowane lub skonfigurowane na stałe (sprzętowe);
- W przypadku sprzętowym sygnały sterujące wychodzą z bloków cyfrowych podzespołów logicznych, kierują dane i instrukcje do odpowiednich części systemu
- W przypadku programowym instrukcje sterujące mogą być w postaci mikroprogramu. Jest to program niskiego poziomu zaimplementowany bezpośrednio w urządzeniu sprzętowym

# Hierarchiczna struktura komputerów (7 z 7)

- **Poziom 0: Cyfrowy poziom logiczny**

- Na tym poziomie znajdują obwody cyfrowe (chipy).
- Obwody cyfrowe składają się z bramek i przewodów.
- Te komponenty realizują operacje logiczne i matematyczne wszystkich innych poziomów

# Model von Neumana (1 z 8)

- Na ENIAC całe programowanie było wykonywane na poziomi logiki cyfrowej
- Programowanie komputera wymagało przełączania wtyczek i przewodów
- do rozwiązania każdego unikalnego typu problemu potrzebna była inna konfiguracja sprzętowa
- Konfigurowanie ENIAC do rozwiązania „prostego” problemu wymagało wielu dni pracy wykwalifikowanych techników.

# Model von Neumana (2 z 8)

- Wynalazcy ENIAC, John Mauchley i J. Presper Eckert, wymyślili później komputer, który może przechowywać instrukcje w pamięci.
  - Ze względu na zaangażowanie w pracę dla rządu USA w czasie II wojny nie mogli opublikować swoich idei
- Jednak za wynalazcę uznano matematyka Johna von Neumanna, który był rówieśnikiem Mauchleya i Eckerta, i który opublikował ich pomysł pod swoim nazwiskiem
- Komputery przechowujące programy w swojej pamięci stały się znane jako ***Systemy von Neumanna*** oparte na ***Architekturze von Neumanna***

# Model von Neumana (3 z 8)

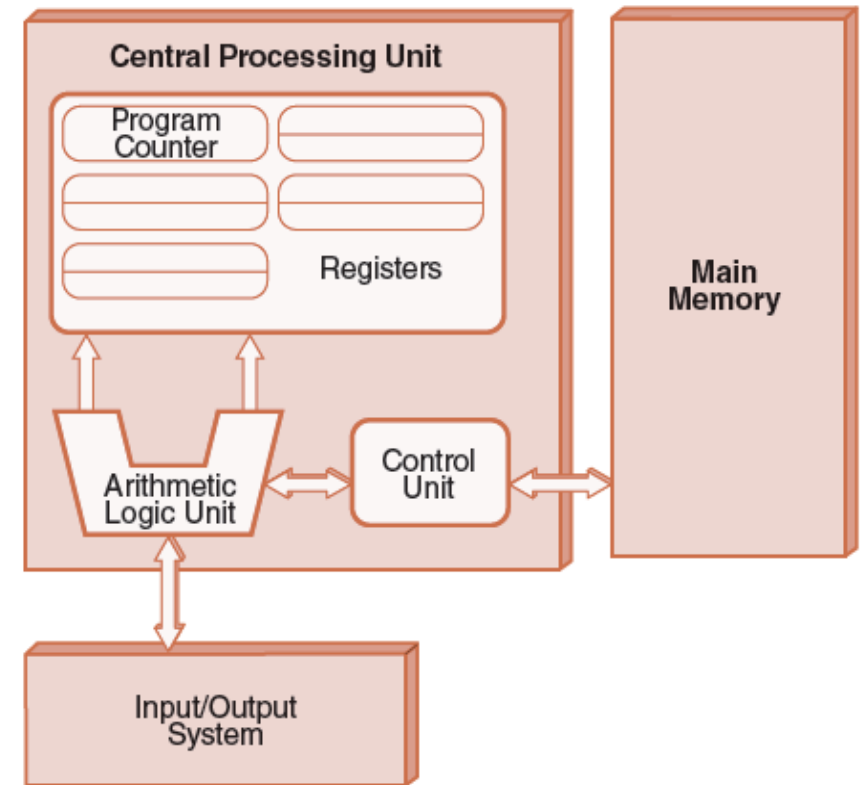
- Dzisiejsze komputery z programami zapisanymi w pamięci mają następujące cechy:
  - Trzy systemy sprzętowe:
    - Jednostka centralna (CPU)
    - Główny system pamięci
    - System we/wy
  - Zdolność do wykonywania instrukcji w sposób sekwencyjny
  - Pojedyncza ścieżka danych między procesorem a pamięcią główną

\*\* Ta pojedyncza ścieżka jest znana jako wąskie gardło von Neumanna (bottleneck)

# Model von Neumana (4 z 8)

Ogólny idea systemu von Neumanna zaprezentowana jest na rysunku :

- Komputer przesyła cały ruch we-wy przez jednostkę arytmetyczno-logiczną (ALU)
- Programy wykonywane są w tzw. *cyklu wykonawczym von Neumanna* (określanym jako „pobierz-dekoduj-wykonaj”\*)
- *Każda iteracja cyklu wygląda następująco:*

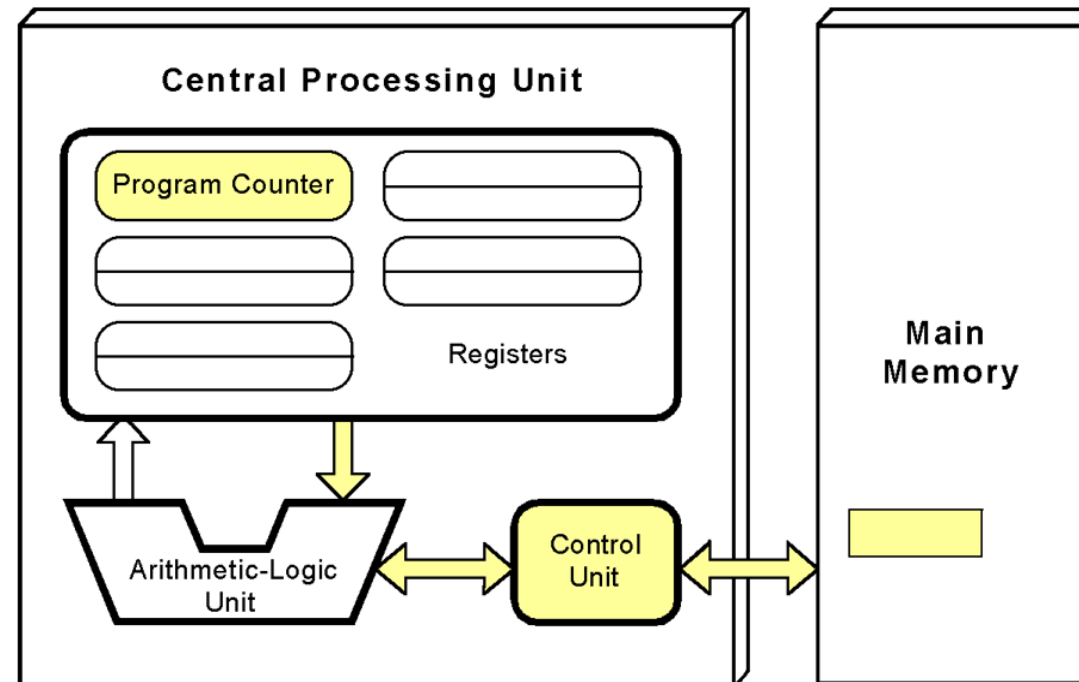


\* *fetch-decode-execute*

# Model von Neumana (5 z 8)

- *cykl wykonawczy von Neumanna:*

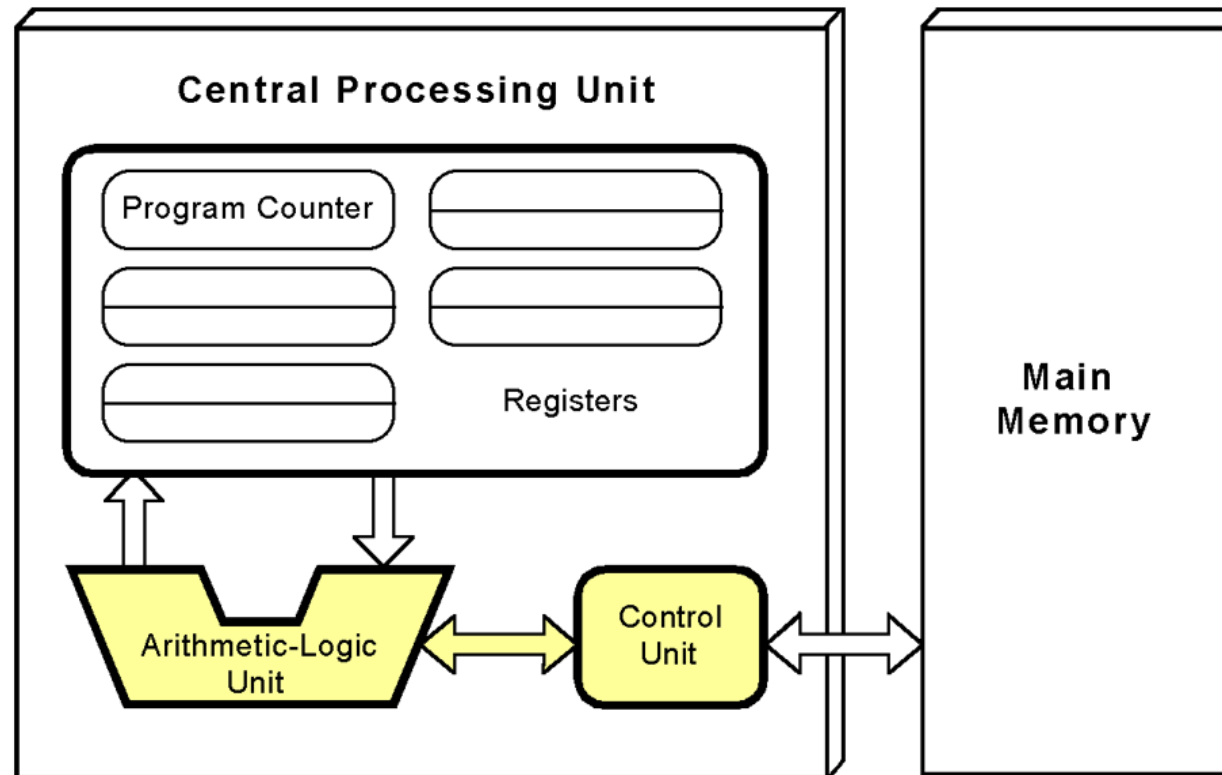
(1). Jednostka sterująca pobiera kolejną instrukcję z pamięci określając jej lokalizację za pomocą licznika instrukcji



# Model von Neumana (6 z 8)

- *cykl wykonawczy von Neumanna:*

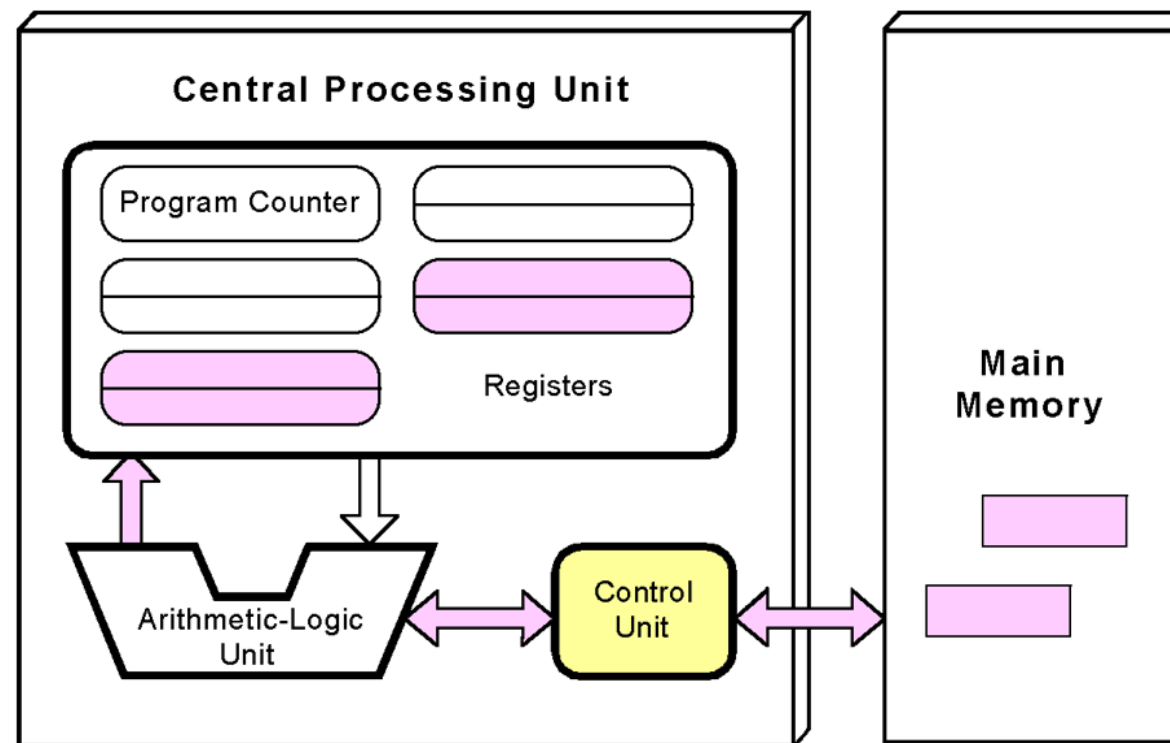
(2). Instrukcja jest dekodowana do języka zrozumiałego dla ALU



# Model von Neumana (7 z 8)

- *cykl wykonawczy von Neumanna:*

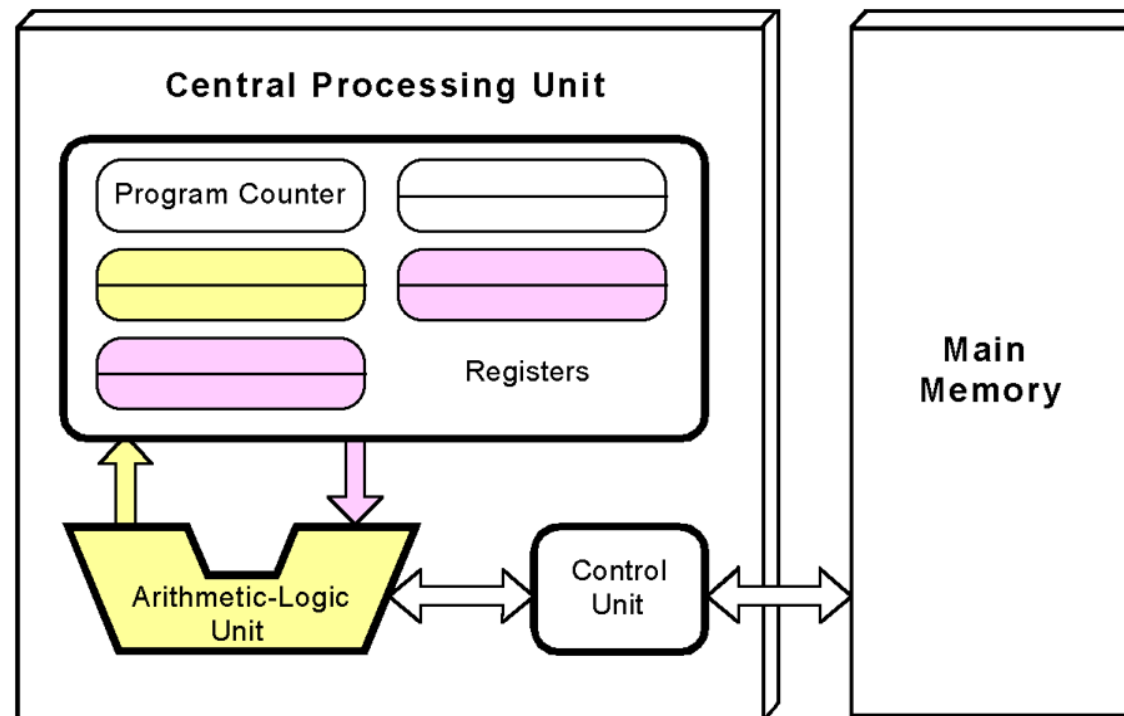
(3). z pamięci pobierane są wszelkie operandy niezbędne do wykonania operacji i zapisywane są w rejestrach w CPU



# Model von Neumana (8 z 8)

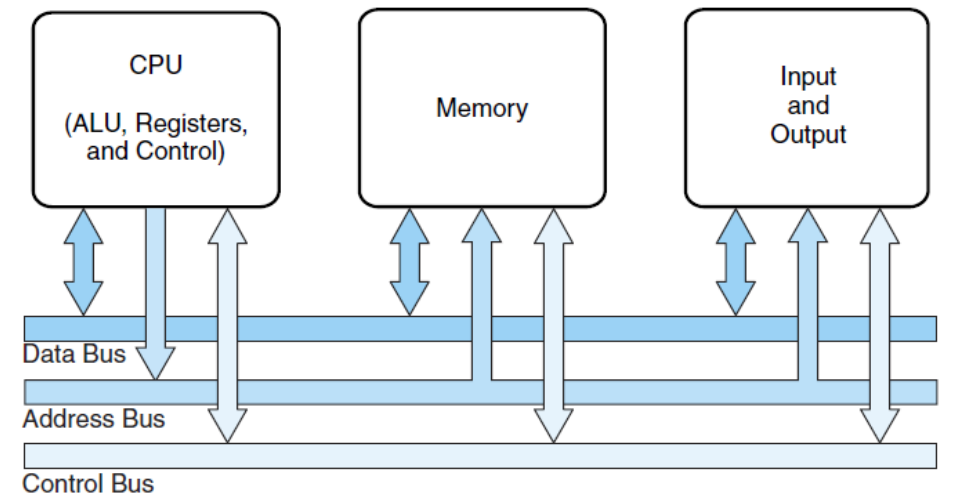
- *cykl wykonawczy von Neumanna:*

(4). ALU wykonuje instrukcję i umieszcza jej wyniki w rejestrach lub pamięci



# Inne architektury (1 z 3)

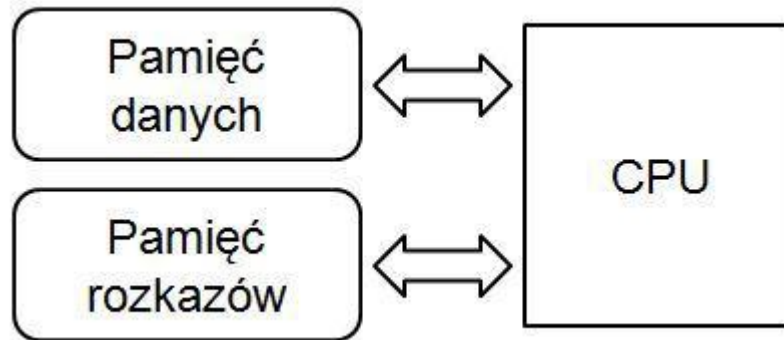
- Konwencjonalne komputery z programami przechowywanymi w pamięci przeszły przez lata wiele stopniowych ulepszeń
- Te ulepszenia obejmują dodawanie specjalizowanych szyn danych, jednostki zmiennoprzecinkowe oraz pamięci podręczne, ....
- Ale radykalne zwiększenie mocy obliczeniowej wymaga odejścia od klasycznej architektury von Neumanna



**The Modified von Neumann Architecture, Adding a System Bus**

# Inne architektury (2 z 3)

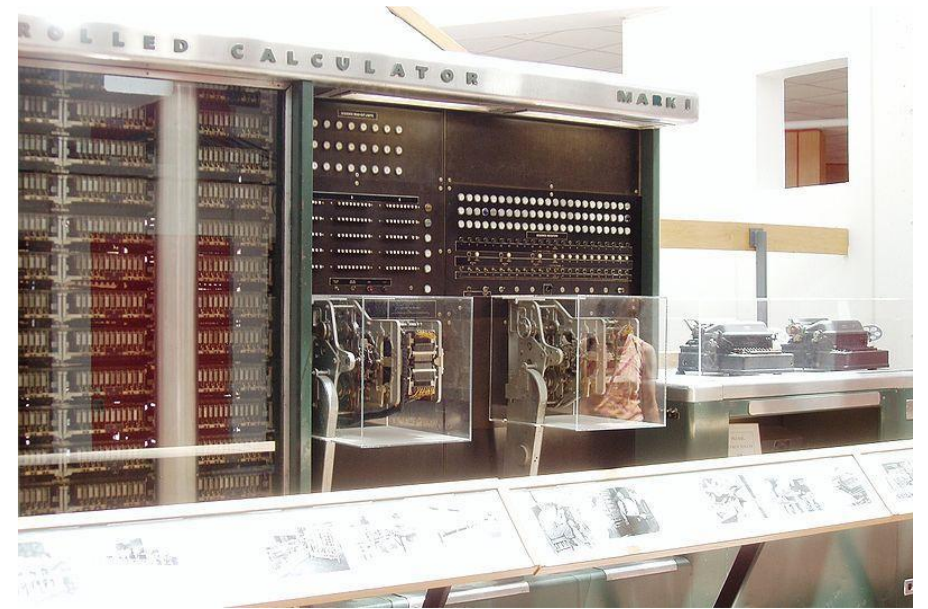
- Niektóre z dzisiejszych systemów mają osobne szyny dla danych i instrukcji
- Nazywane są *Architekturą Harwardzką*:



# Inne architektury (3 z 3)

- Komputer (model Harwardzki!) taki jest, mówiąc najprościej, komputerem zaprojektowanym do wykonywania jednego programu.
- Program ten zapisany jest na stałe w pamięci RAM.
- **Zaletą tego rozwiązania jest duża wydajność**
- W technologii tej wykonywane są obecnie procesory sygnału (DSP) wykorzystywane między innymi w kamerach i aparatach cyfrowych oraz mikrokontrolery jednokładowe, obecne między innymi w nowoczesnych samochodach.

- **CIEKAWOSTKA** Najstarsze komputery (komputery generacji zerowej i pierwsze komputery generacji pierwszej) zaprojektowane były w oparciu o model harwardzki. Algorytm przetwarzania danych, według którego pracowały, wynikał bezpośrednio z ich budowy. Zmiana programu wymagała więc fizycznego przebudowania komputera.



# Literatura

- Linda Null, Julia Lobur, „Struktura i architektura systemów komputerowych, Helion 2004
- Metzger Piotr - Anatomia PC, wydanie XI, Helion 2007
- Wojtuszkiewicz Krzysztof - Urządzenia techniki komputerowej, część I: Jak działa komputer, MIKOM, Warszawa 2000
- Wojtuszkiewicz Krzysztof - Urządzenia techniki komputerowej, część II: Urządzenia peryferyjne i interfejsy, MIKOM, Warszawa 2000
- Komorowski Witold - Krótki kurs architektury i organizacji komputerów, MIKOM Warszawa 2004
- Gook Michael - Interfejsy sprzętowe komputerów PC, Helion, 2005

# Architektura Systemów Komputerowych

Reprezentacja danych w systemach komputerowych

Dr inż. Krzysztof Kosmowski

# Plan Wykładu

- Pozycyjne systemy liczbowe
- Kodowanie liczb całkowitych (ze znakiem i bez)
- Reprezentacja zmiennoprzecinkowa
- Kodowanie znaków

# CELE WYKŁADU

- Zrozumienie podstaw reprezentacji liczb i manipulacji danymi w komputerach
- Opanowanie umiejętności konwertowania między systemami o różnych podstawach
- Zrozumienie podstaw reprezentacji zmiennoprzecinkowej
- Świadomość możliwych błędów w obliczeniach z powodu przepełnienia i obciążenia
- Poznanie najpopularniejszych kodów znaków

# Wprowadzenie (1 z 2)

- **Bit** jest najbardziej podstawową jednostką informacji w komputerze
  - Jest to stan „włączony” lub „wyłączony” w obwodzie cyfrowym
  - Czasami stany te to „wysokie” lub „niskie” napięcie zamiast „włączony” lub „wyłączony”
  - Jego nazwa to skrót od ang. *binary digit*
- **Bajt to grupa ośmiu bitów**
  - Bajt jest najmniejszą możliwą adresowalną jednostką przechowywaną w pamięci komputera
  - Termin „adresowalny” oznacza, że określony bajt można pobrać zgodnie z jego lokalizacją w pamięci.

# Wprowadzenie (2 z 2)

- **Word (Słowo) to ciągła grupa bajtów**
  - *Word* może być dowolną liczbą bitów lub bajtów
  - Najbardziej popularne są „słowa” o rozmiarach 16, 32 lub 64 bitów
- W systemie adresowalnym za pomocą „Słów” - Word jest najmniejszą adresowalną jednostką pamięci

# Pozycyjne systemy liczbowe (1 z 3)

- Bajty przechowują liczby przy użyciu pozycji każdy bit reprezentuje potęgę 2
  - System binarny nazywany jest również systemem z bazą 2
- Nasz system dziesiętny to system o podstawie 10.
  - Używa on potęgi 10 dla każdej pozycji dla reprezentacji danej liczby.
- Dowolną liczbę całkowitą można przedstawić w sposób dokładny używając systemu o dowolnej podstawie

# Pozycyjne systemy liczbowe (2 z 3)

- Liczba dziesiętna 947 w potęgach 10 to:

$$9 \times 10^2 + 4 \times 10^1 + 7 \times 10^0$$

- Liczba dziesiętna 5836,47 w potęgach 10 to:

$$5 \times 10^3 + 8 \times 10^2 + 3 \times 10^1 + 6 \times 10^0 \\ + 4 \times 10^{-1} + 7 \times 10^{-2}$$

# Pozycyjne systemy liczbowe (3 z 3)

- Liczba binarna 11001 w potęgach 2 to:

$$\begin{aligned} & 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ & = 16 + 8 + 0 + 0 + 1 = 25 \end{aligned}$$

- Kiedy podstawa liczby jest inna niż 10, jest oznaczona indeksem dolnym
  - Czasami dolny indeks 10 jest dodawany dla podkreślenia:

$$11001_2 = 25_{10}$$

# Konwersja między systemami o różnych podstawach (1 z 18)

- Na wcześniejszym slajdzie powiedziano, że każda liczba całkowita może być przedstawiona za pomocą systemu o dowolnej podstawie
- Istnieją dwie metody konwersji podstawy:
  - metoda odejmowania
  - metoda dzielenia z resztą
- Metoda odejmowania jest bardziej intuicyjna, ale nieporęczna.

# Konwersja między systemami o różnych podstawach (2 z 18)

- Załóżmy, że chcemy przeliczyć **liczbę dziesiętną 190** do bazy 3:
  - Wiemy, że  $3^5 = 243$ , więc wynik będzie mniejszy niż sześć cyfr.
  - Zatem największa potęga 3, której potrzebujemy, to  $3^4 = 81$ , a  $81 \times 2 = 162$ .
  - Zapisz 2 i odejmij 162 od 190, co daje 28.

$$\begin{array}{r} 190 \\ - 162 \\ \hline 28 \end{array} = 3^4 \times 2$$

# Konwersja między systemami o różnych podstawach (3 z 18)

- Konwersja 190 do podstawy 3:
  - Następna potęga 3 to  $3^3 = 27$ . Będziemy potrzebować jedynki na tej pozycji, więc odejmujemy 27 i zapisujemy numer 1 w naszym wyniku
  - Następna potęga 3,  $3^2 = 9$ , jest za dużo, ale my musimy przypisać symbol zastępczy zero i przenieść jedynkę w dół

$$\begin{array}{r} 190 \\ - 162 \\ \hline 28 \end{array} = 3^4 \times 2$$
$$\begin{array}{r} - 27 \\ \hline 1 \end{array} = 3^3 \times 1$$
$$\begin{array}{r} - 0 \\ \hline 1 \end{array} = 3^2 \times 0$$

# Konwersja między systemami o różnych podstawach (4 z 18)

- Konwersja 190 do podstawy 3:
  - $3^1 = 3$  jest znowu za dużo, więc przypisujemy zero jako symbol zastępczy
  - Ostatnia potęga 3,  $3^0 = 1$ , to nasz ostatni wybór - to ostatecznie daje nam różnicę **zero**.
- Nasz wynik, czytany z **od góry do dołu to:**

$$190_{10} = 21001_3$$

The image shows a handwritten conversion of the decimal number 190 to base 3. It consists of five subtraction steps, each followed by an equation showing the quotient and remainder. A yellow vertical highlight is placed to the right of the remainders, which read from top to bottom: 2, 1, 0, 0, 1. These remainders form the base-3 number 21001.

$$\begin{array}{r} 190 \\ - 162 \\ \hline 28 \\ - 27 \\ \hline 1 \\ - 0 \\ \hline 1 \\ - 0 \\ \hline 1 \\ - 1 \\ \hline 0 \end{array} \begin{array}{l} = 3^4 \times 2 \\ = 3^3 \times 1 \\ = 3^2 \times 0 \\ = 3^1 \times 0 \\ = 3^0 \times 1 \end{array}$$

# Konwersja między systemami o różnych podstawach (5 z 18)

- Inna metoda konwersji liczb całkowitych z systemu dziesiętnego do jakiejś innej podstawy wykorzystuje dzielenie z resztą
- Ta metoda jest mechaniczna i łatwa
- Wykorzystuje ideę, że sukcesywny podział przez podstawę jest równoznaczny z kolejnym odejmowaniem przez kolejne potęgi bazy
- Użyjmy metody dzielenia z resztą, aby ponownie przekształć 190 w postaci dziesiętnej na podstawę 3.

# Konwersja między systemami o różnych podstawach (6 z 18)

- Konwersja 190 do podstawy 3:
  - Najpierw bierzemy liczbę systemu, którą chcemy skonwertować i podzielimy przez podstawę systemu, którym chcemy wyrazić nasz wynik
  - W tym przypadku 3 dzieli 190 63 razy, z resztą 1
  - Zapisz iloraz i resztę:

$$\begin{array}{r} 3 \overline{) 190} \quad 1 \\ \underline{63} \end{array}$$

# Konwersja między systemami o różnych podstawach (7 z 18)

- Konwersja 190 do podstawy 3
  - 63 jest równo podzielne przez 3
  - Nasza reszta to zero a iloraz wynosi 21

$$\begin{array}{r} 3 \overline{) 190} \quad 1 \\ \underline{3 \phantom{0} 63} \phantom{0} \\ 21 \end{array}$$

# Konwersja między systemami o różnych podstawach (8 z 18)

- Konwersja 190 do podstawy 3:
  - Kontynuuj w ten sposób, aż iloraz wyniesie zero.
  - W ostatecznym rachunku zauważamy, że 3 dzieli 2 zero razy a reszta wynosi 2
- Nasz wynik, czytanie **z dołu do góry** to:

$$190_{10} = 21001_3$$

$$\begin{array}{r|l} 3 & 190 & 1 \\ 3 & 63 & 0 \\ 3 & 21 & 0 \\ 3 & 7 & 1 \\ 3 & 2 & 2 \\ & 0 & \end{array}$$

# Konwersja między systemami o różnych podstawach (9 z 18)

- Wartości ułamkowe można aproksymować w systemach o dowolnych podstawach
- W przeciwieństwie do liczb całkowitych, ułamki niekoniecznie będą mieć dokładne reprezentacje we wszystkich systemach liczbowych
- liczba  $\frac{1}{2}$  jest dokładnie reprezentowana w systemie binarnym i dziesiętnym, ale nie jest w trójkowym (o podstawie 3)

# Konwersja między systemami o różnych podstawach (10 z 18)

- Ułamkowe wartości dziesiętne mają cyfry różne od zera po prawej stronie przecinka dziesiętnego
- Wartości ułamkowe innych systemów o innych podstawach mają niezerowe cyfry na prawo od punktu podstawy.
- Liczby na prawo od punktu podstawy reprezentują ujemne potęgi podstawy:

$$0.47_{10} = 4 \times 10^{-1} + 7 \times 10^{-2}$$

$$0.11_2 = 1 \times 2^{-1} + 1 \times 2^{-2}$$

$$= \frac{1}{2} + \frac{1}{4}$$

$$= 0.5 + 0.25 = 0.75$$

# Konwersja między systemami o różnych podstawach (11 z 18)

- Podobnie jak w przypadku konwersji liczb całkowitych, można użyć jednej z dwóch metod:
  - metody odejmowania
  - metody mnożenia.
- Metoda odejmowania ułamków jest identyczna z metodą odejmowania liczb całkowitych.
  - Zamiast odejmowania dodatnich potęg podstawy celu, odejmujemy ujemne potęgi podstawy.
  - Zawsze zaczynamy od największej wartości,  $n^{-1}$ , gdzie  $n$  to nasza podstawa, używając coraz większe ujemne wykładniki.

# Konwersja między systemami o różnych podstawach (12 z 18)

- Obliczenie po prawej jest przykładem użycia metody konwersji za pomocą odejmowania dziesiętnej liczby 0,8125 na binarną
- Wynik, czytany **od góry do dołu** to:  
 $0,8125_{10} = 0,1101_2$
- Oczywiście ta metoda działa z każdą bazą, nie tylko binarnie.

$$\begin{array}{rcl} 0.8125 & & \\ - 0.5000 & = 2^{-1} \times 1 & \\ \hline 0.3125 & & \\ - 0.2500 & = 2^{-2} \times 1 & \\ \hline 0.0625 & & \\ - 0 & = 2^{-3} \times 0 & \\ \hline 0.0625 & & \\ - 0.0625 & = 2^{-4} \times 1 & \\ \hline 0 & & \end{array}$$

# Konwersja między systemami o różnych podstawach (13 z 18)

- Używając metody mnożenia przekonwertować ułamek dziesiętny 0,8125 do binarnego:

- Pomnóż przez podstawę 2
- Pierwszy produkt przenosimy na miejsce jednostek.

$$\begin{array}{r} .8125 \\ \times \quad 2 \\ \hline 1.6250 \end{array}$$

# Konwersja między systemami o różnych podstawach (14 z 18)

- Konwersja 0,8125 do systemu dwójkowego
  - Ignoruj wartości na pozycji jedności w każdym kroku,
  - kontynuuj mnożąc za każdym razem część ułamkową przez podstawę

$$\begin{array}{r} .8125 \\ \times \quad 2 \\ \hline 1.6250 \end{array}$$

$$\begin{array}{r} .6250 \\ \times \quad 2 \\ \hline 1.2500 \end{array}$$

$$\begin{array}{r} .2500 \\ \times \quad 2 \\ \hline 0.5000 \end{array}$$

# Konwersja między systemami o różnych podstawach (15 z 18)

- Konwersja 0,8125 do systemu dwójkowego
  - Koniec następuje, gdy iloczyn wynosi zero lub do czasu, gdy osiągnięto pożądaną liczbę miejsc binarnych.
  - Wynik, czytany **od góry do dołu** to:

$$0,8125_{10} = 0,1101_2$$

$$\begin{array}{r} .8125 \\ \times 2 \\ \hline 1.6250 \\ \\ .6250 \\ \times 2 \\ \hline 1.2500 \\ \\ .2500 \\ \times 2 \\ \hline 0.5000 \\ \\ .5000 \\ \times 2 \\ \hline 1.0000 \end{array}$$

# Konwersja między systemami o różnych podstawach (16 z 18)

- Dla komputerów najważniejszy jest binarny system liczbowy, czyli system o podstawie 2
- Jednak trudno jest odczytać długie ciągi binarne, nawet niezbyt duża liczba dziesiętna staje się bardzo długą liczbą binarną.
  - Na przykład:  $11010100011011_2 = 13595_{10}$
- Dla zwartości i łatwości odczytu wartości binarne są zwykle wyrażane za pomocą systemu szesnastkowego (o podstawie 16)

# Konwersja między systemami o różnych podstawach (17 z 18)

- Szesnastkowy system liczbowy wykorzystuje cyfry od 0 do 9 oraz litery od A do F.
  - Liczba dziesiętna 12 to  $C_{16}$
  - Liczba dziesiętna 26 to  $1A_{16}$
- Łatwo jest przekonwertować z bazy 16 na bazę 2, ponieważ  $16 = 2^4$
- Tak więc, aby przekonwertować z binarnego na szesnastkowy, wszyscy pogrupować cyfry binarne w grupy po cztery

# Konwersja między systemami o różnych podstawach (18 z 18)

- Używając grup heksetów (czwórek bitów), liczba binarna
- $11010100011011_2 (= 13595_{10})$  w systemie szesnastkowym

|      |      |      |      |
|------|------|------|------|
| 0011 | 0101 | 0001 | 1011 |
| 3    | 5    | 1    | B    |

Jeśli liczba bitów nie jest wielokrotnością 4 to należy uzupełnić zerami po lewej stronie

- Liczby ósemkowe (podstawa 8) są wyprowadzane z systemu binarnego przez grupowanie trzech bitów  $8 = 2^3$

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 011 | 010 | 100 | 011 | 011 |
| 3   | 2   | 4   | 3   | 3   |

- System ten był bardzo przydatny, gdy komputery używały sześciu bitowych słów

# Reprezentacja liczb całkowitych ze znakiem (1 z 28)

- Istnieją trzy sposoby, w jakie binarne liczby całkowite ze znakiem mogą być wyrażone:
  - Kod prosty ( Znak-Moduł)
  - Dopełnienie do 1 (U1)
  - Dopełnienie do 2 (U2)
- W 8-bitowym słowie w notacji Znak-Moduł ze znakiem wielkości umieszcza się wartość bezwzględną liczby zapisana na 7 bitach po prawej stronie znak bitu

# Reprezentacja liczb całkowitych ze znakiem (2 z 28)

- Na przykład w 8-bitowej reprezentacji liczby ze znakiem zapisanej w kodzie Znak-Moduł:
- +3 to: **00000011**
- -3 to: **10000011**
- Komputery wykonują operacje arytmetyczne na liczbach ze znakiem w podobny sposób jak ludzie wykonują arytmetykę za pomocą ołówka i papieru.
- Ludzie często ignorują znaki operandów podczas wykonywania obliczeń zaś odpowiedni znaku jest ustalany po zakończeniu obliczeń.

# Reprezentacja liczb całkowitych ze znakiem (3 z 28)

- Dodawanie binarne jest tak proste, jak to tylko możliwe.
- Trzeba znać tylko cztery zasady:
  - $0 + 0 = 0$
  - $0 + 1 = 1$
  - $1 + 0 = 1$
  - $1 + 1 = 10$
- Prostota tego systemu sprawia, że działania arytmetyczne możliwe są do wykonania przez układy cyfrowe

# Reprezentacja liczb całkowitych ze znakiem (4 z 28)

Przykład:

- Używając reprezentacji Znak-Moduł i zasad arytmetyki znajdź sumę liczb 75 i 46.
- Najpierw przelicz 75 i 46 do systemu binarnego (reprezentacji Z-M), i zapisz osobno(dodatnie) bity znaku oraz bity wielkości.

$$\begin{array}{r} 0 \quad 1001011 \\ 0 + 0101110 \\ \hline \end{array}$$

# Reprezentacja liczb całkowitych ze znakiem (5 z 28)

- Tak jak w arytmetyce dziesiętnej znajdujemy sumę zaczynając od prawej strony i kontynuując do lewej.

$$\begin{array}{r} 0 \quad 1001011 \\ 0 + 0101110 \\ \hline 1 \end{array}$$

# Reprezentacja liczb całkowitych ze znakiem (6 z 28)

- W drugim bicie mamy przeniesienie, więc zaznaczmy to nad trzecim bitem

$$\begin{array}{r} \phantom{0}\phantom{+}\phantom{0}1\phantom{00}1\overset{1}{0}11 \\ 0\phantom{+}\phantom{0}101110 \\ \hline \phantom{0}\phantom{+}\phantom{0}\phantom{0}\phantom{0}\phantom{0}01 \end{array}$$

- Podobnie kolejne dwa bity

[illegible]

# Reprezentacja liczb całkowitych ze znakiem (7 z 28)

- Kiedy już mamy „przepracowaną” całą ósemkę bitów - skończyliśmy.

$$\begin{array}{r} \phantom{0}\phantom{+}\phantom{\overline{}}\phantom{0}\phantom{1}\phantom{1}\phantom{1}\phantom{0}\phantom{0}\phantom{1} \\ 0\phantom{+}\phantom{\overline{}}\phantom{0}\phantom{1}\phantom{0}\phantom{1}\phantom{1}\phantom{1}\phantom{0} \\ 0 + 0101110 \\ \hline 0\phantom{+}\phantom{\overline{}}\phantom{0}\phantom{1}\phantom{0}\phantom{1}\phantom{1}\phantom{0}\phantom{1} \end{array}$$

W tym przykładzie dobrano dwie wartości, których suma daje się zapisać na siedmiu bitach. Jeśli tak nie jest mamy problem!

# Reprezentacja liczb całkowitych ze znakiem (8 z 28)

## Przykład:

- znajđ sumę liczb 107 i 46.
- Widzimy, że przeniesienie od siódmego bitu przepęlnia się (*overflow*) i jest odrzucone, dając błędny wynik:  $107 + 46 = 25$

1

$$\begin{array}{r} 00011011 \\ + 0101110 \\ \hline 00011001 \end{array}$$

# Reprezentacja liczb całkowitych ze znakiem (9 z 28)

- Liczby w reprezentacji Znak-Moduł „działają” tak jak znaki w arytmetyce ołówka i papieru.
- Przykład: za pomocą reprezentacji Z-M znajdź sumę liczb -46 i -25.
- Ponieważ znaki są takie same wszystko co robimy, to dodawanie liczb i znaku ujemnego, gdy obliczenia są skończone.

$$\begin{array}{r} \phantom{1} \phantom{0} \phantom{1} \phantom{0} \phantom{1} \phantom{1} \phantom{1} \phantom{0} \\ \phantom{1} \phantom{0} \phantom{1} \phantom{0} \phantom{1} \phantom{1} \phantom{1} \phantom{0} \\ 1 \phantom{0} \phantom{1} \phantom{0} \phantom{1} \phantom{1} \phantom{1} \phantom{0} \\ 1 + 0 \phantom{0} \phantom{1} \phantom{1} \phantom{0} \phantom{0} \phantom{1} \\ \hline 1 \phantom{0} \phantom{1} \phantom{0} \phantom{0} \phantom{1} \phantom{1} \phantom{1} \end{array}$$

# Reprezentacja liczb całkowitych ze znakiem (10 z 28)

- Działania z mieszanymi znakami: dodawanie (lub odejmowanie) odbywa się tą samą drogą.
- Przykład: za reprezentacji Z-M znajdź sumą 46 i -25.
- Znak wyniku otrzymuje znak liczby, która jest większa.
- Zwróć uwagę na „pożyczki” od drugiego i szóstego bitu.

$$\begin{array}{rcccccccc} & & & 0 & 2 & & 0 & 2 \\ 0 & & 0 & \cancel{1} & 0 & 1 & 1 & \cancel{1} & 0 \\ 1 & + & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ \hline 0 & & 0 & 0 & 1 & 0 & 1 & 0 & 1 \end{array}$$

# Reprezentacja liczb całkowitych ze znakiem (11 z 28)

- Reprezentacja Znak-Moduł jest łatwa do zrozumienia dla ludzi, ale to wymaga skomplikowanego sprzętu komputerowego.
- Inną wadą Z-M jest to, że pozwala na dwie różne reprezentacje dla zera: dodatnie zero i ujemne zero.
- Z tych powodów (m.in.) systemy komputerowe wykorzystują systemy komplementarne (z uzupełnieniem do wartości).

# Reprezentacja liczb całkowitych ze znakiem (12 z 28)

- W reprezentacji z dopełnieniem do jedności wartości ujemne reprezentowane są przez pewną różnicę między liczbą i jej podstawą
  - niezerowa liczba  $N$
  - o bazie  $r$
  - z  $d$  cyframi :

$$(r^d - 1) - N$$

- W systemie binarnym daje nam to dopełnienie.
  - W rzeczywistości to niewiele więcej niż przerzucanie wartości (*flipping*) bitów liczby binarnej.

# Reprezentacja liczb całkowitych ze znakiem (13 z 28)

- Na przykład przy użyciu 8-bitowego dopełnienia jedynki (U1) reprezentacja:
  - +3 to: **0**0000011
  - -3 to: **1**1111100
- W reprezentacji U1, podobnie jak w reprezentacji Z-M, wartości ujemne są oznaczone 1 na pozycji najstarszego bitu (MSB)
- Systemy reprezentacji z dopełnieniem są przydatne, ponieważ: eliminują potrzebę odejmowania.
- Różnicę dwóch wartości można znaleźć, dodając liczby w odpowiedniej postaci.

# Reprezentacja liczb całkowitych ze znakiem (14 z 28)

- W reprezentacji U1 bit przeniesienia (*carry bit*) jest „przenoszony na około” i dodawany do sumy
- Przykład:
  - znajdź sumę liczb 48 i -19
  - zauważmy, że 19 w systemie binarnym to:  
**00010011**
  - więc -19 w swoim dopełnieniu do jedynki to:  
**11101100**


$$\begin{array}{r} \phantom{00}11 \\ 00110000 \\ 11101100 \\ \hline 00011100 \\ \phantom{000}+1 \\ \hline 00011101 \end{array}$$

# Reprezentacja liczb całkowitych ze znakiem (15 z 28)

- Chociaż operacja „przenoszenia na około” dodaje trochę złożoności, reprezentacja U1 jest prostsza w implementacji sprzętowej niż Z-M
- Ale nadal ma tę wadę, że ma dwa różne reprezentacje dla zera:
  - zero dodatnie  
i
  - zero ujemne
- Ten problem rozwiązuje reprezentacja z uzupełnieniem do dwóch (U2)
- Uzupełnienie do dwójki jest uzupełnieniem do podstawy binarny systemu numeracji:
  - dopełnienie podstawy niezerowej liczby  $N$
  - o bazie  $r$
  - z  $d$  cyframi to:

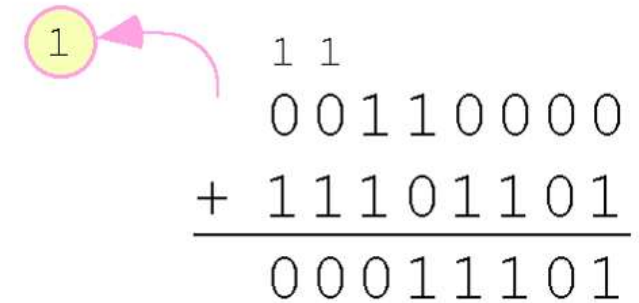
$$r^d - N$$

# Reprezentacja liczb całkowitych ze znakiem (16 z 28)

- Aby wyrazić wartość reprezentacji U2:
  - Jeśli liczba jest dodatnia, po prostu przekonwertuj ją na binarną
  - Jeśli liczba jest ujemna, znajdź dopełnienie do jedynki (U1) liczby, a następnie dodaj 1.
- Przykład:
  - 8-bitowa wersja binarna 3 to: 00000011
  - -3 w reprezentacji dopełnienia do jedynki to: 11111100
  - Dodanie 1 daje nam -3 w formie uzupełnienia do dwóch: 11111101

# Reprezentacja liczb całkowitych ze znakiem (17 z 28)

- W arytmetyce reprezentacji z dopełnieniem do dwójki, wszystko co robimy to dodawanie dwóch liczb binarnych!
- Odrzucane są wszystkie bity przeniesienia z najstarszych pozycji
- Przykład:
  - znajdź sumę liczb 48 i -19.
  - Zauważamy, że 19 w systemie binarnym to:  
00010011
  - więc -19 U1 to: **11101100**
  - czyli -19 U2 to: **11101101**


$$\begin{array}{r} 1\ 1 \\ 00110000 \\ + 11101101 \\ \hline 00011101 \end{array}$$

# Reprezentacja liczb całkowitych ze znakiem (18 z 28)

- Przykład:
  - Używając reprezentacji U2 znajdź sumę liczb 107 i 46
- Widzimy „niezerowe” przeniesienie z 7-mego bitu na bit znaku dając błędny wynik:  $107 + 46 = -103$ .
- Przeniesienie na bit znaku nie zawsze prowadzi do błędu!

$$\begin{array}{r} \overset{1}{\text{1}} \text{ 1} \quad \text{1 1 1} \\ 0 \text{ 1 1 0 1 0 1 1} \\ + 0 \text{ 0 1 0 1 1 1 0} \\ \hline 1 \text{ 0 0 1 1 0 0 1} \end{array}$$

# Reprezentacja liczb całkowitych ze znakiem

(19 z 28)

- Przykład:
  - Korzystając z reprezentacji U2, znajdź sumę liczb 23 i -9.
- Widzimy, że jest przeniesienie na bit znaku i przeniesienie „na zewnątrz”.
- Ostateczny wynik jest poprawny:  
 $23 + (-9) = 14$ .

$$\begin{array}{r} \textcircled{1} \leftarrow \textcircled{1} 1 1 \quad 1 1 1 \\ 00010111 \\ + 11110111 \\ \hline 00001110 \end{array}$$

**Reguła wykrywania błędów przepełnienia:** Jeśli wartość przeniesienia wchodzącego na bit znaku jest taka sama, jak wartość przeniesienia z niego wychodząca to nie nastąpiło przepełnienie. Jeżeli zaś przeniesienia te mają różne wartości, mamy do czynienia z przepełnieniem, a więc błędem.

# Reprezentacja liczb całkowitych ze znakiem (20 z 28)

- W systemach komputerowych przydatne są zarówno liczby ze znakiem, jak i bez znaku
  - Na przykład adresy pamięci są zawsze reprezentowane jako liczby bez znaku
- Używając tej samej liczby bitów, w przypadku liczb całkowitych bez znaku może wyrazić dwa razy więcej „dodatnich” wartości niż w przypadku reprezentacji ze znakiem
- Problem pojawia się, gdy „zawija się” wartość bez znaku:
  - Na czterech bitach:  $1111 + 1 = 0000$
  - Programiści muszą być czujni na tego rodzaju problemy

# Reprezentacja liczb całkowitych ze znakiem (21 z 28)

- Przepełnienie (*overflow*) i przeniesienie (*carry*) to „podstępne” koncepcje
- Przepełnienie liczby ze znakiem nie oznacza nic w kontekście liczb bez znaku, które ustawiają flagę przeniesienia zamiast flagi przepełnienia
- Jeśli przeniesienie skrajnego lewego bitu występuje w liczbie bez znaku, wystąpiło przepełnienie
- Przeniesienie i przepełnienie występują niezależnie od siebie wzajemnie
- Tabela na następnym slajdzie podsumowuje te idee.

# Reprezentacja liczb całkowitych ze znakiem (22 z 28)

| Expression  | Result | Carry? | Overflow? | Correct Result? |
|-------------|--------|--------|-----------|-----------------|
| 0100 + 0010 | 0110   | No     | No        | Yes             |
| 0100 + 0110 | 1010   | No     | Yes       | No              |
| 1100 + 1110 | 1010   | Yes    | No        | Yes             |
| 1100 + 1010 | 0110   | Yes    | Yes       | No              |

# Reprezentacja liczb całkowitych ze znakiem (23 z 28)

- Możemy wykonać binarne mnożenie i dzielenie przez 2 bardzo łatwo, przy użyciu operacji przesunięcia arytmetycznego
- przesunięcie arytmetyczne w lewo wstawia 0 w skrajnym prawym bicie i przesuwa wszystko w lewo o jeden bit;
  - w efekcie mnoży przez 2
- Prawe przesunięcie arytmetyczne przesuwa wszystko o jeden bit do prawej strony, przy czym kopiuje bit znaku;
  - dzieli przez 2
- Spójrzmy na kilka przykładów.

# Reprezentacja liczb całkowitych ze znakiem (24 z 28)

- Przykład:
- Pomnóż wartość 11 (wyrażoną za pomocą 8-bitowej liczby ze znakiem w reprezentacji dopełnienia do dwóch) przez 2:
- Zaczynamy od wartości binarnej dla 11: 00001011 (+11)
- Przesuwamy się w lewo o jedno miejsce, w wyniku czego:  
00010110 (+22)
- Bit znaku się nie zmienił, więc wartość jest prawidłowa.
- Aby pomnożyć 11 przez 4, po prostu dwukrotnie wykonujemy przesunięcie w lewo.

# Reprezentacja liczb całkowitych ze znakiem (25 z 28)

- Przykład:
- Podziel wartość 12 (wyrażoną za pomocą 8-bitowej liczby ze znakiem w reprezentacji dopełnienia do dwóch) przez 2:
- Zaczynamy od wartości binarnej dla 12: 00001100 (+12)
- Przesuwamy się w lewo o jedno miejsce, w wyniku czego:  
00000110 (+6)
- (Pamiętaj, że przenosimy bit znaku w lewo). Aby podzielić 12 przez 4, dwukrotnie przesuwamy w prawo

# Reprezentacja liczb całkowitych ze znakiem (26 z 28)

- Badania nad znalezieniem lepszych algorytmów dla arytmetyki binarnej trwają od ponad 50 lat
- Jeden z wielu produktów tych prac to algorytm Bootha (mnożenie)
- Ogólną ideą jest zastąpienie operacji arytmetycznych z przesunięciem bitów.
- Istnieje bardzo dużo bardziej lub mniej skomplikowanych metod mnożenia i dzielenia liczb całkowitych. Ten jest jednym z prostszych

# Reprezentacja liczb całkowitych ze znakiem (27 z 28)

Można go opisać następująco:

- **mnożnik i mnożna są umieszczane odpowiednio w rejestrach Q i M.**
- Istnieje także rejestr **l-bitowy, umieszczony logicznie na prawo od najmniej znaczącego bitu ( $Q_0$ ) rejestru Q i oznaczony  $Q_{-1}$ .**

Wyniki mnożenia będą się pojawiały w rejestrach A i Q. Rejestry A i Q są początkowo ustawiane na 0. Układy logiczne sterowania przeglądają mnożnik (Q) bit po bicie.

| A |   |   |   | Q |   |   |   | Q-1 | (M zawiera 0111) |               |  |
|---|---|---|---|---|---|---|---|-----|------------------|---------------|--|
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0   | Stan początkowy  |               |  |
| 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0   | A<-A-M           |               |  |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1   | Przesunięcie     | Pierwszy cykl |  |
| 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1   | Przesunięcie     | Drugi cykl    |  |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1   | A<-A + M         |               |  |
| 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0   | Przesunięcie     | Trzeci cykl   |  |
| 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0   | Przesunięcie     | Czwarty cykl  |  |

# Reprezentacja liczb całkowitych ze znakiem (28 z 28)

1. Podczas badania każdego bitu w rejestrze Q badany jest również bit znajdujący się w rejestrze  $Q_{-1}$ .
2. Jeśli oba bity są takie same (1-1 lub 0-0), to wszystkie bity rejestrów A, Q i  $Q_{-1}$  są przesuwane w prawo o jeden bit.
3. Jeśli te dwa bity różnią się, to mnożna (M) jest dodawana jeśli bity te są 0-1 lub odejmowana od zawartości rejestru A jeśli te bity są 1-0.
4. Po dodawaniu lub odejmowaniu następuje przesunięcie w prawo.
5. W każdym przypadku przesunięcie w prawo jest takie, że najdalej na lewo położony bit A, tzn.  $A_{n-1}$  nie tylko jest przesuwany do  $A_{n-2}$ , ale także pozostaje w  $A_{n-1}$ . Jest to wymagane dla zachowania znaku liczby w A i Q. Poniżej przykład działania dla mnożenia liczby 7 przez 3:

| A |   |   |   | Q |   |   |   | Q-1 | (M zawiera 0111)     |               |  |
|---|---|---|---|---|---|---|---|-----|----------------------|---------------|--|
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0   | Stan początkowy      |               |  |
| 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0   | $A \leftarrow A - M$ |               |  |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1   | Przesunięcie         | Pierwszy cykl |  |
| 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1   | Przesunięcie         | Drugi cykl    |  |
| 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1   | $A \leftarrow A + M$ |               |  |
| 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0   | Przesunięcie         | Trzeci cykl   |  |
| 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0   | Przesunięcie         | Czwarty cykl  |  |

# Reprezentacja zmiennoprzecinkowa (1 z 30)

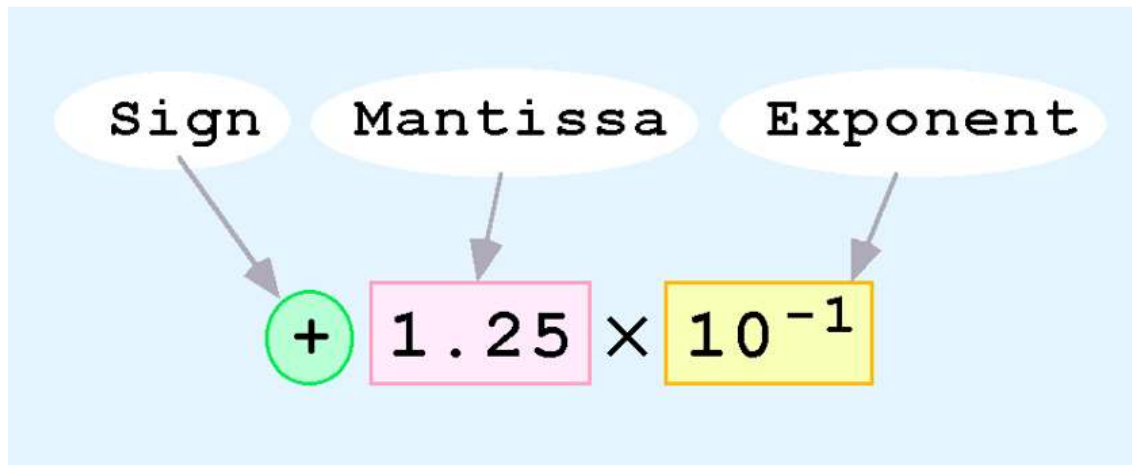
- Reprezentacja Znak-Moduł, dopełnienia do jedynki i dopełnienia do dwójki dobrze się sprawdzają przy liczbach całkowitych
- Bez modyfikacji formaty te nie są przydatne w zastosowaniach naukowych lub biznesowych, które operują na liczbach rzeczywistych
- Rozwiązaniem problemu jest reprezentacja zmiennoprzecinkowa.

# Reprezentacja zmiennoprzecinkowa (2 z 30)

- Liczby zmiennoprzecinkowe pozwalają na dowolne określenie liczby miejsc po przecinku na prawo od kropki dziesiętnej.
  - Na przykład:  $0,5 \times 0,25 = 0,125$
- Często wyrażane są w notacji naukowej,
  - na przykład:
    - $0.125 = 1.25 \times 10^{-1}$
    - $5\,000\,000 = 5.0 \times 10^6$

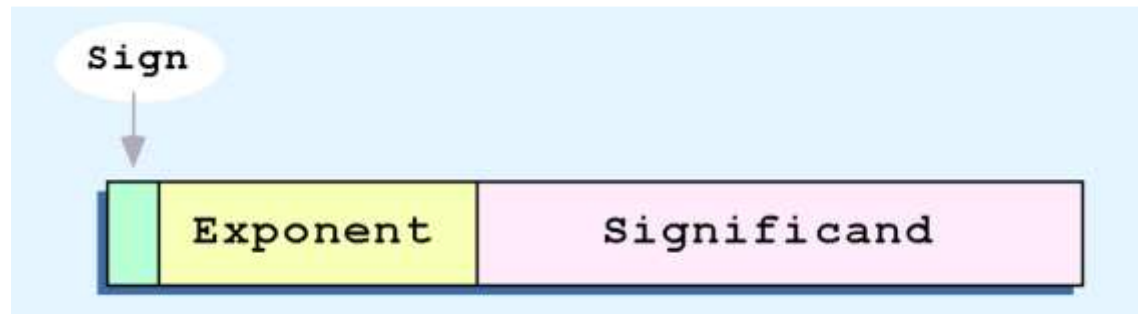
# Reprezentacja zmiennoprzecinkowa (3 z 30)

- **Komputery używają formy notacji naukowej do reprezentacji zmiennoprzecinkowej**
- Liczby zapisane w notacji naukowej mają trzy składniki:



# Reprezentacja zmiennoprzecinkowa (4 z 30)

- Komputerowa reprezentacja liczby zmiennoprzecinkowej składa się z trzech pól o stałej wielkości:



| znak | cecha /<br>wykładnik | mantysa /<br>precyzja |
|------|----------------------|-----------------------|
| 1    | 10010                | 1010110101            |

- To jest ich standardowy układ tych pól.
- Uwaga: chociaż „precyzja” i „mantysa” technicznie nie oznaczają tego samego wiele osób używa tych terminów zamiennie

# Reprezentacja zmiennoprzecinkowa (5 z 30)



5 973 600 000 000 000 000 000 000 kg  
(Masa Ziemi)

$5,9736 \cdot 10^{24}$

5,9736 E+24

**mantysa (precyzja)**

**cecha (wykładnik)**

Notacja naukowa pozwala na kodowanie  
bardzo dużych / małych liczb

# Reprezentacja zmiennoprzecinkowa (6 z 30)

- Jednabitowe pole znaku - jest znakiem przechowywanej wartości
- Wielkość pola wykładnika określa zakres wartości, które mogą być reprezentowane.
- Wielkość pola mantysy określa dokładność przedstawienia liczby.



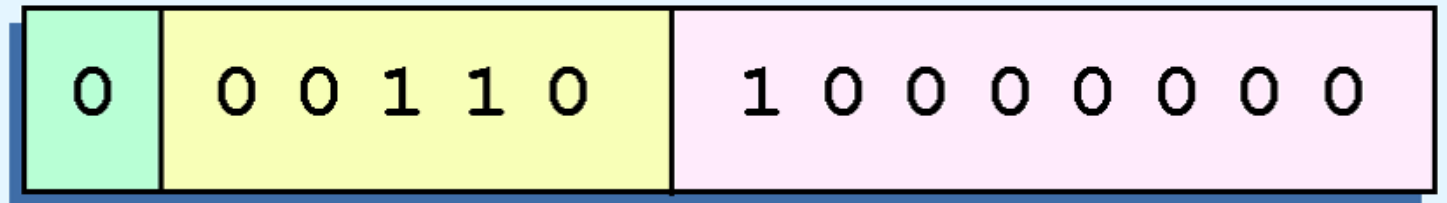
| znak | cecha /<br>wykładnik | mantysa /<br>precyzja |
|------|----------------------|-----------------------|
| 1    | 10010                | 1010110101            |

# Reprezentacja zmiennoprzecinkowa (7 z 30)

- Przedstawmy hipotetyczny „Prosty Model” dla wyjaśnienia pojęcia.
- W tym modelu:
  - Liczba zmiennoprzecinkowa ma długość 14 bitów.
  - Pole wykładnika ma 5 bitów
  - Pole mantysy ma 8 bitów.
  - Mantysa zawsze jest poprzedzona „domyślnym” przecinkiem - zatem zawsze zawiera liczbę ułamkową (zapisujemy część po przecinku)
  - Wykładnik wskazuje potęgę 2, przez którą mantysa jest mnożona

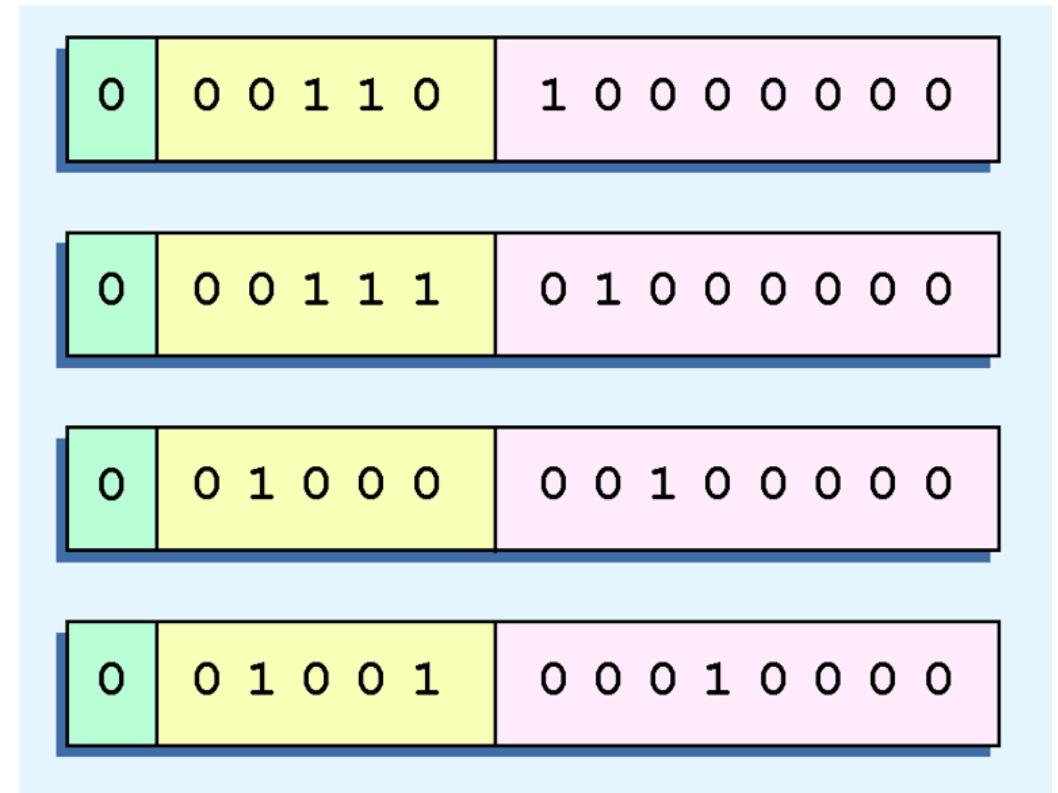
# Reprezentacja zmiennoprzecinkowa (8 z 30)

- Przykład:
- Wyrazić liczbę  $32_{10}$  w uproszczonym 14-bitowym modelu zmiennoprzecinkowym
  - Wiemy, że 32 to  $2^5$ .
  - Czyli w notacji naukowej (binarnej)  $32 = 1.0 \times 2^5 = \mathbf{0.1 \times 2^6}$
  - preferowana jest druga notacja w porównaniu z pierwszą (zostanie wyjaśnione dlaczego)
  - Korzystając z powyższych informacji, wstawiamy:
    - w polu wykładnika: **110** ( $= 6_{10}$ )
    - w polu mantysy: **1**



# Reprezentacja zmiennoprzecinkowa (9 z 30)

- Na ilustracji po prawej pokazane są różne reprezentacje liczby 32 (korzystając z modelu uproszczonego)
- Nie tylko oznacza to marnotrawstwo przestrzeni, ale może też powodować zamieszanie.



# Reprezentacja zmiennoprzecinkowa (10 z 30)

- Aby rozwiązać problem sformułowany na poprzednim slajdzie wprowadźmy regułę, że pierwsza cyfra znaczącego musi wynosić 1, bez żadnych jedynek na lewo od przecinka.
- Ten proces, zwany normalizacją, skutkuje unikatowością zapisu dla każdej liczby zmiennoprzecinkowej.
- W naszym prostym modelu wszystkie znaczące pola mantysy muszą mieć postać **0.1xxxxxxxx**
  - Na przykład:  $4.5_{(10)} = 100,1_{(2)} \times 2^0 = 1,001 \times 2^2 = 0,1001 \times 2^3$
  - Ostatnie wyrażenie jest poprawnie znormalizowane

# Reprezentacja zmiennoprzecinkowa (11 z 30)

- Pierwsze pole w reprezentacji zmiennoprzecinkowej oznacza znak przechowywanej liczby
- Jak przechowywać wartości dodatnie i ujemne dla cechy (*exponent*) ?
- Do tego celu wykorzystuje się tzw. reprezentację nadmiarową *excess-M representation*

# Reprezentacja zmiennoprzecinkowa (12 z 30)

- Jeśli  $n$  bitów jest używanych do reprezentacji całkowitych liczb binarnych, możemy wybrać próg w taki sposób, aby podzielić zasięg oferowany przez te  $n$ -bitów na dwie równe części
- Jedna z części będzie odpowiadała liczbom dodatnim, druga zaś ujemnym
- Zazwyczaj wybieramy „miejsce przegięcia”  $2^{n-1} - 1$ .
- Intuicyjnie: na 4 bitach można zapisać liczby od 0 do 15. Jeśli ustalimy że zero = 7, to uzyskujemy przestrzeń:
  - 0-6 – liczby ujemne (0000 – 0110)
  - 7 – zero (0111)
  - 8-15 – liczby dodatnie (1000-1111)

# Reprezentacja zmiennoprzecinkowa (13 z 30)

- Na przykład:
- Używając reprezentacji excess-7 liczba całkowita  $0_{10}$  jest reprezentowana jako  $0 + 7 = 7_{10} = 0111_2$
- Liczba  $3_{10}$  jest reprezentowana jako  $3 + 7 = 10_{10} = 1010_2$
- Liczba całkowita  $-7$  jest reprezentowana jako  $-7 + 7 = 0_{10} = 0000_2$
- Aby znaleźć wartość dziesiętną liczby binarnej  $1111_2$  w reprezentacji excess-7 odejmij 7 od reprezentacji dziesiętnej tej liczby:

$$1111_2 = 15_{10}, \text{ czyli } 15 - 7 = 8;$$

zatem  $1111_2$ , w excess-7 to  $+8_{10}$

# Reprezentacja zmiennoprzecinkowa (14 z 30)

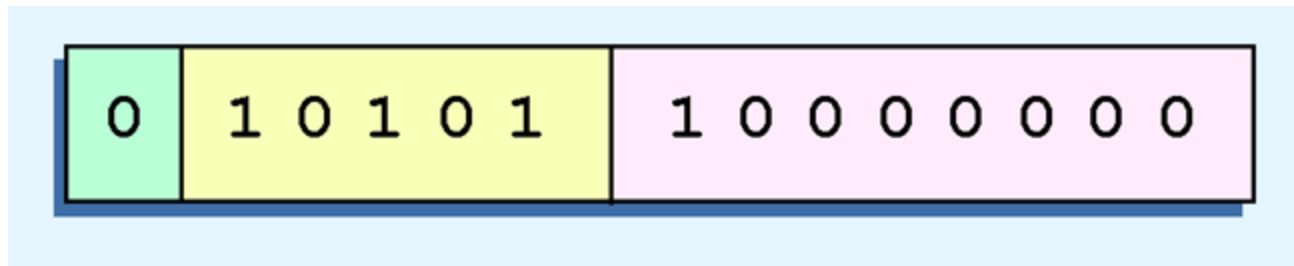
- Rozszerzmy ten przykład na nasz model 14-bitowy
  - W tym przypadku mamy 5-bitowy *Exponent*, czyli:

$$2^{5-1}-1 = 2^4 - 1 = 15$$

- Zatem 15 będziemy używać jako naszego „miejsca przegięcia”
- Nasze pole *Exponent (cecha, wykładnik)* będzie przechowywać dane w reprezentacji excess-15.

# Reprezentacja zmiennoprzecinkowa (15 z 30)

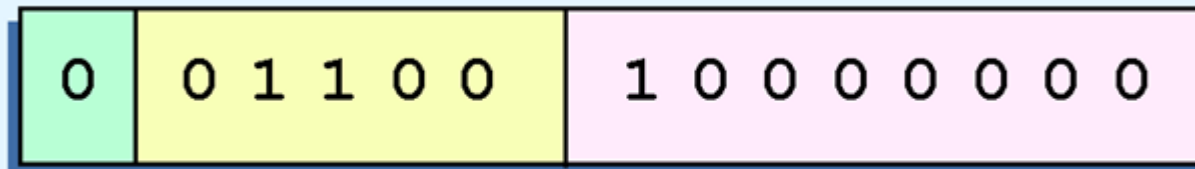
- Przykład: wyrazić  $32_{10}$  w „poprawionym” 14-bitowym modelu zmiennoprzecinkowym
- Wiemy, że  $32 = 1,0 \times 2^5 = 0,1 \times 2^6$
- Aby użyć naszej notacji nadmiarowej excess-15, dodajemy 15 do **6**, co daje  $21_{10}$  (= **10101**<sub>2</sub>)
- Więc mamy:



# Reprezentacja zmiennoprzecinkowa (16 z 30)

- Przykład: wyrazić  $0.0625_{10}$  w „poprawionym” 14-bitowym modelu zmiennoprzecinkowym
- Wiemy, że  $0.0625$  to  $2^{-4}$ .
- W (binarnej) notacji naukowej  $0.0625 = 1.0 \times 2^{-4} = 0.1 \times 2^{-3}$
- Używając notacji excess-15 do zapisania exponentu:  
dodajemy 15 do  $-3$ , co daje:

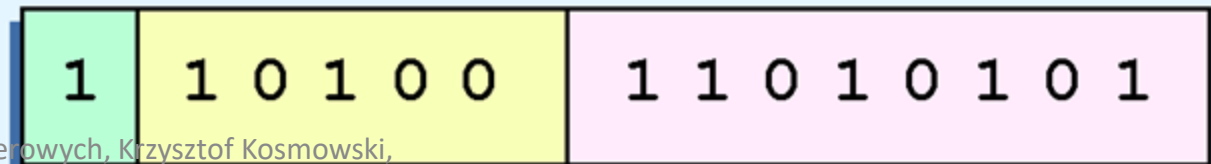
$$12_{10}(= 01100_2)$$



# Reprezentacja zmiennoprzecinkowa (17 z 30)

Przykład:

- Wyraż  $-26.625_{10}$  w poprawionej 14-bitowej liczbie zmiennoprzecinkowej
- Zapisujemy binarnie  $26.625_{10} = 11010.101_2$
- Normalizując, mamy:  $26.625_{10} = 0.11010101 \times 2^5$
- Ustalamy wartość wykładnika w notacji excess-15, (dodajemy 15 do 5),
- $20_{10} (= 10100_2)$ .
- Potrzebujemy również **1** w bicie znaku.



# Reprezentacja zmiennoprzecinkowa (18 z 30)

- IEEE ustanowiło standard dla liczb zmiennoprzecinkowych
- Standard IEEE 754 dla liczb zmiennoprzecinkowych pojedynczej precyzji definiuje wykładnik 8-bitowy (z excess-127) i 23-bitową mantysę.
- Standard IEEE 754 podwójnej precyzji definiuje wykładnik 11-bitowy (z excess-1023) i 52-bitową mantysę.



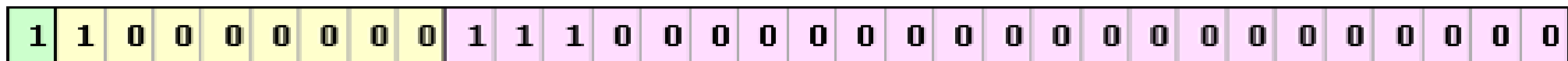
| znak | cecha /<br>wykładnik | mantysa /<br>precyzja |
|------|----------------------|-----------------------|
| 1    | 10010                | 1010110101            |

# Reprezentacja zmiennoprzecinkowa (19 z 30)

- Zarówno standard IEEE w pojedynczej precyzji, jak i w podwójnej precyzji, dla mantysy ustanawia **domyślną 1 na LEWO od przecinka**
- Mantysa (precyzja, significant) formatu IEEE to: **1.xxx...**
- Na przykład  $4.5 = 0.1001 \times 2^3$  w formacie IEEE to:  $4.5 = 1.001 \times 2^2$
- **Domyślna 1** oznacza, że nie musi być zapisana na najbardziej znaczącej pozycji (zapisuje się tylko 001).

# Reprezentacja zmiennoprzecinkowa (20 z 30)

- Przykład: Wyrazić -3,75 jako liczbę zmiennoprzecinkową za pomocą IEEE 754 w pojedynczej precyzji
- Najpierw znormalizujemy zgodnie z zasadami IEEE:  
$$-3.75 = -11.11_2 = -1.111 \times 2^1$$
- „punkt przegięcia” (*bias*) wynosi 127, więc dodajemy  $127 + 1 = 128$  (to jest nasz wykładnik)
- **Pierwsza 1 w mantysie jest domyślna**, więc mamy:



# Reprezentacja zmiennoprzecinkowa (21 z 30)

W standardzie IEEE 754 przewidziano wartości specjalne:

- przy pojedynczej precyzji wykładnik (exponent) 255 wskazuje na wartość specjalną.
  - Jeśli mantysa jest równa zero, oznacza to nieskończoność.
  - Jeśli mantysa jest niezerowa, wartością jest NaN, „Not-a-Number”, często używany do oznaczania stanu błędu.
- Przy podwójnej precyzji:
  - „Specjalna” wartość wykładnika to 2047 (zamiast 255 wg. standardu pojedynczej precyzji)

# Reprezentacja zmiennoprzecinkowa (22 z 30)

- Oba zaprezentowane modele 14-bitowy oraz standard zmiennoprzecinkowy IEEE 754 **pozwalają na dwie reprezentacje dla zera.**
- Zero jest oznaczone wszystkimi zerami w wykładniku i mantysie, **ale bit znaku może być równy 0 lub 1.**
- Dlatego programiści powinni unikać testowania wartości zmiennoprzecinkowych dla warunku równości do zera.
- **Ujemne zero nie jest równe zeru dodatniemu!**

# Reprezentacja zmiennoprzecinkowa (23 z 30)

- Bez względu na to, ile bitów użyjemy do reprezentacji liczby zmiennoprzecinkowej, nasz model musi być skończony ze względu na skończoną wielkość rejestrów
- System liczb rzeczywistych jest oczywiście nieskończony, więc nasze modele nie dają nic więcej niż przybliżenie wartości rzeczywistej.
- W pewnym momencie każdy model się „psuje”, wprowadzając błędy do naszych obliczeń.
- Używając większej liczby bitów w naszym modelu, możemy zredukować te błędy, ale nigdy nie możemy ich całkowicie wyeliminować

# Reprezentacja zmiennoprzecinkowa (24 z 30)

- Musimy mieć świadomość wielkości błędów w naszych obliczeniach.
- Musimy też mieć świadomość, że błędy mogą się nakładać przez powtarzanie operacji arytmetycznych
- Na przykład nasz 14-bitowy model nie może dokładnie reprezentować wartości dziesiętnej 128.5.
- W systemie binarnym liczba ta wymaga 9 bitów:

$$10000000,1_2 = 128,5_{10}$$

# Reprezentacja zmiennoprzecinkowa (25 z 30)

- Kiedy próbujemy wyrazić liczbę 128.5 w naszym 14-bitowym modelu, tracimy najmłodszy bit (reprezentujący wartość po przecinku)
- błąd względny:

$$\frac{128.5 - 120}{128.5} \approx 0.39\%$$

- Gdybyśmy mieli procedurę, która czterokrotnie (w czterech iteracjach) dodawała 0.5 do 128.5 nasz błąd wyniósłby prawie 2%

# Reprezentacja zmiennoprzecinkowa (26 z 30)

- Błędy zmiennoprzecinkowe można zmniejszyć, gdy używa się operandów o podobnej wielkości
- Gdybyśmy wielokrotnie dodawali 0.5 do 128.5, to lepiej byłoby dodać iteracyjnie 0.5 do siebie, a następnie dodać 128.5 do tej sumy
- W tym przykładzie przyczyną błędu była strata najniższego rzędu (najmłodszego bitu)
- Bardziej problematyczna jest utrata bitu wyższego rzędu

# Reprezentacja zmiennoprzecinkowa (27 z 30)

- Przepelnienie i niedomiar zmiennoprzecinkowy może powodować awarię programu
- Przepelnienie występuje, gdy nie ma miejsca do przechowywania bitów wyższego rzędu wynikające z obliczeń
- Niedomiar występuje, gdy wartość jest zbyt mała, aby ją zapisać
- lepiej program zawiesić, niż spowodować, że będzie produkował niepoprawnie, ale „wiarygodne” wyniki.

# Reprezentacja zmiennoprzecinkowa (28 z 30)

- Omawiając liczby zmiennoprzecinkowe, jest ważne, aby rozumieć pojęcia zakres (*range*), precyzja (*precision*) i dokładność (*accuracy*)
- ***Range*** - dla formatu liczb całkowitych to różnica między największą a najmniejszą wartością, które można wyrazić
- ***Accuracy*** - odnosi się do tego, jak bardzo reprezentacja tej liczby aproksymuje jej prawdziwą wartość
- ***Precision*** - wskazuje, ile posiadamy informacji o wartości tej liczby

# Reprezentacja zmiennoprzecinkowa (29 z 30)

- W większości przypadków większa precyzja prowadzi do lepszej dokładności, ale nie zawsze tak jest
- Na przykład:  
~~3.1333 to wartość pi, która jest z dokładnością do dwóch cyfr, ale z precyzją do 5 cyfr~~
- Są inne problemy związane z liczbami zmiennoprzecinkowymi
  - Z powodu obciętych bitów nie zawsze można założyć, że konkretna operacja zmiennoprzecinkowa ma charakter asocjacyjny lub dystrybucyjny

# Reprezentacja zmiennoprzecinkowa (30 z 30)

- Oznacza to, że nie możemy założyć, że:
  - $(a + b) + c = a + (b + c)$  lub
  - $a(b + c) = ab + ac$
- Co więcej, aby przetestować wartość zmiennoprzecinkową pod kątem równości z inną liczbą najlepiej zadeklarować wartość *epsilon* i badać „bliskość x”.
- Na przykład zamiast tego sprawdzania:  
if **x = 2** then ...  
lepiej jest użyć: if (**abs(x - 2) < epsilon**) then ...

(zakładając, że mamy zdefiniowany poprawnie *epsilon*)

# Kodowanie znaków (1 z 6)

- Obliczenia nie przydają się do czasu uzyskania wyników, które mogą być wyświetlane w sposób, który jest zrozumiały dla ludzi
- Musimy również przechowywać wyniki obliczenia i dostarczać do danych wejście
- Tak więc postacie znaków i liczb zrozumiałe dla ludzki muszą być przekonwertowane na bity zrozumiały dla komputera używając jakiegoś schemat kodowania znaków

# Kodowanie znaków (2 z 6)

- Wraz z ewolucją komputerów kody znaków również ewoluowały
- Większe pamięci komputerowe i pamięci masowe umożliwiają stosowanie „bogatszych” kodów znaków
- Najwcześniejsze komputerowe systemy kodowania używały w tym celu sześciu bitów
- Jednym z nich był binarny kod dziesiętny (BCD), przez komputery mainframe IBM w latach 50. i 60. XX wieku

# Kodowanie znaków (3 z 6)

- W 1964 BCD został rozszerzony do 8-bitowego kodu *Extended Binary-Coded Decimal Interchange Code* (EBCDIC)
- EBCDIC był jednym z pierwszych powszechnie stosowanych kodów komputerowych obsługującym duże i małe litery alfabetu, znaki specjalne, takie jak interpunkcja oraz znaki kontrolne
- EBCDIC i BCD są nadal używane (np. przez mainframe IBM).

# Kodowanie znaków (4 z 6)

- Inni producenci komputerów wybrali 7-bitowy kod ASCII (*American Standard Code for Information Interchange*) jako zamiennik dla BCD
- Podczas gdy BCD i EBCDIC były oparte na kodach kart perforowanych, kody ASCII bazują na stosowanych w telekomunikacji (Telex)
- Do niedawna ASCII był dominującym systemem kodowym

# Kodowanie znaków (5 z 6)

- Wiele dzisiejszych systemów obsługuje **Unicode**, 16-bitowy system, który może kodować znaki wszystkich języków świata
- Język programowania Java, a także niektóre systemy operacyjne używają teraz *Unicode* jako ich domyślnego systemu kodowania znaków.
- Przestrzeń kodowa *Unicode* jest podzielona na 6 części.
  - Pierwsza część dotyczy alfabetu zachodniego, w tym kody dla języków: angielski, grecki i rosyjski.

# Kodowanie znaków (6 z 6)

- alokacja przestrzeni kodowej Unicode jest pokazana po prawej stronie.
- Najniższe numery Unicode zawierają znaki kodu ASCII.
- Najwyższa pula przeznaczona jest dla definiowania kodów przez użytkownika.

| Character Types | Language                                                        | Number of Characters | Hexadecimal Values |
|-----------------|-----------------------------------------------------------------|----------------------|--------------------|
| Alphabets       | Latin, Greek, Cyrillic, etc.                                    | 8192                 | 0000 to 1FFF       |
| Symbols         | Dingbats, Mathematical, etc.                                    | 4096                 | 2000 to 2FFF       |
| CJK             | Chinese, Japanese, and Korean phonetic symbols and punctuation. | 4096                 | 3000 to 3FFF       |
| Han             | Unified Chinese, Japanese, and Korean                           | 40,960               | 4000 to DFFF       |
|                 | Han Expansion                                                   | 4096                 | E000 to EFFF       |
| User Defined    |                                                                 | 4095                 | F000 to FFFE       |

# Koniec

# Architektura Systemów Komputerowych

Budowa prostego komputera  
na przykładzie symulatora MARIE

Dr inż. Krzysztof Kosmowski

# Cele wykładu

- Poznanie elementów wspólnych dla wszystkich nowoczesnych systemów komputerowych
- wyjaśnienie w jaki sposób każdy element przyczynia się do realizacji programu
- Poznanie prostej architektury opracowanej, aby wyjaśnić te podstawowe pojęcia oraz jak to odnosi się do niektórych prawdziwych architektur
- Pokazać jak wygląda proces wykonywania programu

# WSTĘP

- Na wykładzie 1 przedstawiono ogólny przegląd systemów komputerowych
- Na wykładzie 2 omówiliśmy sposób przechowywania i manipulowania danymi przez systemy komputerowe
- BRAKUJACY ELEMENT - podstawowe elementy obwodów cyfrowych (powinny być przedstawione w ramach innego przedmiotu)
- Mając powyższe tło, postaramy się zrozumieć jak działają komponenty komputera

# CPU - Podstawy (1 z 2)

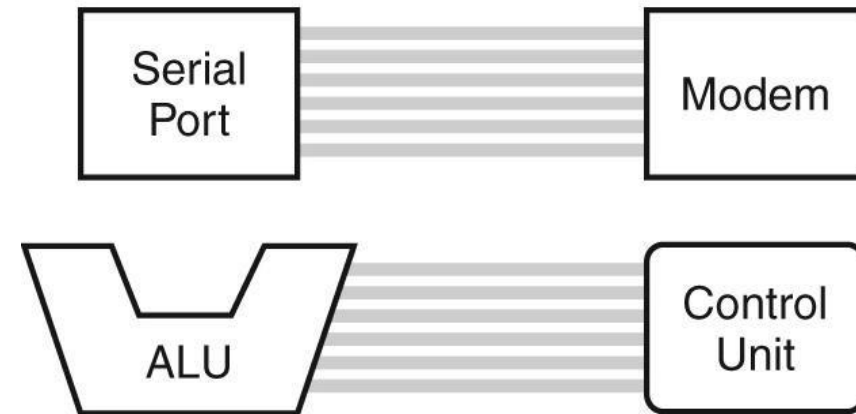
- Procesor komputera pobiera, dekoduje i wykonuje instrukcje programu
- Dwie główne części procesora to ścieżka danych (*datapath*) i jednostka sterująca (*Control Unit*)
  - Ścieżka danych składa się z jednostki arytmetyczno-logicznej (arithmetic-logic unit ALU) i jednostek magazynujących (rejestry), które są połączone za pomocą magistrali danych, która jest również podłączona do pamięci głównej
- Różne komponenty procesora wykonują sekwencje operacji zgodnie z sygnałami dostarczonymi przez jego jednostkę sterującą.

# CPU - Podstawy (2 z 2)

- Rejestry przechowują dane, które są natychmiastowo dostępne dla CPU
- Można je zaimplementować za pomocą przerzutników typu D
- Rejestr 32-bitowy wymaga 32 przerzutników D
- Jednostka arytmetyczno-logiczna (ALU) wykonuje operacje logiczne i arytmetyczne zgodnie z poleceniami jednostki sterującej
- Jednostka sterująca określa, jakie działania należy przeprowadzić zgodnie z wartościami zapisanymi w liczniku rozkazów (*counter register*) i rejestrze stanu (*status register*)

# Magistrala (Szyna, The Bus) (1 z 5)

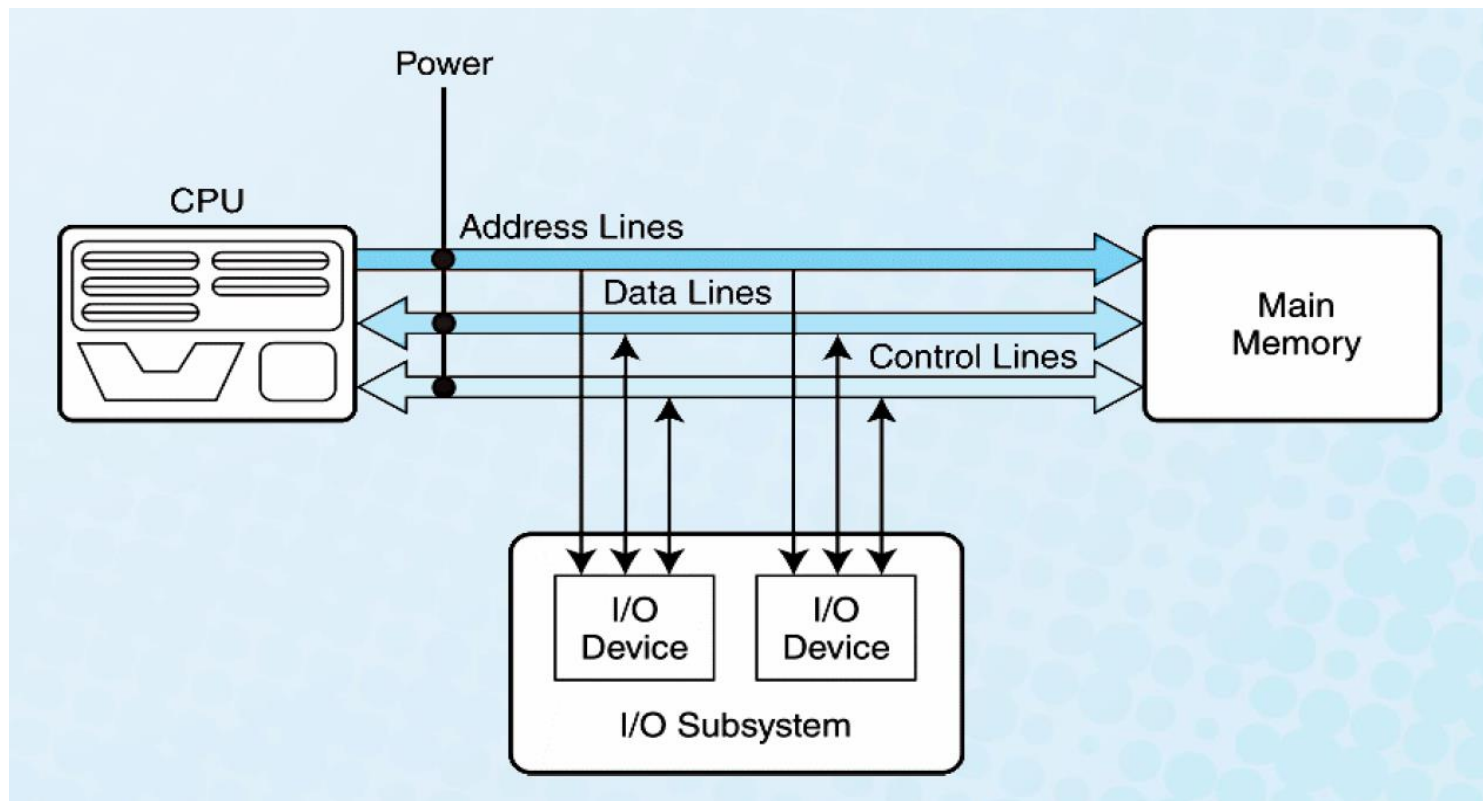
- Procesor współdzieli dane z innym komponentami systemem za pomocą magistrali danych
  - Magistrala to zestaw „przewodów”, które równocześnie przenoszą jeden bit wzdłuż każdej linii
- Powszechnie spotyka się dwa rodzaje magistrali w systemach komputerowych:
  - punkt-punkt
  - i wielopunktowe.
- Tutaj magistrale punkt-punkt:



# Magistrala (Szyna, The Bus) (2 z 5)

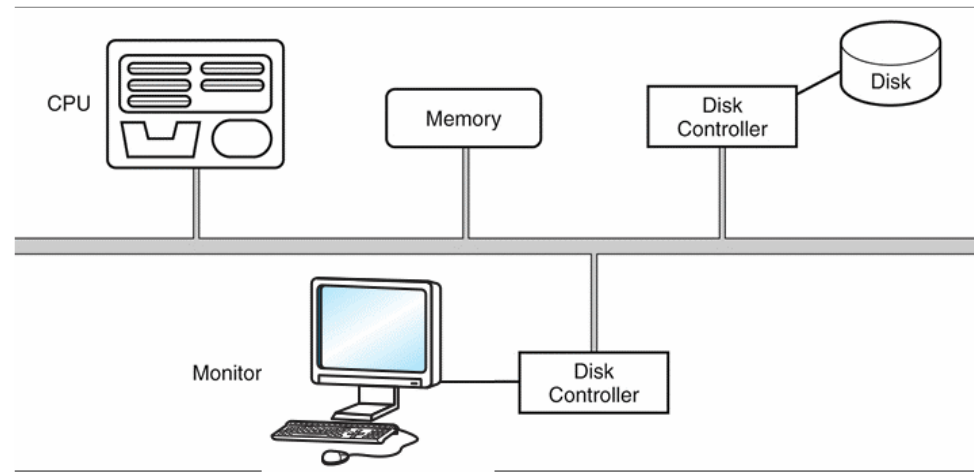
- Magistrale składają się z linii danych, linii sterujących i linii adresowych
  - Linie danych przekazują bity z jednego urządzenia do innego
  - Linie sterujące określają kierunek przepływu danych i kiedy każde urządzenie może uzyskać dostęp do magistrali
  - Linie adresowe określają lokalizację (źródła lub miejsce docelowego) danych

# Magistrala (Szyna, The Bus) (3 z 5)



# Magistrala (Szyna, The Bus) (4 z 5)

- Magistralę wielopunktową pokazano poniżej
- Ponieważ magistrala wielopunktowa jest zasobem współdzielonym, dostęp do niego jest kontrolowany przez protokoły wbudowane w sprzęt komputerowy (*hardware*)



# Magistrala (Szyna, The Bus) (5 z 5)

- W konfiguracji master-slave, gdzie więcej niż jedno urządzenie może być nadrzędnym dla magistrali (*bus master*), konkurujące o dostęp urządzenia *master* muszą podlegać arbitrażowi
- Mechanizmy arbitrażu możemy podzielić na cztery kategorie:
  - **Arbitraż szeregowy**: metoda wykorzystuje linię sterującą „przydziel magistralę”, która przekazuje uprawnienia od urządzenia z najwyższym priorytetem do najniższego
  - **Scentralizowany arbitraż równoległy**: Każde urządzenie ma linię sterującą żądającą dostępu do magistrali, centralny arbiter go przydziela
  - **Zdecentralizowany arbitraż oparty na samodzielnym wyborze**: urządzenia między sobą decydują, które z nich ma wyższy priorytet i powinno otrzymać dostęp do magistrali jako pierwsze.
  - **Zdecentralizowany arbitraż z wykrywaniem kolizji**: każde urządzenie może zażądać dostępu do magistrali. Jeśli magistrala wykryje jakiegokolwiek kolizje (wiele żądań równocześnie) urządzenie to będzie musiało wystosować żądanie jeszcze raz

# Zegary (Clocks) (1 z 2)

- Każdy komputer zawiera co najmniej jeden zegar, który synchronizuje aktywność jego komponentów
- Do przeprowadzenia każdej operacji przeniesienia danych lub wykonania obliczeń wymagana jest stała liczba cykli zegara
- Częstotliwość zegara mierzona w megahercach (MHz) lub gigahercach (GHz) określa prędkość z jaką wszystkie operacje są przeprowadzane
- Czas cyklu zegara jest odwrotnością częstotliwości zegara.
  - Zegar 800 MHz ma czas cyklu 1,25 ns

# Zegary (Clocks) (2 z 2)

- Nie należy mylić szybkości zegara z wydajnością procesora
- Czas procesora wymagany do uruchomienia programu jest określony przez ogólne równanie wydajności:
- $$CZAS_{procesora} = \frac{liczba\_sekund}{program} = \frac{liczba\_instrukcji}{program} \times \frac{średnia\_liczba\_cykli}{instrukcję} \times \frac{liczba\_sekund}{cykl}$$
- Widzimy, że możemy poprawić przepustowość procesora, gdy zmniejszymy:
  - ilość instrukcji w programie lub
  - liczbę cykli na instrukcję lub
  - liczbę nanosekund na cykl zegara

# Podsystem wejścia-wyjścia

- Komputer komunikuje się ze światem zewnętrznym poprzez swój podsystem wejścia/wyjścia (I/O)
- Urządzenia we/wy nie komunikują się z procesorem bezpośrednio - łączą się za pomocą różnych interfejsów
- Komunikacja może być dwojaka:
  - We/wy odwzorowywane jest pamięciowo – rejestry interfejsu pojawiają się na mapie pamięci komputera, z punktu widzenia procesora mogą być traktowane jak pamięć główna
  - We/wy oparte na instrukcjach (rozkazach) gdzie procesor ma specjalistyczny zestaw instrukcji we/wy

# Organizacja pamięci (1 z 7)

- Pamięć komputera składa się z liniowej tablicy adresowalnych komórek pamięci, które są podobne do rejestrów
- Pamięć może być adresowalna bajtowo lub słowem, gdzie słowo zazwyczaj składa się z dwóch lub więcej bajtów
- Pamięć jest zbudowana z kości RAM, często mówimy w kontekście *długość x szerokość*
- Jeśli rozmiar słowa pamięci maszyny wynosi 16 bitów, wówczas układ 4Mx16RAM udostępnia nam 4 „mega” 16-bitowych lokalizacji pamięci

# Organizacja pamięci (2 z 7)

- W jaki sposób komputer uzyskuje dostęp do lokalizacji w pamięci, która odpowiada konkretnemu adresowi?
- zauważmy, że 4M można wyrazić jako  $2^2 \times 2^{20} = 2^{22}$  słów
- Lokalizacje komórek pamięci dla tej pamięci są ponumerowane od  $0 \div 2^{22} - 1$
- W związku z tym magistrala pamięci tego systemu wymaga co najmniej 22 linii adresowych
  - Linie adresowe „liczą” od  $0 \div 2^{22} - 1$  binarnie.
  - Każda linia jest „włączona” lub „wyłączona”, wskazując lokalizację żądanego elementu pamięci.

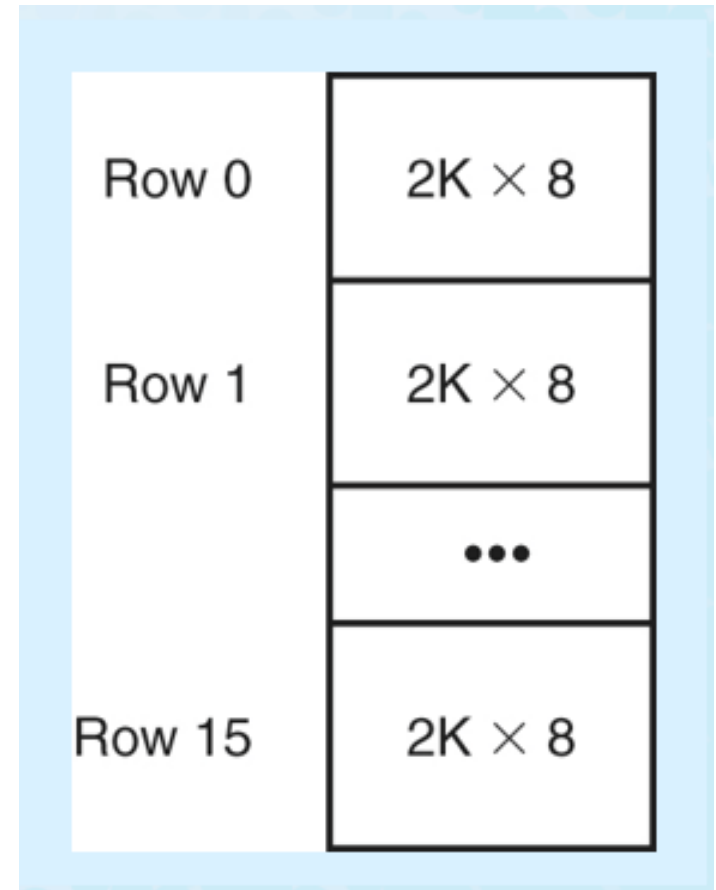
# Organizacja pamięci (3 z 7)

- Pamięć fizyczna zwykle składa się z więcej niż jednego kości pamięci RAM.
- Dostęp jest bardziej wydajny, gdy pamięć jest zorganizowana w postaci banków kości z przeplatanymi adresami
- Dwa rodzaje przeplotu:
  - ***Niskopozycyjny przeplot pamięci***
  - ***Wysokopozycyjny przeplot pamięci***
- w przypadku przeplatania niskopozycyjnego bity niskiego rzędu w adresie komórki określają, który bank pamięci zawiera adres zainteresowania.
- w wysokopozycyjnym przeploci, bity wyższego rzędu w adresie określają bank pamięci.

Kolejne dwa slajdy ilustrują te dwa rozwiązania

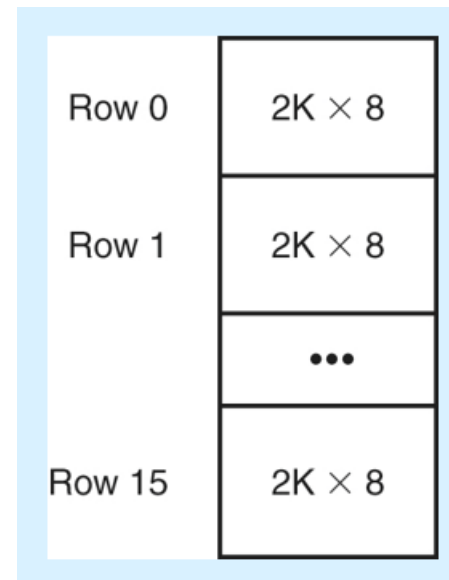
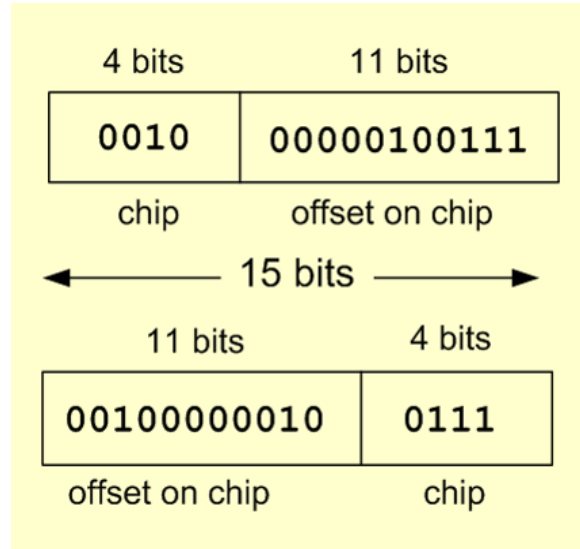
# Organizacja pamięci (4 z 7)

- Przykład: Załóżmy, że mamy pamięć składająca się z 16  $2K \times 8$ -bitów kości pamięci.
- Cała pamięć to  $32K = 2^5 \times 2^{10} = 2^{15}$
- potrzeba 15 bitów aby zaadresować każdą komórkę
- Potrzebujemy 4 bitów, aby wybrać kość i 11 bitów na offset do chipa, który wybiera odpowiedni bajt.



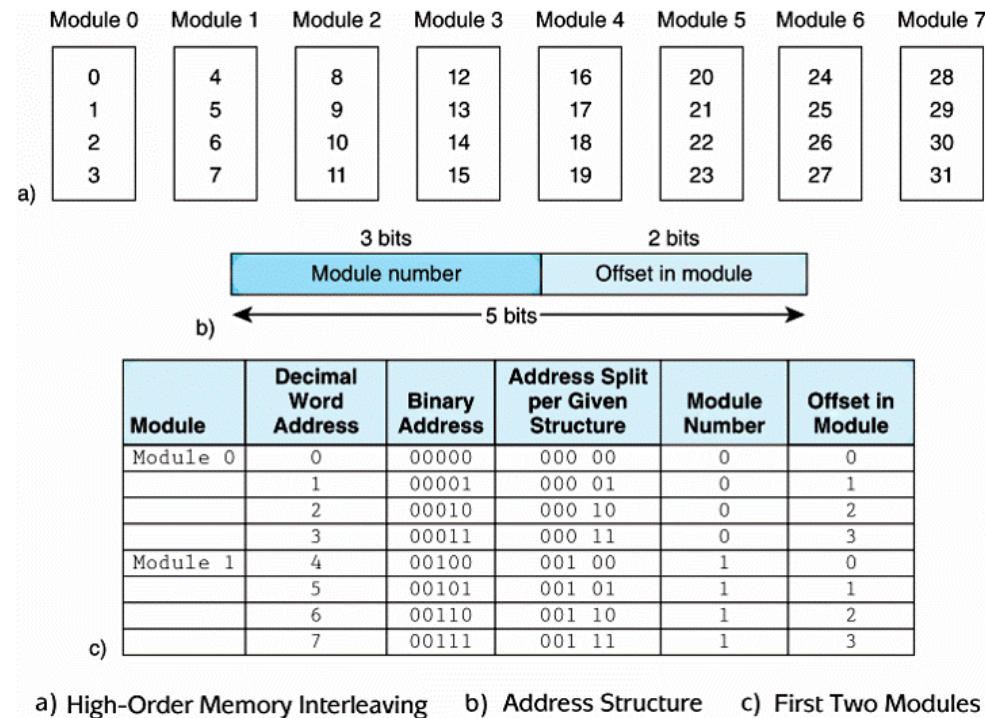
# Organizacja pamięci (5 z 7)

- **Niskopozycyjny przeplot pamięci** – 4 bity niskiego rzędu (low-order) określają kość
- **Wysokopozycyjny przeplot pamięci** – 4 bity wysokiego rzędu (high-order) określają kość



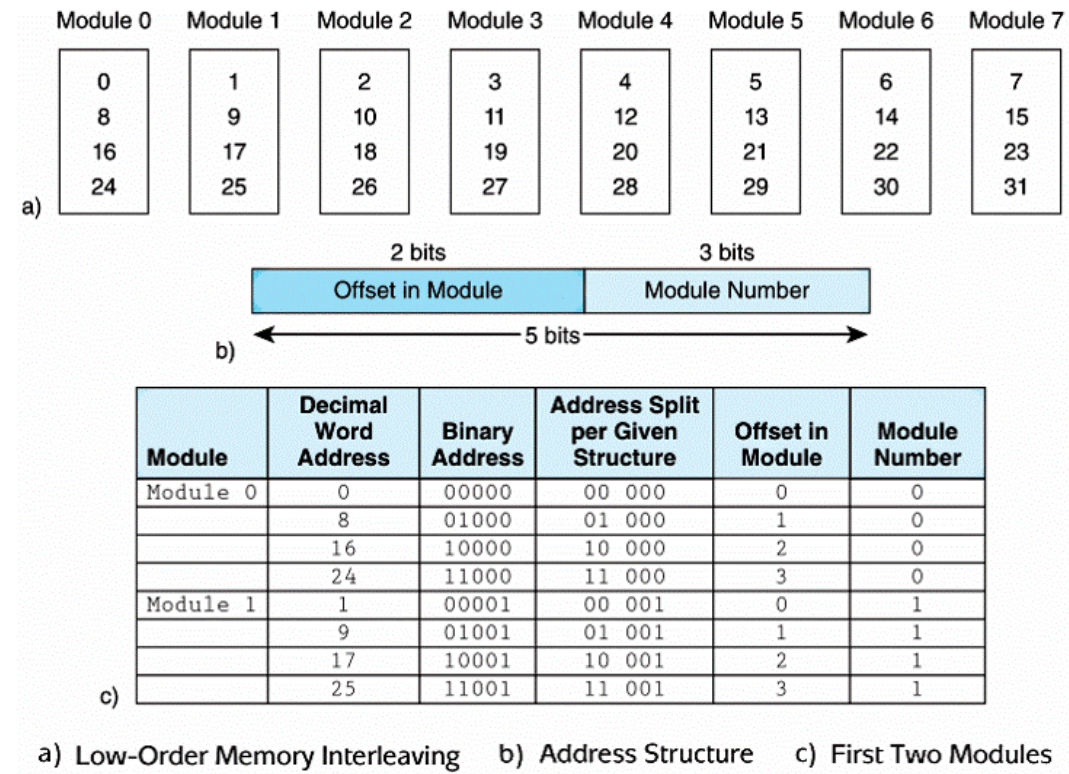
# Organizacja pamięci (6 z 7)

- Wysokopozycyjny przeplot pamięci



# Organizacja pamięci (7 z 7)

- Niskopozycyjny przeplot pamięci



# Przerwania

- Przerwania to zdarzenia, które zakłócają normalny tryb wykonywania programu. Mogą być inicjowane w różnych sytuacjach
- Procesor jest powiadomiony o takim zdarzeniu poprzez przerwanie
- Przerwania mogą być wyzwalane przez żądania I/O, błędy arytmetyczne (takie jak dzielenie przez zero) lub kiedy napotkano nieprawidłową instrukcję
- Każde przerwanie jest powiązane z procedurą, która kieruje działaniami procesora w przypadku wystąpienia przerwania występuje.
- Przerwania mogą być inicjowane przez użytkownika lub przez sam komputer
  - *Maskowalne* – można zignorować lub wyłączyć
  - *Niemaskowalne* – są przerwaniami o wysokim priorytecie, nie można zignorować.

# MARIE (1 z 14)

- Możemy teraz połączyć przedstawione wyżej idee za pomocą bardzo prostego modelu komputera.
- Nasz modelowy komputer został zaprojektowany w celu zilustrowania podstawowych koncepcji systemów komputerowych:
  - MARIE – A ***M**achine **A**rchitecture that is **R**eally **I**ntuitive and **E**asy*
- system jest zbyt prosty, aby zrobić cokolwiek przydatnego w prawdziwym świecie pozwala jednak zrozumieć jak funkcjonują znacznie bardziej złożone architektury rzeczywiste

# MARIE (2 z 14)

- Architektura MARIE:
  - Binarna, reprezentacja danych w zapisie uzupełnieniowym do dwóch.
  - przechowywanie programu, stała długość słowa danych i instrukcji
  - 4K pamięci głównej
  - 16-bitowe dane (słowa mają długość 16 bitów)
  - 16-bitowe instrukcje, 4 na kod operacji (opcode) i 12 na adres
  - 16-bitowa jednostka arytmetyczno-logiczna (ALU)
  - Siedem rejestrów do sterowania i przenoszenia danych.

# MARIE (3 z 14)

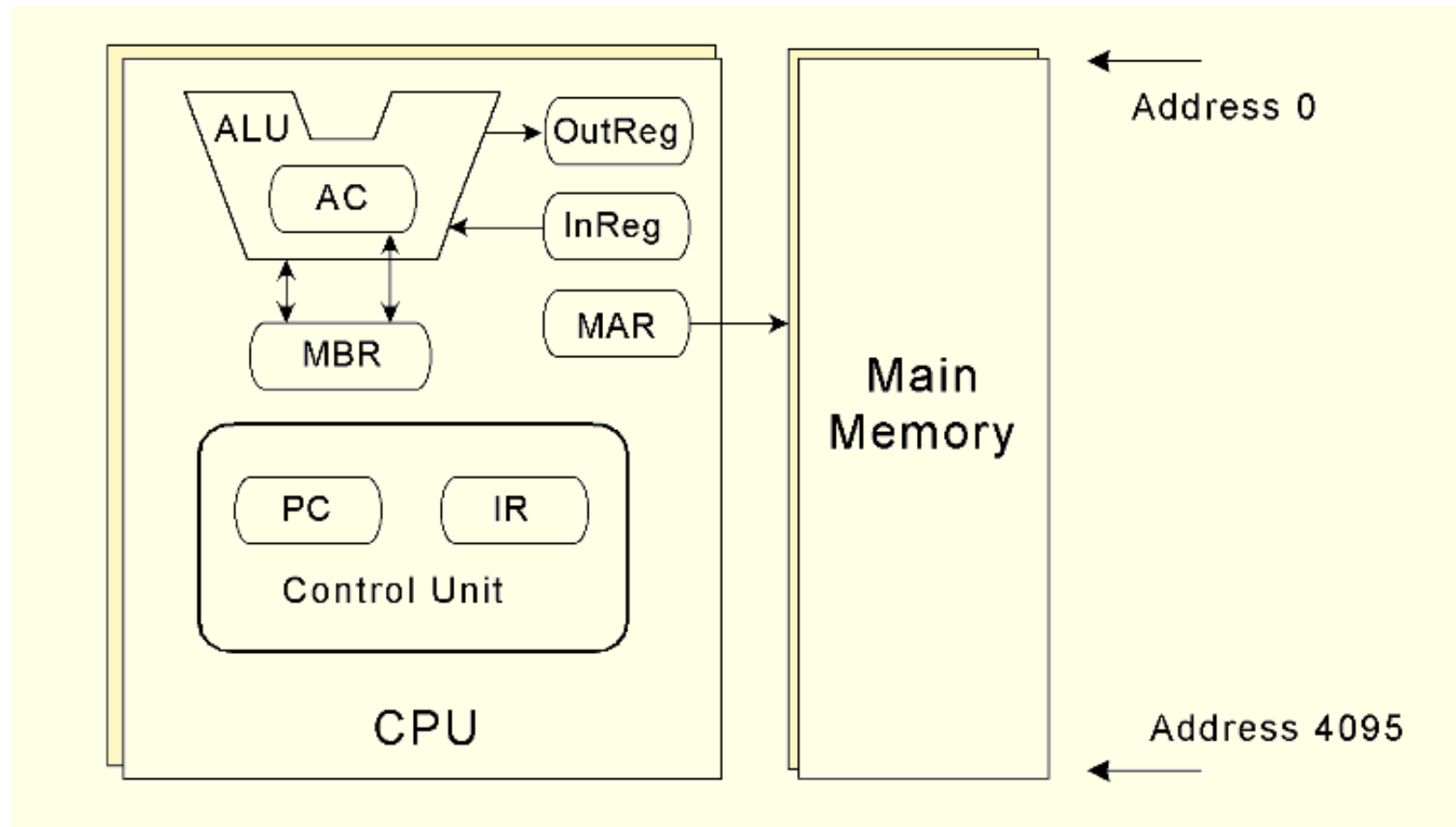
- Siedem rejestrów MARIE to:
- (1) **Akumulator (AC)** - 16-bitowy rejestr uniwersalny. Przechowuje wartości danych (jeden z dwóch operandów dwuargumentowej instrukcji) lub operatory warunkowe (np. „mniejszy niż”). Większość współczesnych komputerów ma wiele rejestrów uniwersalnych
- (2) **Rejestr adresu pamięci (MAR)** - 12-bitowy rejestr, który przechowuje adres komórki pamięci:
  - z danymi do których się odnosimy (argument instrukcji) lub
  - adres instrukcji.
- (3) **Rejestr bufora pamięci (MBR)** - rejestr 16-bitowy, który przechowuje dane właśnie odczytane z pamięci, albo dane gotowe do zapisania

# MARIE (4 z 14)

- (4) ***Licznik programu (PC)*** - 12-bitowy rejestr, który przechowuje adres następnej instrukcji jaką program ma do wykonania
- (5) ***Rejestr instrukcji (IR)*** - przechowuje następny rozkaz jakie należy wykonać
- (6) ***Rejestr wejściowy (InREG)*** - 8-bitowy rejestr, przechowuje dane odczytane z urządzenia wejściowego
- (7) ***Rejestr wyjściowy (OutREG)*** - rejestr 8-bitowy, przechowuje dane, dla urządzenia wyjściowego

# MARIE (5 z 14)

- *Akumulator (AC)*
- *Rejestr adresu pamięci (MAR)*
- *Rejestr bufora pamięci (MBR)*
- *Licznik programu (PC)*
- *Rejestr instrukcji (IR)*
- *Rejestr wejściowy (InREG)*
- *Rejestr wyjściowy (OutREG)*



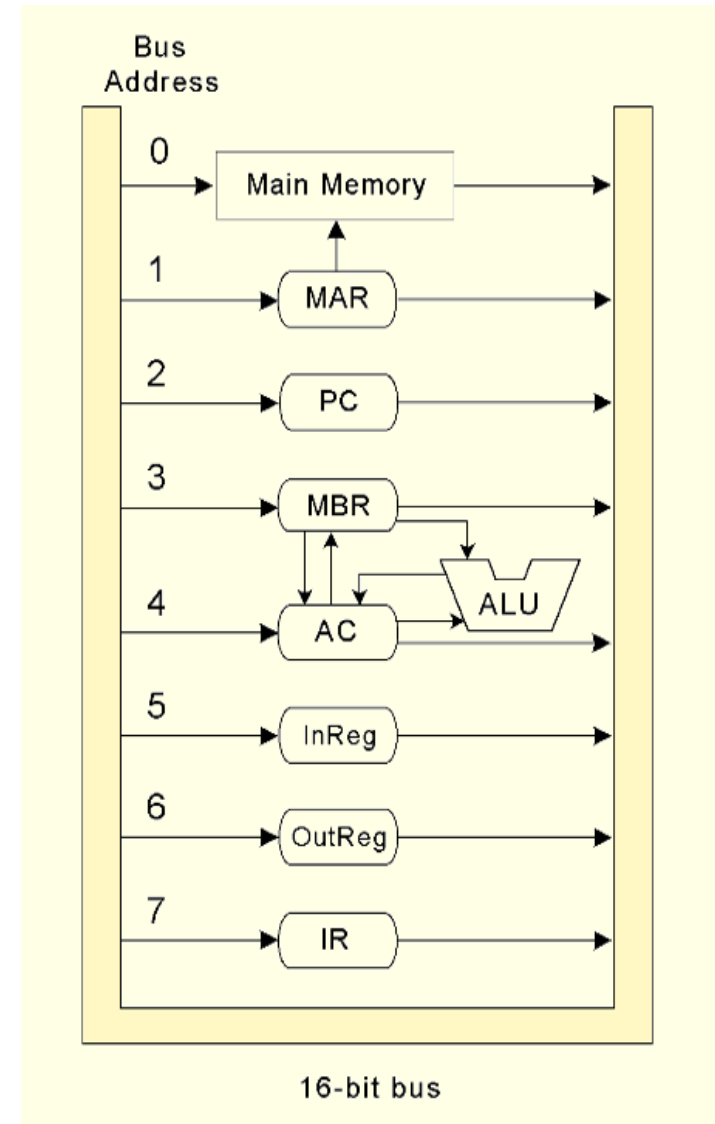
# MARIE (6 z 14)

- Rejestry są ze sobą połączone i połączone z pamięcią główną przez wspólną magistralę danych
- Każde urządzenie w magistrali jest identyfikowane przez unikalny numer, który jest ustawiany na liniach kontrolnych, za każdym razem gdy to urządzenie jest wymagane do przeprowadzenia operacji
- Dostępne są również oddzielne połączenia między akumulatorem (AC), rejestrem bufora pamięci (MBR) i ALU
- Pozwala to na przesyłanie danych między tymi urządzeniami bez użycia głównej magistrali danych.

# MARIE (7 z 14)

- Ścieżka danych w MARIE

- *Akumulator (AC)*
- *Rejestr adresu pamięci (MAR)*
- *Rejestr bufora pamięci (MBR)*
- *Licznik programu (PC)*
- *Rejestr instrukcji (IR)*
- *Rejestr wejściowy (InREG)*
- *Rejestr wyjściowy (OutREG)*

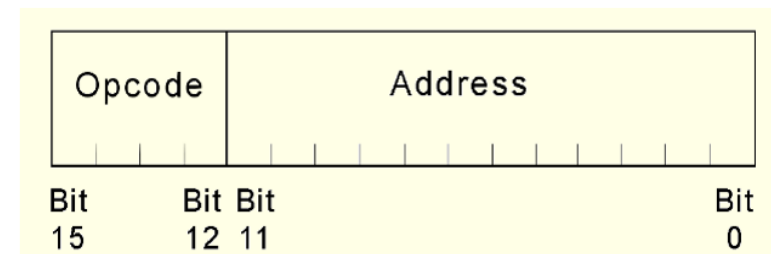


# MARIE (8 z 14)

- *Architektura zestawu instrukcji* komputera (ISA) określa zestaw i format instrukcji, które maszyna może wykonać
- ISA jest interfejsem między sprzętem (hardware) i jego oprogramowaniem (software)
- Niektóre ISA zawierają setki różnych instrukcji
- MARIE ISA składa się tylko z 13 instrukcji.

# MARIE (9 z 14)

- format instrukcji MARIE:
- Podstawowe instrukcje MARIE:

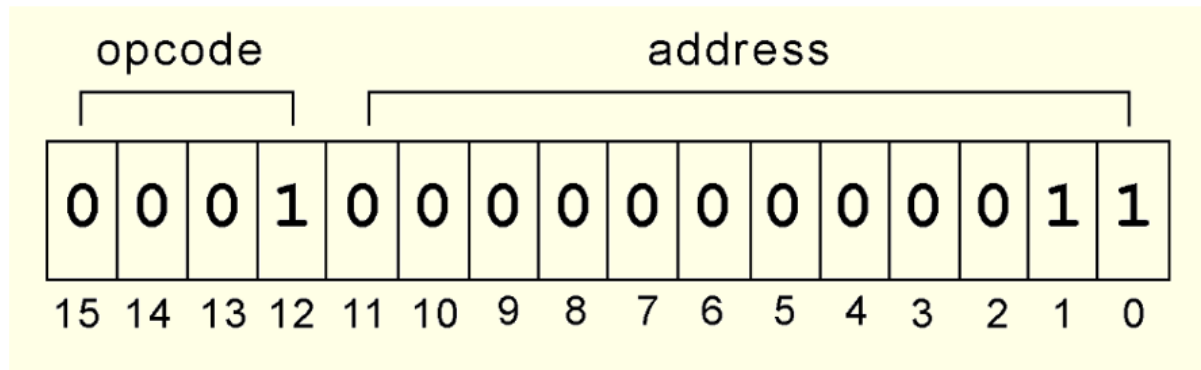


| Instruction Number |     |             |                                             |
|--------------------|-----|-------------|---------------------------------------------|
| Binary             | Hex | Instruction | Meaning                                     |
| 0001               | 1   | Load X      | Load contents of address X into AC.         |
| 0010               | 2   | Store X     | Store the contents of AC at address X.      |
| 0011               | 3   | Add X       | Add the contents of address X to AC.        |
| 0100               | 4   | Subt X      | Subtract the contents of address X from AC. |
| 0101               | 5   | Input       | Input a value from the keyboard into AC.    |
| 0110               | 6   | Output      | Output the value in AC to the display.      |
| 0111               | 7   | Halt        | Terminate program.                          |
| 1000               | 8   | Skipcond    | Skip next instruction on condition.         |
| 1001               | 9   | Jump X      | Load the value of X into PC.                |

| Numer instrukcji |              | Instrukcja | Znaczenie                                                                      |
|------------------|--------------|------------|--------------------------------------------------------------------------------|
| Dwójkowo         | Szesnastkowo |            |                                                                                |
| 0001             | 1            | Load X     | Załaduj zawartość spod adresu X do akumulatora                                 |
| 0010             | 2            | Store X    | Zachowaj zawartość akumulatora pod adresem X                                   |
| 0011             | 3            | Add X      | Dodaj zawartość spod adresu X do akumulatora i zachowaj wynik w akumulatorze   |
| 0100             | 4            | Subt X     | Odejmij zawartość spod adresu X od akumulatora i zachowaj wynik w akumulatorze |
| 0101             | 5            | Input      | Wprowadź wartość z klawiatury do akumulatora                                   |
| 0110             | 6            | Output     | Wyślij wartość przechowywaną w akumulatorze na ekran                           |
| 0111             | 7            | Halt       | Zakończ program                                                                |
| 1000             | 8            | Skipcond   | Pomiń następny rozkaz, jeżeli spełniony jest określony warunek                 |
| 1001             | 9            | Jump X     | Wczytaj wartość X do licznika rozkazów                                         |

# MARIE (10 z 14)

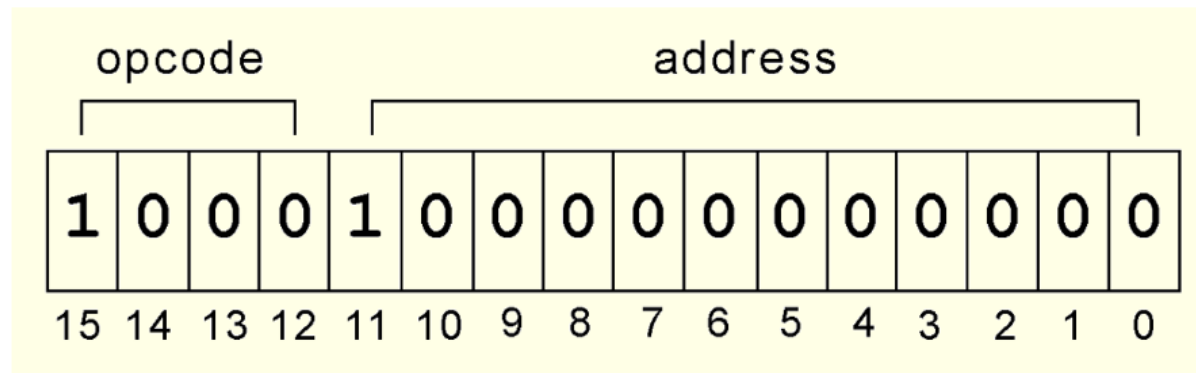
- „wzór” bitowy dla instrukcji LOAD tak jak mogłoby to wyglądać w IR:



Widzimy, że *opcode* to 1, a adres, z którego należy wczytać dane to 3.

# MARIE (11 z 14)

- „wzór” bitowy dla instrukcji SKIPCOND tak jak mogłoby to wyglądać w IR:



- Widzimy, że *opcode* to 8, a bity 11 i 10 to 10, co oznacza, że następna instrukcja będzie pomijana, jeśli wartość w AC jest większa od zera.

# MARIE (12 z 14)

- Każda z naszych instrukcji w rzeczywistości składa się z sekwencji mniejszych instrukcji zwanych mikrooperacjami
- Dokładną sekwencję mikrooperacji, które są realizowane przez instrukcję można określić za pomocą *Register Transfer Language* (RTL) – notacja „rejestr-przesunięcie”
- W MARIE RTL używamy notacji  $M[X]$  do wskazania rzeczywistej wartości danych przechowywanych w pamięci w lokalizacji  $X$  i  $\leftarrow$  w celu wskazania transferu bajtów do rejestru lub lokalizacji w pamięci.

# MARIE (13 z 14)

- Notacja RTL dla polecenia **LOAD**:

```
MAR ← X  
MBR ← M[MAR]  
AC ← MBR
```

- Notacja RTL dla polecenia **ADD**:

```
MAR ← X  
MBR ← M[MAR]  
AC ← AC + MBR
```

- *Akumulator (AC)*
- *Rejestr adresu pamięci (MAR)*
- *Rejestr bufora pamięci (MBR)*

# MARIE (14 z 14)

- Przypomnijmy sobie, że **SKIPCOND** pomija następną instrukcję zgodnie z wartością AC
- Notacja RTL dla tej instrukcji jest najbardziej złożona:

```
If IR[11 - 10] = 00 then
    If AC < 0 then PC ← PC + 1
else If IR[11 - 10] = 01 then
    If AC = 0 then PC ← PC + 1
else If IR[11 - 10] = 10 then
    If AC > 0 then PC ← PC + 1
```

- *Akumulator (AC)*
- *Rejestr adresu pamięci (MAR)*
- *Rejestr bufora pamięci (MBR)*
- *Licznik programu (PC)*
- *Rejestr instrukcji (IR)*

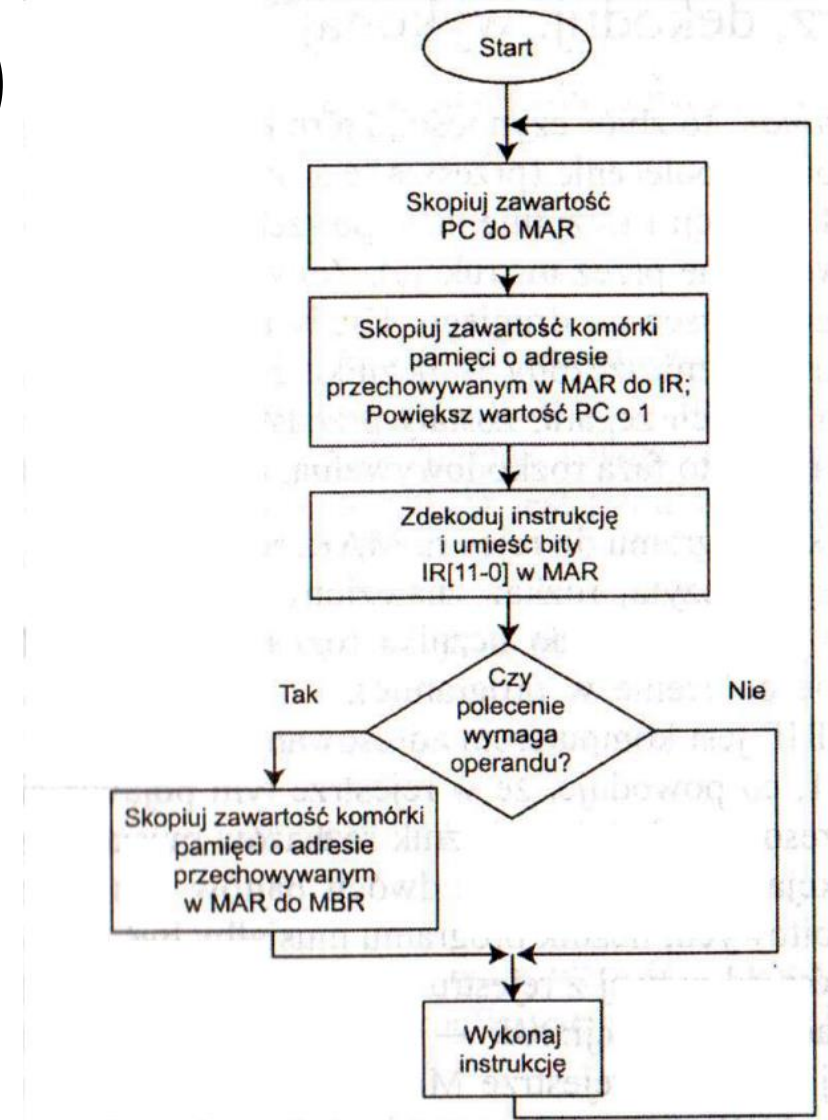
# Przetwarzanie instrukcji (1 z 5)

- Cykl pobierz-dekoduj-wykonaj (*fetch-decode-execute*) to seria kroków, które komputer wykonuje podczas uruchamiania programu
- Najpierw musimy **pobrać instrukcję z pamięci i umieścić ją w Rejestrze instrukcji (IR)** (*faza fetch*)
- W IR jest ona **dekodowana** (*decode*), aby określić co należy zrobić dalej
- Jeśli do wykonania operacji konieczna jest wartość zapisana w pamięci (operand) jest ona pobierana i umieszczana w **rejestrze bufora pamięci (MBR)**
- Gdy wszystko jest na swoim miejscu, instrukcja jest wykonana

Następny slajd przedstawia schemat blokowy tego procesu.

# Przetwarzanie instrukcji (2 z 5)

- 1. Kopiuj zawartość licznika programu do rejestru MAR:  $MAR \leftarrow PC$
- 2. Idź do pamięci głównej i wczytaj rozkaz znaleziony pod adresem zapisanym w MAR, umieść go w IR, dodaj do licznika rozkazów 1
- 3. Skopiuj pierwsze 12 bitów (od prawej) z rejestru IR do rejestru MAR; Rozkoduj cztery pierwsze bity (od lewej), aby poznać kod operacji, a następnie zdekoduj IR
- 4. Jeśli to konieczne, użyj adresu w rejestrze MAR, aby z tego miejsca pamięci pobrać dane i umieścić je w rejestrze MBR(i w razie potrzeby również w akumulatorze). Następnie wykonaj polecenie



# Przetwarzanie instrukcji (3 z 5)

- Wszystkie komputery zapewniają sposób na przerwanie cyklu pobierz-dekoduj-wykonaj
- Przerwania są asynchroniczne i wskazują na rodzaj usługi jaki jest wymagany.
- Przerwania występują, gdy:
  - Następuje przerwanie użytkownika (np. Control+C)
  - Operacja We/Wy jest wymagana przez użytkownika lub program
  - Wystąpił błąd krytyczny
- Przerwania mogą być wywołane przez sprzęt lub oprogramowanie
  - Przerwania programowe są również nazywane pułapkami (*trap*)

# Przetwarzanie instrukcji (4 z 5)

- W przypadku systemów ogólnego przeznaczenia często wyłącza się wszystkie przerwania w czasie, w którym przetwarzane jest przerwanie.
- Zazwyczaj osiąga się to poprzez ustawienie bitów w rejestrze flag.
- Przerwania, które są w tym przypadku ignorowane, nazywamy *maskowalnymi*
- Przerwania *niemaskowalne* to te przerwania, które muszą zostać przetworzone, aby zachować stabilność systemu

# Przetwarzanie instrukcji (5 z 5)

- Przerwania są bardzo przydatne w przetwarzaniu I/O
- sterowane przerwaniem we/wy jest skomplikowane i wykracza poza zakres obecnej dyskusji
- MARIE, będąc najprostszym z prostych systemów, używa zmodyfikowanej formy programowych wejść/wyjść.
- Całe wyjście jest umieszczone w rejestrze wyjściowym (OutREG) a CPU odpytuje rejestr wejściowy (InREG) przy „wykryciu wejścia”, w wartość jest kopiowana do akumulatora

# Prosty program (1 z 3)

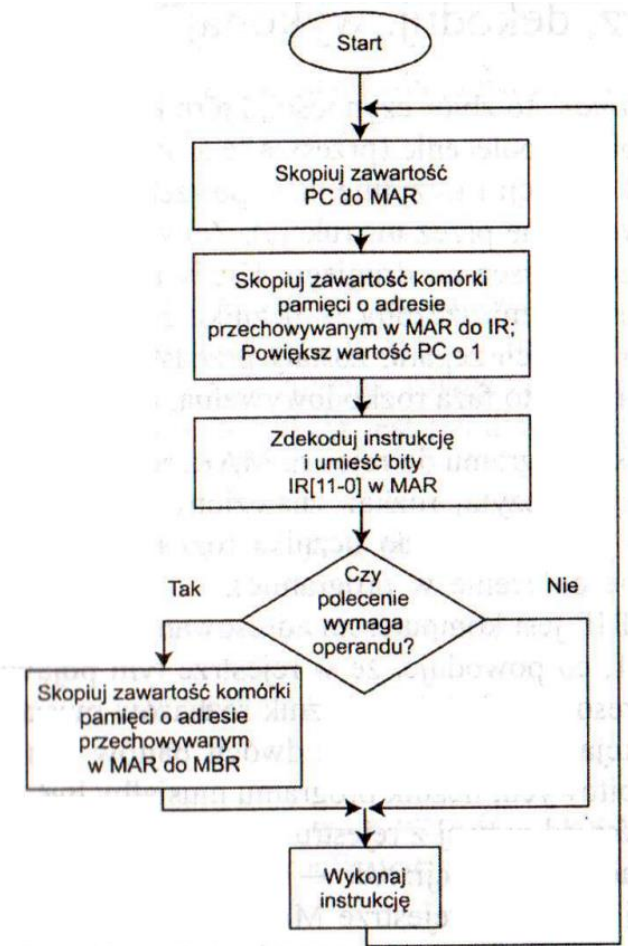
- Rozważmy podany prosty program MARIE jak poniżej.
- Zestaw mnemoników instrukcji przechowywanych pod adresami 0x100 - 0x106 (HEX):

| Address | Instruction | Binary Contents of Memory Address | Hex Contents of Memory |
|---------|-------------|-----------------------------------|------------------------|
| 100     | Load 104    | 0001000100000100                  | 1104                   |
| 101     | Add 105     | 0011000100000101                  | 3105                   |
| 102     | Store 106   | 0100000100000110                  | 4106                   |
| 103     | Halt        | 0111000000000000                  | 7000                   |
| 104     | 0023        | 000000000100011                   | 0023                   |
| 105     | FFE9        | 1111111111101001                  | FFE9                   |
| 106     | 0000        | 0000000000000000                  | 0000                   |

# Prosty program (2 z 3)

- Przyjrzyjmy się, co dzieje się wewnątrz komputera, gdy nasz program zostanie uruchomiony
- To jest instrukcja **LOAD 104**:

| Step             | RTN                | PC  | IR    | MAR   | MBR   | AC    |
|------------------|--------------------|-----|-------|-------|-------|-------|
| (initial values) |                    | 100 | ----- | ----- | ----- | ----- |
| Fetch            | MAR ← PC           | 100 | ----- | 100   | ----- | ----- |
|                  | IR ← M[MAR]        | 100 | 1104  | 100   | ----- | ----- |
|                  | PC ← PC + 1        | 101 | 1104  | 100   | ----- | ----- |
| Decode           | MAR ← IR[11-0]     | 101 | 1104  | 104   | ----- | ----- |
|                  | (Decode IR[15-12]) | 101 | 1104  | 104   | ----- | ----- |
| Get operand      | MBR ← M[MAR]       | 101 | 1104  | 104   | 0023  | ----- |
| Execute          | AC ← MBR           | 101 | 1104  | 104   | 0023  | 0023  |



# Prosty program (3 z 3)

- Nasza druga instrukcja to **ADD 105**

| Step             | RTN                       | PC  | IR   | MAR | MBR  | AC   |
|------------------|---------------------------|-----|------|-----|------|------|
| (initial values) |                           | 101 | 1104 | 104 | 0023 | 0023 |
| Fetch            | $MAR \leftarrow PC$       | 101 | 1104 | 101 | 0023 | 0023 |
|                  | $IR \leftarrow M[MAR]$    | 101 | 3105 | 101 | 0023 | 0023 |
|                  | $PC \leftarrow PC + 1$    | 102 | 3105 | 101 | 0023 | 0023 |
| Decode           | $MAR \leftarrow IR[11-0]$ | 102 | 3105 | 105 | 0023 | 0023 |
|                  | (Decode $IR[15-12]$ )     | 102 | 3105 | 105 | 0023 | 0023 |
| Get operand      | $MBR \leftarrow M[MAR]$   | 102 | 3105 | 105 | FFE9 | 0023 |
| Execute          | $AC \leftarrow AC + MBR$  | 102 | 3105 | 105 | FFE9 | 000C |

# Dyskusja na temat assemblerów (1 z 4)

- Instrukcje mnemoniczne takie jak **LOAD 104** są łatwe do pisania i rozumienia przez ludzi
- Komputery nie są w stanie ich zrozumieć.
- **Asemblery** tłumaczą instrukcje, które są zrozumiałe dla ludzi na język maszynowy zrozumiały dla komputerów
- Zwróćcie uwagę na różnicę między assemblerem a kompilatorem: w assemblerze instrukcja mnemoniczna koresponduje jeden do jednego z jej kodem maszynowy. W przypadku kompilatorów zwykle tak nie jest.

# Dyskusja na temat assemblerów (2 z 4)

- Asemblery tworzą obiekt pliku programu obiektowego z mnemonicznego kodu źródłowego w dwóch przebiegach
- Podczas pierwszego przebiegu assembler montuje tyle programu, ile się da. W tym czasie buduje tablicę symboli, która zawiera referencje do pamięci dla wszystkich symboli w programie
- Podczas drugiego przebiegu instrukcje są uzupełniane wartościami z tablicy symboli

# Dyskusja na temat assemblerów (3 z 4)

- Rozważmy nasz przykładowy – mamy w zestawie dwie dyrektywy HEX i DEC, które określają podstawy stałych
- Pierwsze przejście, tworzy tablicę symboli i częściowo montowane są instrukcje

| Address | Instruction |
|---------|-------------|
| 100     | Load X      |
| 101     | Add Y       |
| 102     | Store Z     |
| 103     | Halt        |
| 104 X,  | DEC 35      |
| 105 Y,  | DEC -23     |
| 106 Z,  | HEX 0000    |

|   |     |
|---|-----|
| X | 104 |
| Y | 105 |
| Z | 106 |

|   |      |
|---|------|
| 1 | X    |
| 3 | Y    |
| 2 | Z    |
| 7 | 0000 |

# Dyskusja na temat asemblerów (4 z 4)

- Po drugim przejściu montaż jest kompletny.

|         |
|---------|
| 1 1 0 4 |
| 3 1 0 5 |
| 2 1 0 6 |
| 7 0 0 0 |
| 0 0 2 3 |
| F F E 9 |
| 0 0 0 0 |

|   |     |
|---|-----|
| X | 104 |
| Y | 105 |
| Z | 106 |

| Address | Instruction |
|---------|-------------|
| 100     | Load X      |
| 101     | Add Y       |
| 102     | Store Z     |
| 103     | Halt        |
| 104 X,  | DEC 35      |
| 105 Y,  | DEC -23     |
| 106 Z,  | HEX 0000    |

# Rozszerzenie zestawu instrukcji

- Jak dotąd wszystkie instrukcje MARIE, które omówiłem używały trybu *adresowania bezpośredniego*
- Oznacza to, że adres operandu jest wyraźnie określony w instrukcji
- Często warto wykorzystać *adresowanie pośrednie* , gdzie adres adresu argumentu podano w instrukcji
- Jeśli kiedykolwiek używaliście wskaźników podczas pisania programów to znacie już adresowanie pośrednie.

# Dyskusja na temat dekodowania (1 z 5)

- Jednostka sterująca komputerem dba o zsynchronizowanie całości procesu, zapewnia, że bity trafiają do prawidłowych komponentów, które to komponenty są potrzebne akurat w tym momencie
- Istnieją dwa ogólne sposoby w jakie jednostka sterująca może być zaimplementowana: sterowanie przewodowe i sterowanie mikroprogramowe.
- Dzięki sterowaniu mikroprogramowemu mały program jest umieszczony w pamięci tylko do odczytu (ROM) w mikrokontrolerze.
- Sterowniki przewodowe implementują ten program za pomocą elementów logiki cyfrowej.

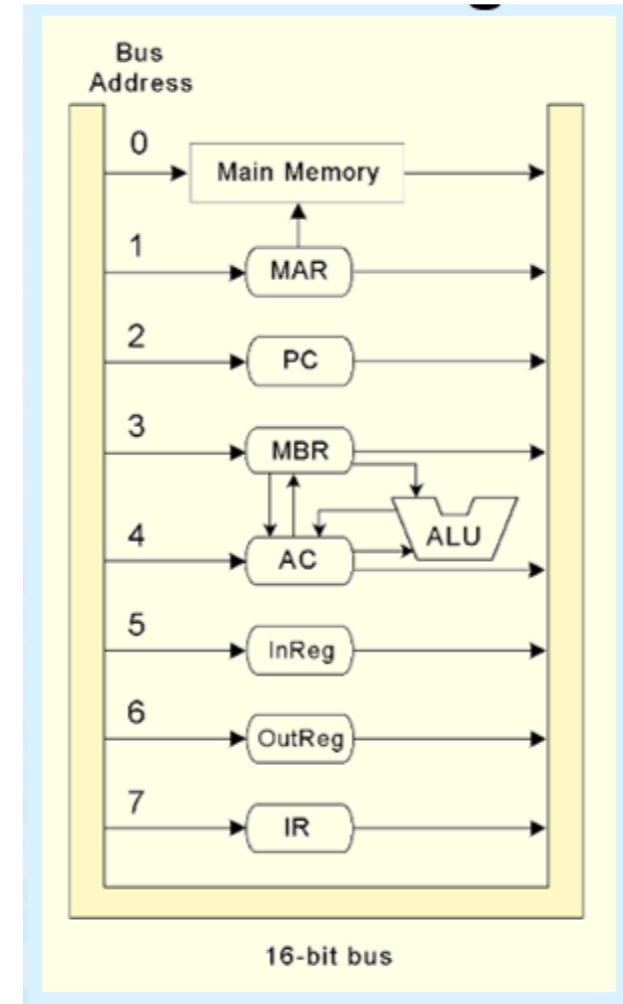
# Dyskusja na temat dekodowania (2 z 5)

- Mikrooperacje podane przez każdy RTL definiują działanie jednostki sterującej MARIE
- Każda mikrooperacja składa się z charakterystycznego wzoru sygnałów, który jest interpretowany przez jednostkę sterującą i skutkuje wykonaniem polecenia
- Przypomnijmy, RTL dla instrukcji **Add** to:

```
MAR ← X  
MBR ← M[MAR]  
AC ← AC + MBR
```

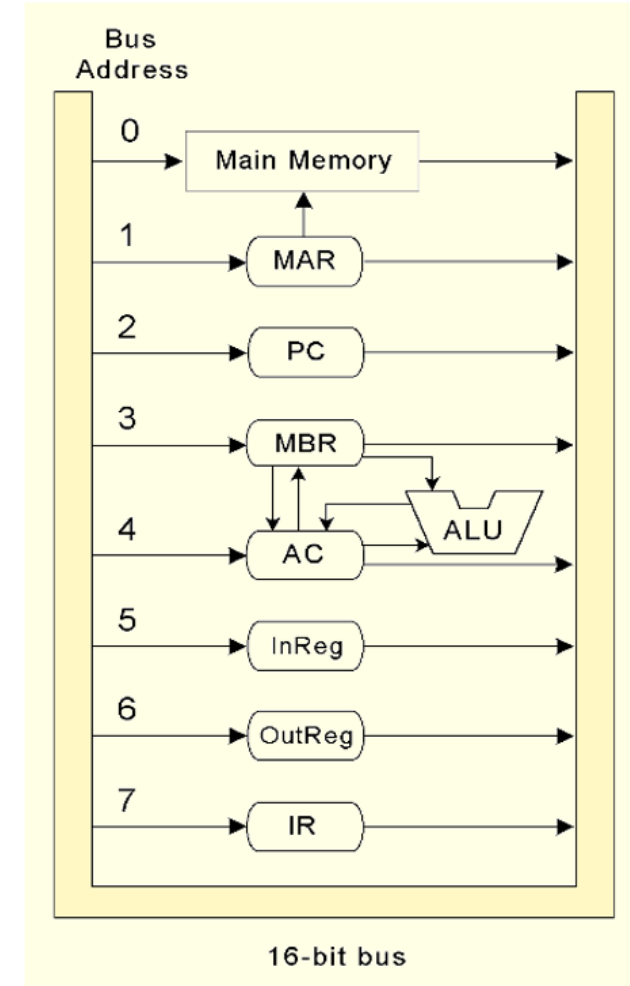
# Dyskusja na temat dekodowania (3 z 5)

- Każdy z elementów MARIE rejestry i pamięć główna ma unikalne adresy wzdłuż ścieżki danych
- Adresy przyjmują formę sygnałów transmitowanych przez jednostkę sterującą
- Ile linii sygnałowych wymagane jest do adresowania elementów MARIE?



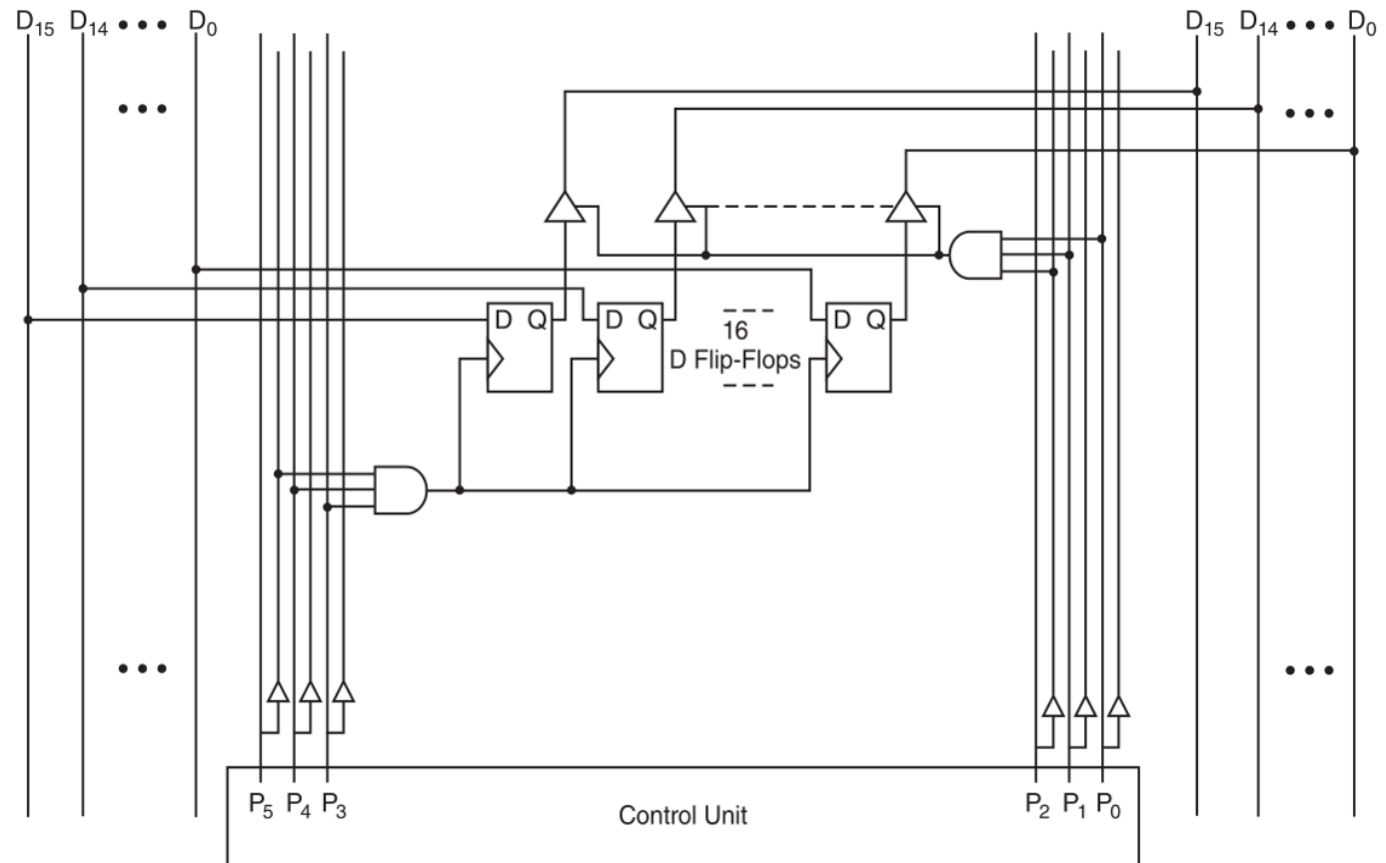
# Dyskusja na temat dekodowania (4 z 5)

- Zdefiniujemy dwa zestawy po trzy linie sygnałowe
- Jeden zestaw: P2 , P1 , P0 steruje odczytem z pamięci lub rejestru, a drugi składający się z P5 , P4 , P3 kontroluje zapis



# Dyskusja na temat dekodowania (5 z 5)

- Widok z bliska na rejestr MBR w MARIE:
- Rejestr MBR jest włączony do odczytu, gdy linie P0 i P1 są w stanie wysokim i można w nim zapisywać, gdy P3 i P4 są w stanie wysokim



# Prawdziwe architektury (1 z 7)

- MARIE dzieli wiele cech z nowoczesnymi architekturami, ale nie jest to dokładne ich przedstawienie
- Na kolejnych slajdach pokrótce przyjrzymy się dwóm architekturom rzeczywistych maszyn:
  - architekturze Intela, zwaną CISC
  - oraz architekturze MIPS, który jest maszyną RISC.
  - CISC to skrót od: komputer o złożonym zestawie instrukcji (*complex instruction set computer*)
  - RISC oznacza komputer o zredukowanej liczbie instrukcji (*reduced instruction set computer*)

# Prawdziwe architektury (2 z 7)

- Klasyczna architektura Intela, 8086, powstała w 1979 roku. **Jest to architektura CISC**
  - Została zaadoptowana przez IBM na potrzeby ich komputera PC, 1981 r.
- 8086 operował na 16-bitowych słowach danych i obsługiwał 20-bitowe adresowanie pamięci
- Później, dla obniżenia kosztów wprowadzono 8-bitowy 8088. Podobnie jak 8086, używał 20 bitów dla adresacji pamięci.

Jaka była największa pamięć, którą 8086 mógł obsłużyć?

# Prawdziwe architektury (3 z 7)

- 8086 miał cztery 16-bitowe rejestry ogólnego przeznaczenia do którego można było uzyskać dostęp za pomocą pół-słów (*half-word*)
- Miał też rejestr flag, rejestr instrukcji i stos - dostępny przez wartości zapisywane w dwóch innych rejestrach
  - wskaźnik bazowy (*base pointer*) i wskaźnik stosu (*stack pointer*)
- 8086 nie miał wbudowanego przetwarzania zmiennoprzecinkowego
- W 1980 roku Intel wypuścił koprocesor 8087, ale niewielu użytkowników zdecydowało się na instalację ze względu na wysoki koszt

# Prawdziwe architektury (4 z 7)

- W 1985 roku Intel wprowadził 32-bitowy 80386
- Też nie miał wbudowanej jednostki zmiennoprzecinkowej
- Model 80486, wprowadzony w 1989 roku, była to wersja 80386 z wbudowanym przetwarzaniem zmiennoprzecinkowe i pamięcią podręczną *cache*
- •80386 i 80486 zapewniały w kompatybilność wstecz z 8086 i 8088.
- Oprogramowanie napisane dla systemów wykorzystujących krótsze słowa mogło działać dzięki wykorzystaniu niższych 16 bitów w 32-bitowych rejestrach

# Prawdziwe architektury (5 z 7)

- Intel Pentium 4 wprowadził zupełnie nową architekturę **NetBurst**
- Funkcje zwiększające prędkość obejmują:
  - Wielowątkowość (Hyperthreading)
  - Hiperpotokowanie (Hyperpipelining)
  - Execution trace cache - pamięć podręczna śledząca wykonywanie poleceń (przechowuje zdekodowane instrukcje dla ich możliwego ponownego użycia) wielopoziomowa pamięć podręczna i potokowanie instrukcji.
- Firma Intel, wraz z wieloma innymi, zaadoptowała wiele z rozwiązań architektur RISC do mikroprocesorów, które są w dużej mierze CISC

# Prawdziwe architektury (6 z 7)

- W klasie procesorów RISC jedną z rodzin procesorów, które odniosły sukces rynkowy jest MIPS
- pierwszy procesor MIPS zaprezentowano w roku 1986
- Miał 32-bitowy rozmiar słowa i mógł adresować 4 GB pamięci.
- Przez lata procesory MIPS były używane w komputerach ogólnego przeznaczenia
- Obecnie architektura MIPS oferuje teraz wersje 32 i 64 bitowe

# Prawdziwe architektury (7 z 7)

- MIPS był jednym z pierwszych mikroprocesorów RISC.
- Oryginalna architektura MIPS miała tylko 55 różnych instrukcji (8086, miał ponad 100)
- MIPS został zaprojektowany z myślą o wydajności: jest to architektura typu załaduj/przechowaj, co oznacza, że tylko instrukcje ładowania i przechowywania mają dostęp do pamięci
- Duża liczba rejestrów w architekturze MIPS ogranicza przesył danych magistralą do minimum

# Podsumowanie (1 z )

- Główne elementy systemu komputerowego to jego jednostka sterująca, rejestry, pamięć, ALU i magistrala danych
- Wbudowany zegar synchronizuje wszystko
- Jednostki sterujące mogą być mikroprogramowane lub „przewodowe” (*hardwired*, połączone na sztywno).
- jednostki sterujące „przewodowe” oferują lepszą wydajność, podczas gdy jednostki mikroprogramowane są lepiej przystosowane do zmian

# Podsumowanie (2 z 2)

- Komputery uruchamiają programy w sposób iteracyjny w cyklu pobierz-dekoduj-wykonaj
- Komputery mogą uruchamiać programy, które są jedynie w języku maszynowym.
- Asembler konwertuje kod mnemoniczny na język maszyny.
- Architektura Intela jest przykładem CISC
- MIPS jest przykładem RISC

- Koniec wykładu

# Architektura Systemów Komputerowych

Bliższe spojrzenie na **architektury zbioru rozkazów (ISA)**

Dr inż. Krzysztof Kosmowski

# Cele wykładu

- Pokazanie czynników wpływających na architektury zbiorów rozkazów procesora
- Zapoznanie z trybami adresowania pamięci
- Przedstawienie koncepcji przetwarzania potokowego (*pipeline*) na poziomie zbioru rozkazów

# WSTĘP

- Ten wykład opiera się na informacjach z wykładu3
- Przedstawione zostanie szczegółowe spojrzenie na różne formaty instrukcji, typy operandów i metody dostępu do pamięci
- Omówiona zostanie współzależność między organizacją komputera i formatami instrukcji

# Formaty instrukcji (1 z 26)

- Zbiory rozkazów różnią się ze względu na:
  - Liczbę bitów na instrukcję
  - Sposobem przechowywania danych w CPU (w oparciu o stos lub w oparciu o rejestry)
  - Liczbę jawnych operandów na instrukcję.
  - Lokalizacje operandu.
  - Rodzaje operacji.
  - Rodzaj i wielkość operandów.

# Formaty instrukcji (2 z 26)

- ISA mogą być porównywane pod względem:
  - Wielkości Pamięci głównej zajmowanej przez program.
  - Złożoność instrukcji.
  - Długość instrukcji (w bitach).
  - Całkowita liczba instrukcji

# Formaty instrukcji (3 z 26)

- Podczas projektowania zbioru instrukcji bierze się pod uwagę:
  - Długość instrukcji
    - krótkie, długie czy zmienne.
  - Liczba operandów
  - Liczba rejestrów adresowalnych
  - Organizacja pamięci
  - Czy adresowalny jest bajt czy słowo
  - Tryby adresowania
    - bezpośrednio, pośrednio lub indeksowane
    - Wszystkie ww., czy jeden z nich?

# Formaty instrukcji (4 z 26)

- Kolejną ważną kwestią jest kolejność bajtów, czyli „*endianowość*” rozpatrywanej architektury
- Praktycznie we wszystkich współczesnych komputerach pamięć adresowana jest bajtowo, co oznacza że musi istnieć standard opisujący adresowanie i przechowywanie danych zawierających więcej niż jeden bajt
- Jeśli mamy dwubajtową liczbę całkowitą, może ona być przechowywana tak, że:
  - najmniej znaczący bajt następuje po najbardziej znaczącym bajcie
  - lub odwrotnie.
- W maszynach ***little endian*** najmniej znaczący bajt następuje po najbardziej znaczącym bajcie
- Maszyny ***big endian*** przechowują najważniejszy bajt pierwszy (pod niższym adresem)

# Formaty instrukcji (5 z 26)

- Jako przykład założmy, że mamy liczbę szesnastkową 0x12345678.
- układy bajtów ***Big Endian*** i ***Little Endian*** pokazane są poniżej:

| Address →     | 00 | 01 | 10 | 11 |
|---------------|----|----|----|----|
| Big Endian    | 12 | 34 | 56 | 78 |
| Little Endian | 78 | 56 | 34 | 12 |

# Formaty instrukcji (6 z 26)

- Większy przykład:
  - komputer używa 32-bit liczb całkowitych
  - Wartości:
    - 0xABCD1234,
    - 0x00FE4321
    - 0x10

mogłyby być przechowywane sekwencyjnie w pamięci, zaczynając od adresu 0x200:

| Address | Big Endian | Little Endian |
|---------|------------|---------------|
| 0x200   | AB         | 34            |
| 0x201   | CD         | 12            |
| 0x202   | 12         | CD            |
| 0x203   | 34         | AB            |
| 0x204   | 00         | 21            |
| 0x205   | FE         | 43            |
| 0x206   | 43         | FE            |
| 0x207   | 21         | 00            |
| 0x208   | 00         | 10            |
| 0x209   | 00         | 00            |
| 0x20A   | 00         | 00            |
| 0x20B   | 10         | 00            |

# Formaty instrukcji (7 z 26)

- Big Endian:
  - Jest bardziej naturalny.
  - Znak liczby można określić patrząc na bajt pod danym adresem z offsetem 0.
  - Łańcuchy i liczby całkowite są przechowywane w tej samej kolejności.
- Small Endian:
  - Ułatwia umieszczanie wartości których granice nie są równe wielkości słowa
  - Konwersja z 16-bitowego adresu (całkowitego) na adres 32-bitowy nie wymaga żadnej arytmetyki.

# Formaty instrukcji (8 z 26 )

- Kolejne rozważania dotyczą sposobu przechowywania danych przez procesor.
- Mamy trzy możliwości:
  1. Architektura oparta o stos
  2. Architektura oparta o akumulator
  3. Architektura oparta o rejestry ogólnego przeznaczenia
- Przy wyborze kompromisem jest: prostota (i koszt) projektowania hardware'u vs. szybkość działania i łatwość obsługi.

# Formaty instrukcji (9 z 26)

- W architekturze stosu
  - instrukcje i operandy są niejawnie pobierane ze stosu
  - Nie można losowo uzyskać dostępu do stosu
- W architekturze opartej o akumulator
  - jeden argument operacji binarnej jest „niejawny” w akumulatorze
  - drugi operand jest w pamięci, co powoduje duży ruch na magistrali
- W architekturze rejestrów ogólnego przeznaczenia (GPR)
  - rejestry mogą być używane zamiast pamięci
  - szybszy niż architektura akumulatorowa
  - wydajna implementacja dla kompilatorów
  - powoduje dłuższe instrukcje

# Formaty instrukcji (10 z 26 )

- Większość dzisiejszych systemów to systemy GPR
- Istnieją trzy typy:
  - **Pamięć-Pamięć**, gdzie dwa lub trzy operandy mogą znajdować się w pamięci
  - **Pamięć-Rejestr**, co najmniej jeden operand musi być w rejestrze
  - **Załaduj-Przechowaj** (*Load-Store*), w pamięci nie mogą znajdować się żadne operandy.
- Liczba operandów i liczba dostępnych rejestrów mają bezpośredni wpływ na długość instrukcji.

# Formaty instrukcji (11 z 26)

- Maszyny wykorzystujące **stos** używają *jedno- i zero-argumentowe* instrukcje
- Instrukcje LOAD i STORE wymagają jednego operandu w postaci adresu pamięci
- Inne instrukcje używają operandów pobieranych ze stosu w sposób niejawny
  - Operacje PUSH i POP obejmują wykonują operacje wyłącznie na górny elemencie stosu
    - PUSH X – umieszcza wartość komórki pamięci o adresie X na szczycie stosu
    - POP X – zdejmuje wartość ze szczytu stosu i umieszcza w komórce pamięci o adresie X
  - Instrukcje binarne (np. ADD , MULT ) używają dwóch elementów zapisanych na górze stosu

# Formaty instrukcji (12 z 26 )

- Architektury stosowe wymagają od nas myślenia o wyrażeniach arytmetycznych w nieco inny sposób
- Jesteśmy przyzwyczajeni do pisania wyrażeń za pomocą notacji *infix*, takiej jak:  **$Z = X + Y$**
- Arytmetyka stosu wymaga użycia notacji *postfix*:  **$Z = XY+$**
- Nazywa się to również odwrotną notacją polską, na cześć jej polskiego wynalazcy, Jan Łukasiewicz (1878 - 1956)

# Formaty instrukcji (13 z 26)

- Główna zaleta notacji *postfix* jest to, że nie używa się nawiasów.

- Na przykład wyrażenie *infix*:

$$Z = (X+T) \times (W+U)$$

- staje się wyrażeniem:

$$Z = X \ Y+W \ U + x \quad \text{w notacji } \textit{postfix}$$

- |         |                         |
|---------|-------------------------|
| • $X+Y$ | – notacja wrostkowa     |
| • $+XY$ | – notacja przedrostkowa |
| • $XY+$ | – notacja przyrostkowa  |

# Formaty instrukcji (14 z 26)

- Przykład:

Skonwertuj wyrażenie wrostkowe **(2+3) - 6/3** do notacji przyrostkowej:

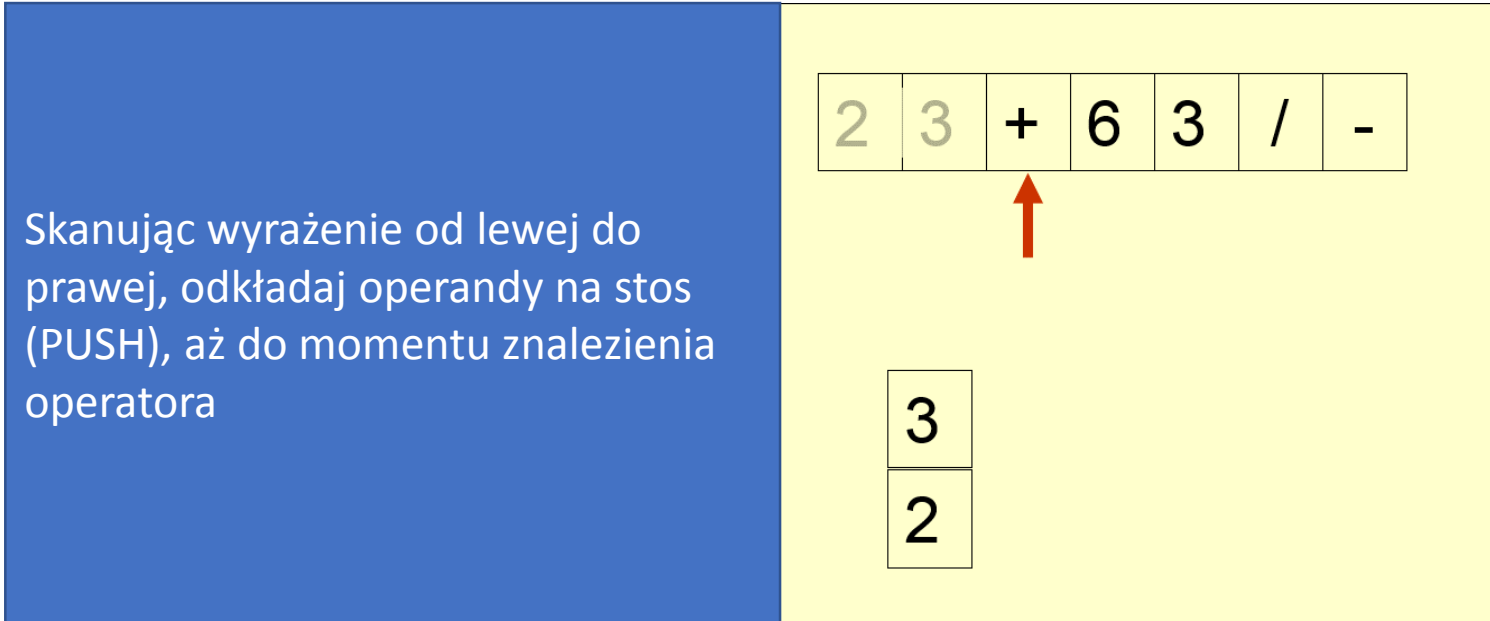
$$(2+3) - 6/3$$

- operacja **(2+3)** ma **pierwszeństwo** – zastępujemy je wyrażeniem **2 3 +**
- następnie pierwszeństwo ma operator dzielenia **6/3** zastępujemy je **6 3/**
- Iloraz 6/3 jest **odejmowany** od sumy 2+3 więc na końcu stawiamy operator odejmowania: **-**

- W rezultacie: **2 3 + 6 3/ -**

# Formaty instrukcji (15 z 26)

- Przykład: użyj stosu, aby obliczyć wyrażenie przyrostkowe **2 3 + 6 3 / -**



# Formaty instrukcji (16 z 26)

- Przykład: użyj stosu, aby obliczyć wyrażenie przyrostkowe **2 3 + 6 3 / -**

Zdejmij argumenty (POP) i wykonaj operację  
wskazaną przez operator (ADD).  
Umieść wynik z powrotem na stosie (PUSH)

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 3 | + | 6 | 3 | / | - |
|---|---|---|---|---|---|---|



|   |
|---|
| 5 |
|---|

# Formaty instrukcji (17 z 26)

- Przykład: użyj stosu, aby obliczyć wyrażenie przyrostkowe **2 3 + 6 3 / -**

Skanuj wyrażenie od lewej do prawej,  
odkładaj operandy na stos (PUSH), aż do  
momentu znalezienia kolejnego operatora

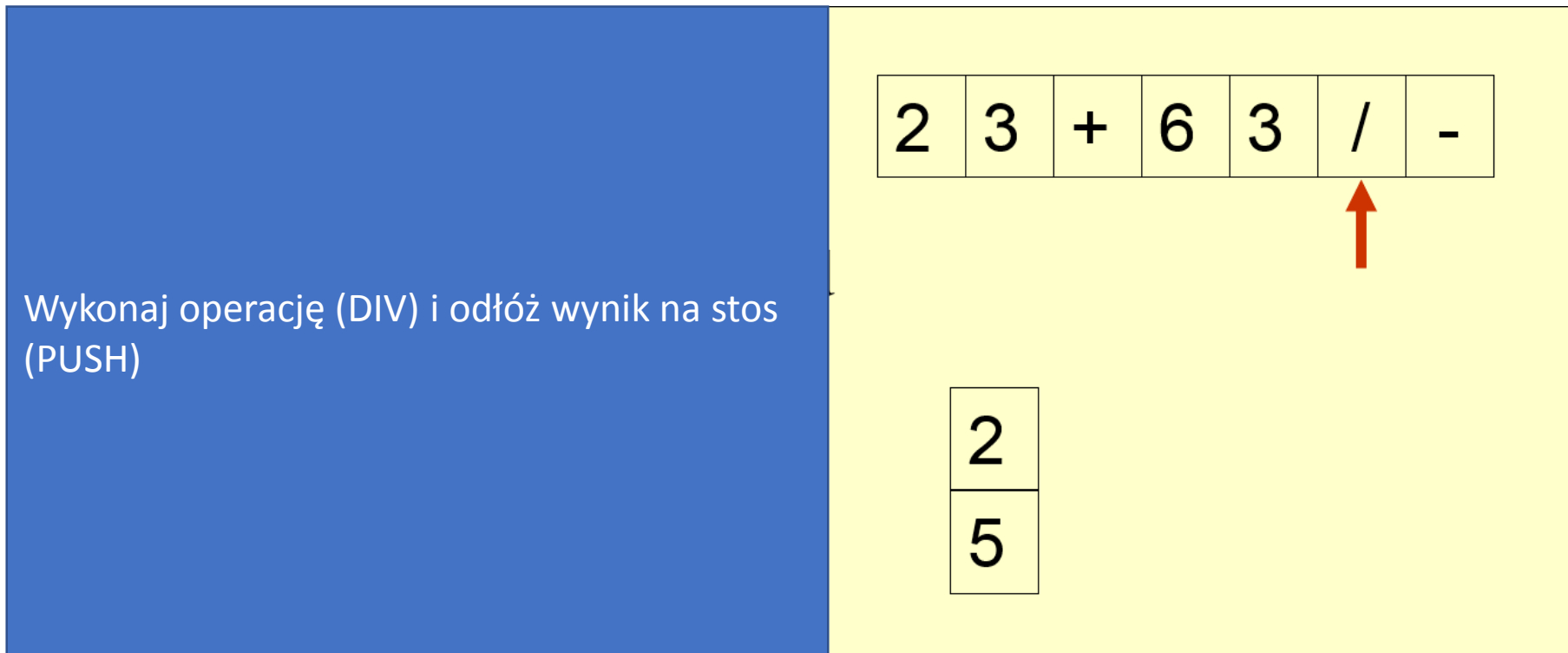
|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 3 | + | 6 | 3 | / | - |
|---|---|---|---|---|---|---|



|   |
|---|
| 3 |
| 6 |
| 5 |

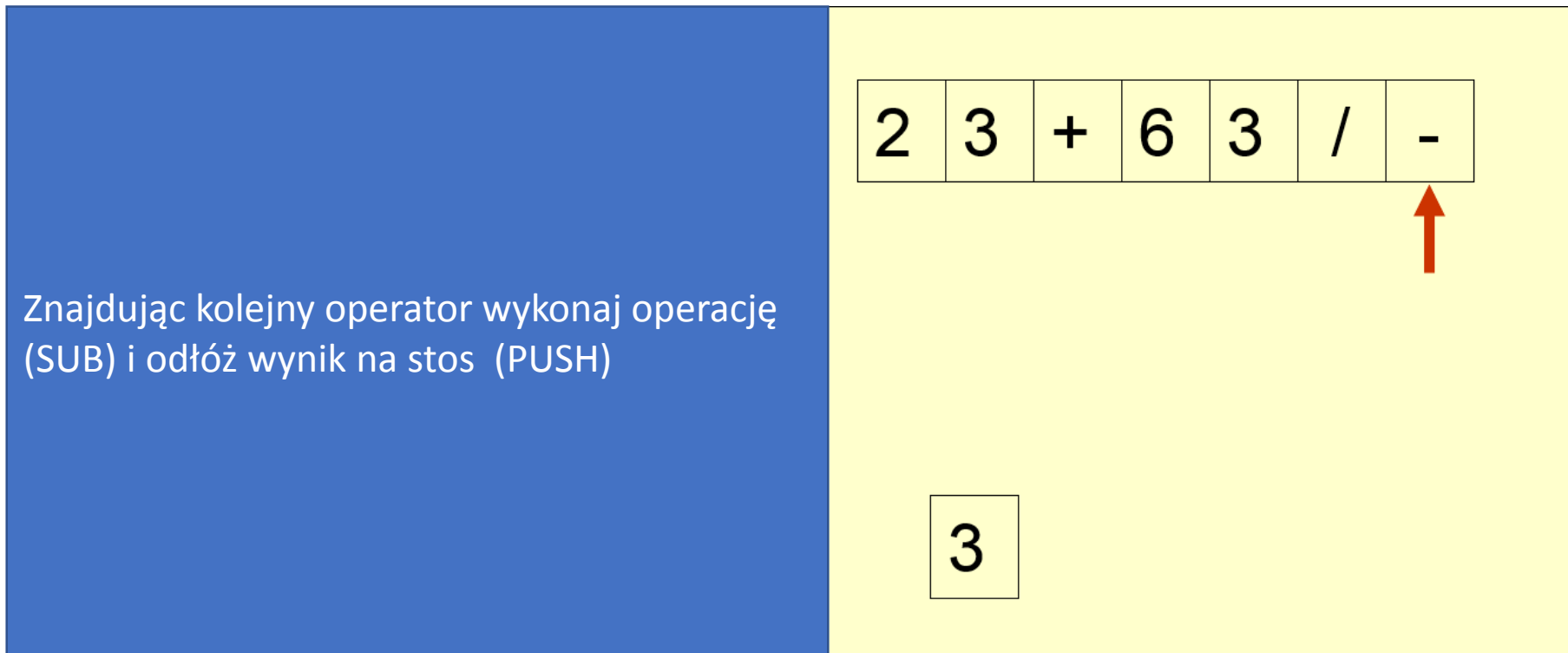
# Formaty instrukcji (18 z 26)

- Przykład: użyj stosu, aby obliczyć wyrażenie przyrostkowe **2 3 + 6 3 / -**



# Formaty instrukcji (19 z 26)

- Przykład: użyj stosu, aby obliczyć wyrażenie przyrostkowe **2 3 + 6 3 / -**



# Formaty instrukcji (20 z 26)

Zobaczmy, jak obliczyć wyrażenie wrostkowe  $Z = (X*Y)+(W*U)$  przy użyciu różnych formatów instrukcji

- **ISA z trzema operandami** - przynajmniej jeden z nich musi wskazywać jakiś rejestr, zazwyczaj pierwszy z nich określa miejsce gdzie należy umieścić wynik
  - Przykładowy program mógłby wyglądać tak:

```
MULT    R1 , X , Y
MULT    R2 , W , U
ADD     Z , R1 , R2
```

# Formaty instrukcji (21 z 26)

Zobaczmy, jak obliczyć wyrażenie wrostkowe  $Z = (X*Y)+(W*U)$  przy użyciu różnych formatów instrukcji

- **W przypadku instrukcji dwuadresowych** – zazwyczaj pierwsza adres wskazuje, zaś drugi albo rejestr albo komórkę pamięci
- Przy takim formacie nasz program ma postać:

```
LOAD R1,X
MULT R1,Y
LOAD R2,W
MULT R2,U
ADD R1,R2
STORE Z,R1
```

# Formaty instrukcji (22 z 26)

Zobaczmy, jak obliczyć wyrażenie wrostkowe  $Z = (X*Y)+(W*U)$  przy użyciu różnych formatów instrukcji

- **Przy użyciu instrukcji jednoadresowych\*** - konieczne jest założenie, że adres docelowy będzie zawsze przechowywany w którymś rejestrze (zazwyczaj w akumulatorze):

```
LOAD X
MULT Y
STORE TEMP
LOAD W
MULT U
ADD TEMP
STORE Z
```

\* MARIE posiada ISA jednoadresowe

# Formaty instrukcji (23 z 26)

W architekturze stosowej wyrażenie przyrostkowe:

$$Z = X \times Y + W \times U$$

mogłoby być wykonane za pomocą programu:

**PUSH X**

**PUSH Y**

**MULT**

**PUSH W**

**PUSH U**

**MULT**

**ADD**

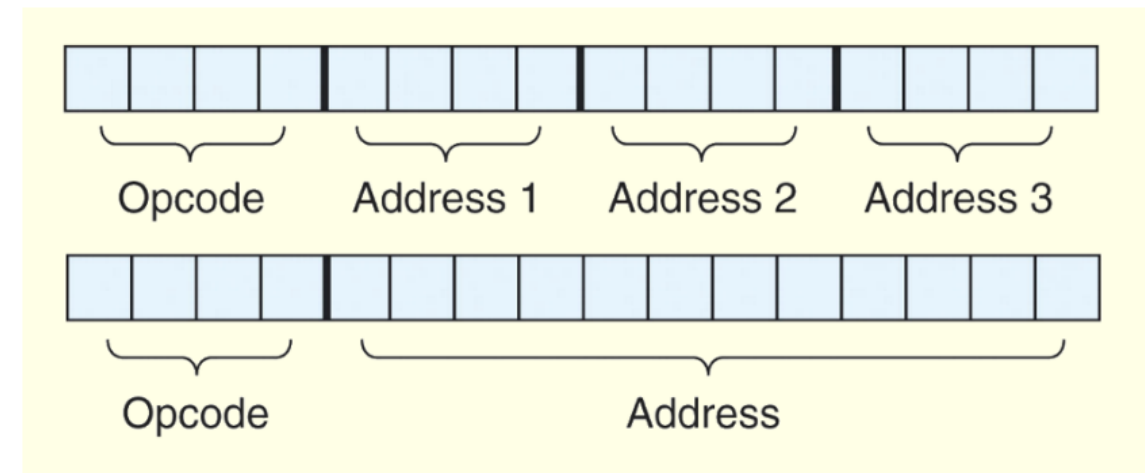
**POP Z**

# Formaty instrukcji (24 z 26)

- Widzieliśmy, jak długość instrukcji jest związana z liczbą operandów wspieranych przez ISA
- W każdym zestawie instrukcji, nie wszystkie instrukcje wymagają takiej samej liczby operandów.
- Operacje niewymagające operandów, np. takie jak HALT powodują marnowanie przestrzeni (gdy stosowane są instrukcje o stałej długości)
- Jednym ze sposobów na odzyskanie części tej przestrzeni jest użycie rozszerzalnych kodów operacji

# Formaty instrukcji (25 z 26 )

- System ma 16 rejestrów i 4K pamięci:
- Aby uzyskać dostęp do rejestrów potrzebujemy 4 bitów.
- Potrzebne jest również 12 bitów do adresacji pamięci.
- Jeśli system ma mieć 16-bitowe instrukcje, mamy dwie opcje:



# Formaty instrukcji (26 z 26 )

- Jeśli pozwolimy, aby długość kodu operacji mogła być różna, możemy stworzyć bardzo bogaty zestaw instrukcji:

|                                |   |                        |
|--------------------------------|---|------------------------|
| 0000 R1 R2 R3                  | } | 15 three-address codes |
| ...                            |   |                        |
| 1110 R1 R2 R3                  |   |                        |
| 1111 - escape opcode           |   |                        |
| 1111 0000 R1 R2                | } | 14 two-address codes   |
| ...                            |   |                        |
| 1111 1101 R1 R2                |   |                        |
| 1111 1110 - escape opcode      |   |                        |
| 1111 1110 0000 R1              | } | 31 one-address codes   |
| ...                            |   |                        |
| 1111 1111 1110 R1              |   |                        |
| 1111 1111 1111 - escape opcode |   |                        |
| 1111 1111 1111 0000            | } | 16 zero-address codes  |
| ...                            |   |                        |
| 1111 1111 1111 1111            |   |                        |

# Rodzaje rozkazów

- Instrukcje dzielą się na kilka szerokich kategorii. Standardowo są to:
  - Przenoszące dane
  - Arytmetyczne
  - Logiczne
  - Manipulacja bitami
  - We/Wy
  - Przekazujące sterowanie
  - Specjalnego przeznaczenia

# Adresowanie (1 z 5)

- Tryby adresowania określają, gdzie operand jest zlokalizowany
- Mogą określać stałą, rejestr lub lokalizację w pamięci
- Rzeczywista lokalizacja operandu to jego adres skuteczny
- Niektóre tryby adresowania pozwalają nam określić adres operandu dynamicznie

# Adresowanie (2 z 5)

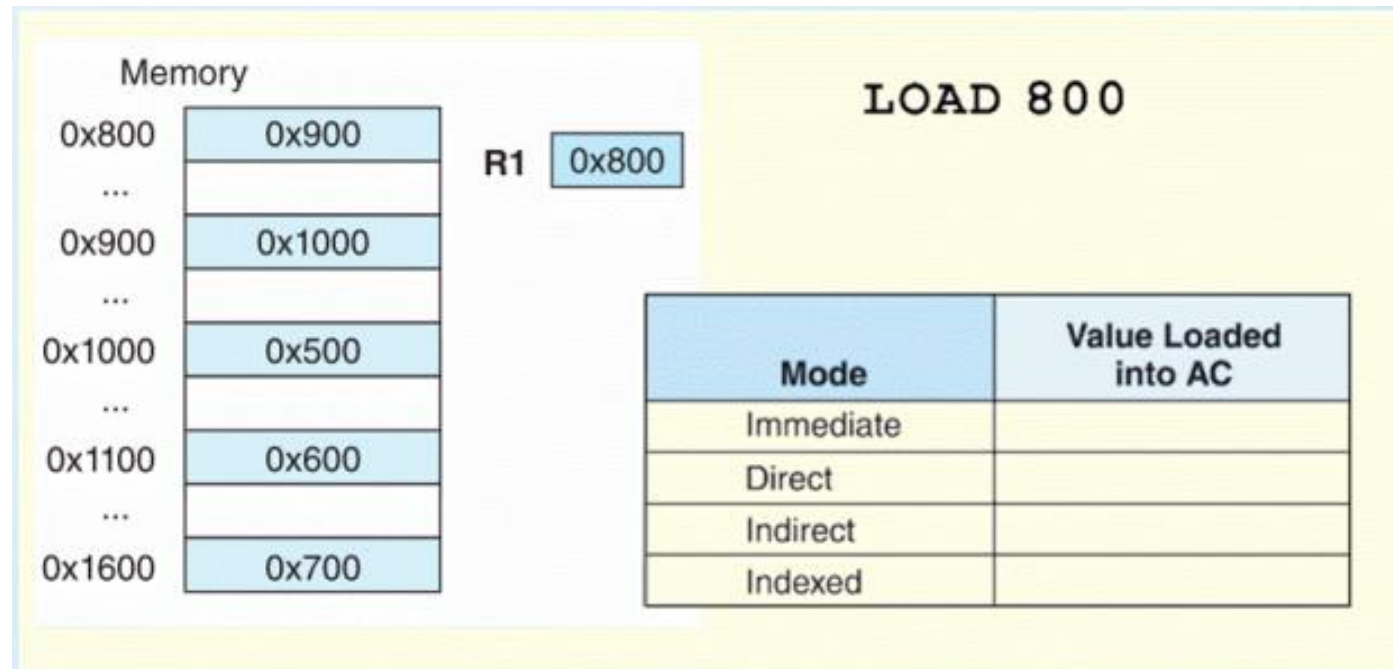
- **Adresowanie natychmiastowe** - ma miejsce, gdy dane są częścią instrukcji,
  - LOAD 008 – oznacza, że do akumulatora (AC) zostanie wpisana wartość liczbową 8
- **Adresowanie bezpośrednie** - adres danych podano w instrukcji,
  - LOAD 008 – oznacza, że do AC zostanie wpisana wartość odnaleziona w pamięci pod adresem 008
- **Adresowanie rejestrowe** – jw. , zamiast adresu pamięci wskazywany jest rejestr
- **Adresowanie pośrednie** - podaje adres komórki, w której zapisany jest adres danych
  - LOAD 008 – oznacza, że do AC zostanie wpisana wartość odnaleziona w pamięci, której adres zapisano pod adresem 008
- **adresowanie pośrednie rejestrowe** – jw. z wykorzystaniem rejestrów

# Adresowanie (3 z 5)

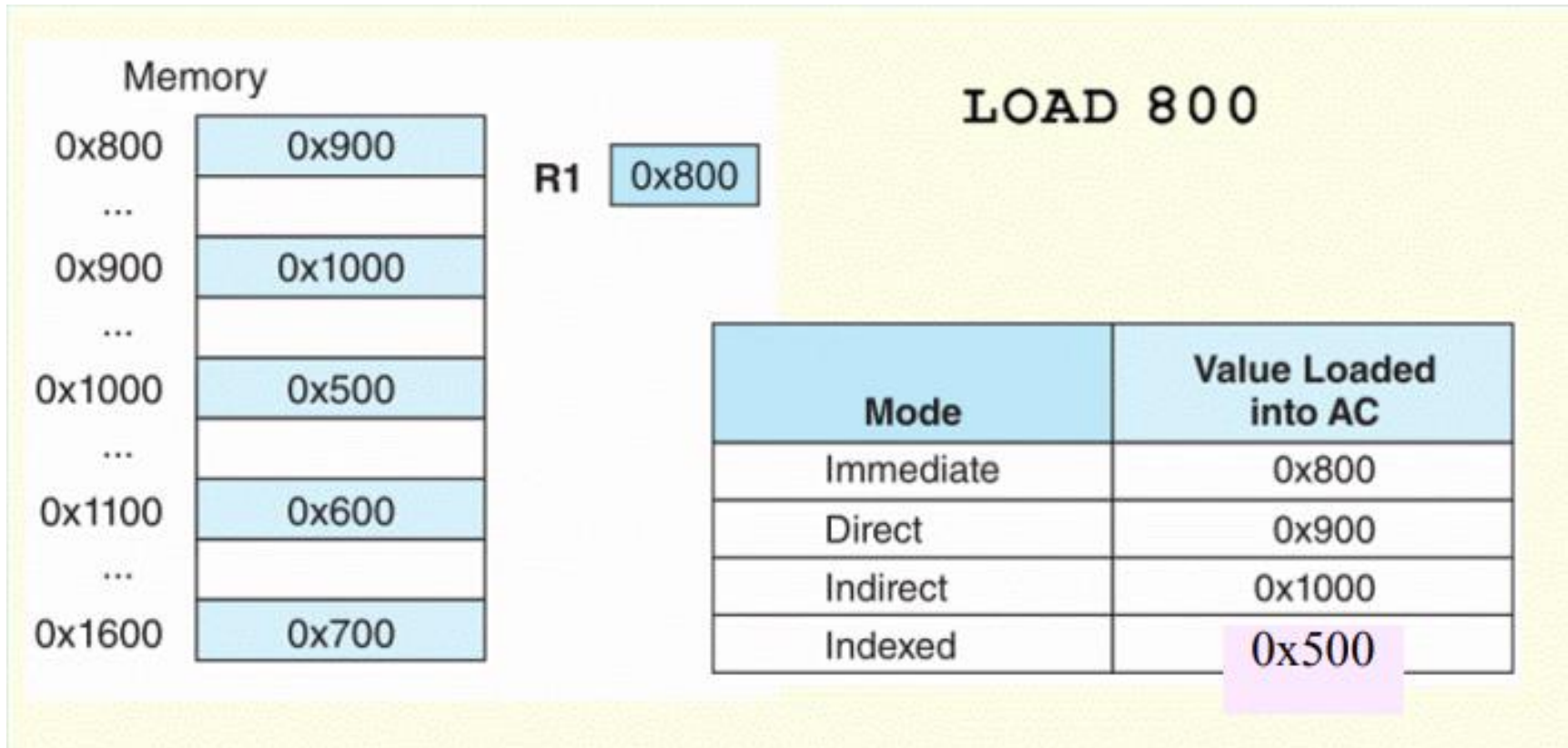
- *Adresowanie indeksowe* wykorzystuje rejestr indeksowy (w sposób ukryty lub jawnie) jako przesunięcie (offset), które jest dodawane do adresu w operandzie w celu określenia efektywnego adresu danych
- *Adresowanie względem bazy* oparte jest podobne, z tym wyjątkiem, że zamiast rejestru indeksowego jest używany rejestr bazowy
- Różnica między tymi dwoma polega na tym, że rejestr indeksowy przechowuje przesunięcie względem podanego adresu w instrukcji, natomiast rejestr bazowy zawiera adres bazowy, gdzie pole adresu reprezentuje offset względem tej bazy
- *Adresowanie stosowe* - zakłada się, że operand znajduje się na wierzchu stosu

# Adresowanie (4 z 5)

- Spójrzmy na przykład głównych trybów adresowania
- Jaka jest wartość załadowana do akumulatora dla każdego z trybów adresowania?



# Adresowanie (5 z 5)



# Potokowość na poziomie instrukcji (1 z 7)

- Niektóre procesory dzielą cykl pobierz – dekoduj-wykonaj na mniejsze kroki
- Te mniejsze kroki często można wykonać równolegle w celu zwiększenia przepustowości
- Takie równoległe wykonywanie nazywa się potokowością na poziomie instrukcji

# Potokowość na poziomie instrukcji (2 z 7)

- Załóżmy, że cykl pobierania-dekodowania-wykonania został rozbity na następujące mniejsze kroki:
  - 1. Pobierz instrukcję
  - 2. Odkoduj kod operacji
  - 3. Oblicz efektywny adres argumentów
  - 4. Pobierz operandy
  - 5. Wykonaj instrukcję
  - 6. Zapisz wynik
- Załóżmy, że mamy sześciostopniowy potok. S1 pobieranie instrukcji, S2 dekoduje ją, S3 określa adres operandów, S4 pobiera je, S5 wykonuje instrukcję, a S6 przechowuje wynik.

# Potokowość na poziomie instrukcji (3 z 7)

- Dla każdego cyklu zegara wykonywany jest jeden mały krok a etapy zachodzą na siebie



S1. Fetch instruction.  
S2. Decode opcode.  
S3. Calculate effective  
address of operands.

S4. Fetch operands.  
S5. Execute.  
S6. Store result.

# Potokowość na poziomie instrukcji (4 z 7)

- Teoretyczne przyspieszenie oferowane przez potokowość może być wyrażona w następujący sposób:
- Niech  **$tp$**  będzie czasem pojedynczego kroku (czas trwania taktu zegara).
- Pierwsze zadanie (instrukcja) wymaga  **$k \times tp$**  czasu na wykonanie w  $k$ -etapowym potoku.
- Pozostałe  $(n - 1)$  zadań wychodzi z potoku w tempie jedno na cykl. Więc łączny czas na wykonanie pozostałych zadań to  **$(n - 1) \times tp$** .
- Tak więc, wykonanie  **$n$**  zadań za pomocą potoku  $k$ -etapowego wymaga:  **$(k \times tp) + (n - 1)tp = (k + n - 1)tp$**

# Potokowość na poziomie instrukcji (5 z 7)

- Jeśli weźmiemy czas potrzebny na wykonanie  $n$  zadań bez potoku i podzielimy przez czas potrzebny do wykonania  $n$  zadań za pomocą potoku, to:

$$\text{Speedup } S = \frac{n t_n}{(k + n - 1) t_p}$$

- Granica dla  $n$  dążącego do nieskończoności:  $(k + n - 1)$  zbliża się do  $n$ , co skutkuje teoretycznym przyspieszeniem:

$$\text{Speedup } S = \frac{k t_p}{t_p} = k$$

# Potokowość na poziomie instrukcji (6 z 7)

- Nasze równania zakładają wielu rzeczy:
- Po pierwsze, że architektura obsługuje pobieranie instrukcji i danych w sposób równoległy
- Po drugie, zakładamy, że potok może być utrzymywany w stanie „napełnionym” przez cały czas - nie zawsze tak jest
- Powstają konflikty, które wymagają rozwiązywania

# Potokowość na poziomie instrukcji (7 z 7)

- Konflikt potoków może wystąpić na skutek:
  - Konflikty zasobów
  - Zależności danych.
  - Warunkowe instrukcje skoku.
- W celu zmniejszenia skutków tych zagrożeń środki zaradcze można podejmować na poziomie oprogramowania jak również na poziomie sprzętowym, ale nie mogą być całkowicie wyeliminowane.

# Rzeczywiste przykłady ISA (1 z 8)

- Wracamy do architektury Intelu i MIPS z ostatniego wykładu,
- Firma Intel wprowadziła potokowanie do swojej linii procesorów Pentium
- Pierwszy Pentium miał dwa pięciostopniowe potoki. Każdy kolejny procesor miał dłuższy potok niż jego poprzednik. Pentium IV posiada potok 24-stopniowy
- Itanium (IA-64) ma potok 10-stopniowy.

# Rzeczywiste przykłady ISA (2 z 8)

- Procesory Intel obsługują szeroki zakres trybów adresowania
- Oryginalny 8086 zapewniał 17 sposobów adresowania pamięci, większość z nich to warianty metod przedstawionych na tym wykładzie
- Ze względu na potrzebę zapewnienia kompatybilności wstecznej, procesory Pentium (i konsekwentnie późniejsze) obsługują również te 17 trybów adresowania
- Itanium, posiadający rdzeń RISC, obsługuje tylko jeden tryb: pośrednie adresowanie rejestrowe.

# Rzeczywiste przykłady ISA (3 z 8)

- MIPS był akronimem od Microprocessor Bez Zablokowanych Etapów Potoku (*Microprocessor Without Interlocked Pipeline Stages*).
- Architektura MIPS jest typu Little Endian i jest adresowalna instrukcjami z trzema adresami o stałej długości
- Podobnie jak Intel, rozmiar potoku procesorów MIPS rósł: R2000 i R3000 posiadają potoki 5-stopniowe; R4000 i R4400 posiada potoki 8-stopniowe

# Rzeczywiste przykłady ISA (4 z 8)

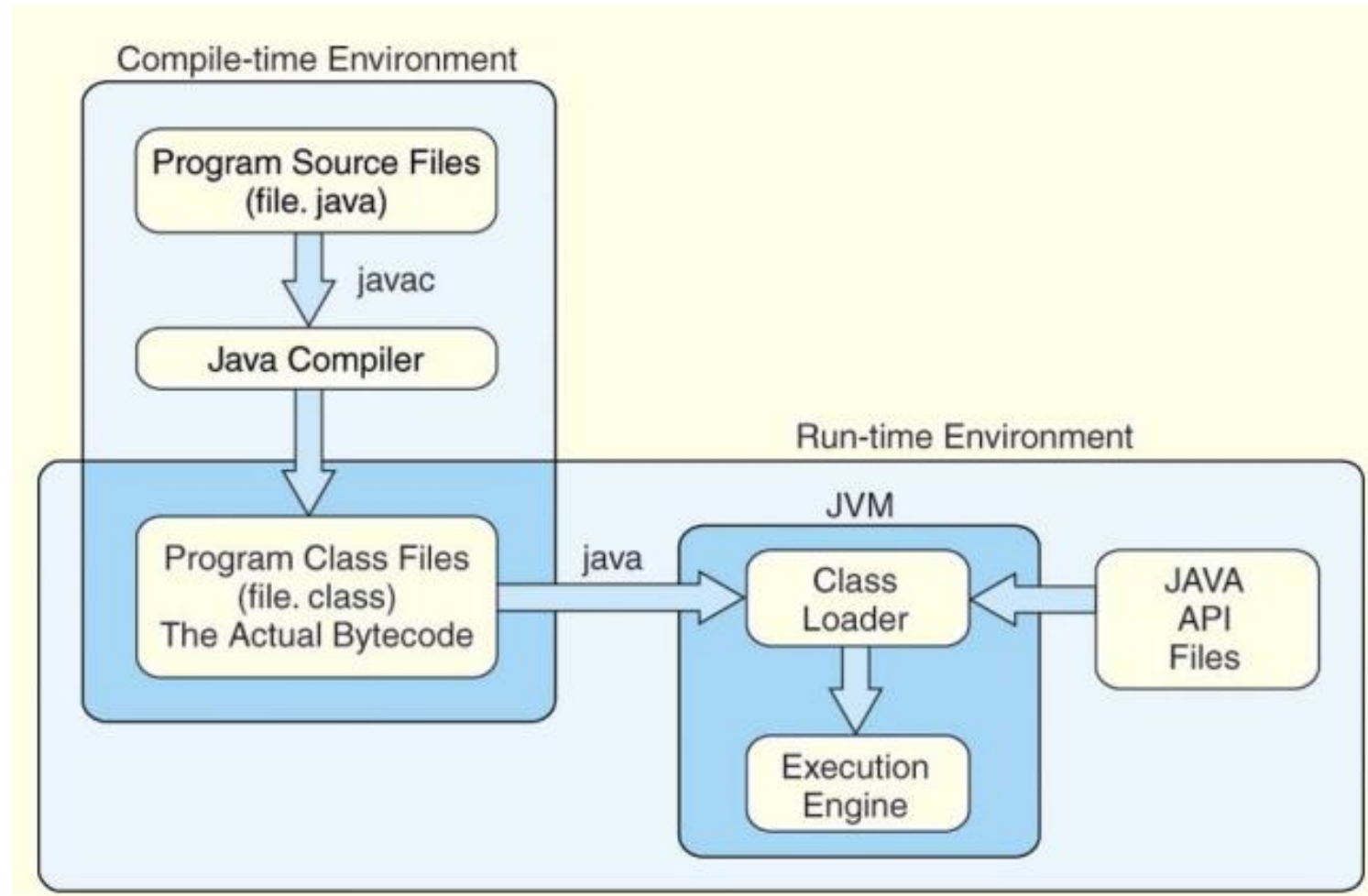
- R10000 ma 3 potoki:
  - 5-stopniowy potok dla instrukcji *integer* (całkowitych),
  - 7-stopniowy potok dla instrukcji *floating-point* (zmiennoprzecinkowych)
  - 6-stopniowy potok dla instrukcji LOAD/STORE.
- We wszystkich ISA procesorów MIPS operacje tylko LOAD i STORE mają dostęp do pamięci.
- ISA MIPS używa tylko trybu adresowania bazowego.

\*Wikipedia: Procesory MIPS stanowiły do roku 2007 jednostkę centralną komputerów firmy SGI. Ponadto są szeroko stosowane w systemach wbudowanych (ang. embedded systems), w szczególności w urządzeniach opartych na systemie operacyjnym Windows CE. Są używane w routerach firmy Cisco i TP-Link, oraz w konsolach do gier takich jak Nintendo 64, Sony PlayStation, Sony PlayStation 2, Sony PSP. Szacuje się, że procesory MIPS stanowią jedną trzecią produkcji mikroprocesorów typu RISC.

# Rzeczywiste przykłady ISA (5 z 8)

- Język programowania Java jest językiem interpretowanym, który działa w maszynie programowej o nazwie Wirtualna Maszyna Javy (JVM).
- JVM jest napisana w języku natywnym dla szerokiego szeregu procesorów, w tym MIPS i Intel (dla każdego osobno!)
- Jak prawdziwa fizyczna maszyna, JVM ma własne rozkazy ISA zwane kodem bajtowym, aby zapewnić kompatybilność z architekturą każdej fizycznej maszyny na której JVM jest uruchomiona.

# Rzeczywiste przykłady ISA (6 z 8)



# Rzeczywiste przykłady ISA (7 z 8)

- Kod bajtowy Java to język oparty na stosie
- Większość instrukcji to instrukcje bezadresowe
- JVM ma cztery rejestry, które zapewniają dostęp do pięciu obszarów pamięci głównej
- Wszystkie odniesienia do pamięci są offsetem (przesunięciami) w odniesieniu do tych rejestrów. Java nie używa wskaźników ani bezpośrednich odwołań do pamięci
- Java została zaprojektowana z myślą o współdziałaniu platform, a nie wydajności!

# Rzeczywiste przykłady ISA (8 z 8)

- Być może nie słyszeliście o ARM, ale najprawdopodobniej używacie procesorów ARM.
- Jest to najszerzej używana architektura 32-bitowa:
  - 95% smartfonów,
  - 80% aparatów cyfrowych
  - 40% wszystkich odbiorników telewizji cyfrowej
- Założona w 1990 roku przez Apple i inne firmy ARM (Advanced RISC Machine) to teraz brytyjska firma ARM Holding.
- ARM Holding nie produkuje procesorów, sprzedaje licencje na ich produkcję.

# Podsumowanie (1 z 2)

- ISA są klasyfikowane według:
  - Liczby bitów na instrukcję,
  - liczby argumentów na instrukcję,
  - lokalizacji i typów operandów
  - rozmiarów operandów.
- *Endianness* to kolejny ważny aspekt architektury.
- Procesor może przechowywać dane w architekturze:
  - stosu
  - Architektura akumulatorów
  - Architektura rejestru ogólnego przeznaczenia.

# Podsumowanie (2 z 2)

- Instrukcje mogą mieć stałą lub zmienną długość.
- Tryb adresowania ISA jest również ważnym czynnikiem.
- Potok k-etapowy może teoretycznie k-krotnie zwiększyć prędkość wykonania w porównaniu do maszyny bez potoku
- Zagrożenia związane z potokami, takie jak konflikty zasobów rozgałęzienia warunkowe ograniczają to w praktyce.
- Architektury Intel, MIPS, JVM i ARM to przykłady koncepcji prezentowanych w tym rozdziale

# Koniec

- Na dziś

# Architektura Systemów Komputerowych

Pamięć

Dr inż. Krzysztof Kosmowski

*RAM /abr./: Rarely Adequate Memory,*

*because the more memory a computer has, the faster it can produce error messages.*

- *Rzadko Wystarczająca Pamięć – im więcej komputer posiada pamięci, tym szybciej jest w stanie generować błędy* —Anonymous

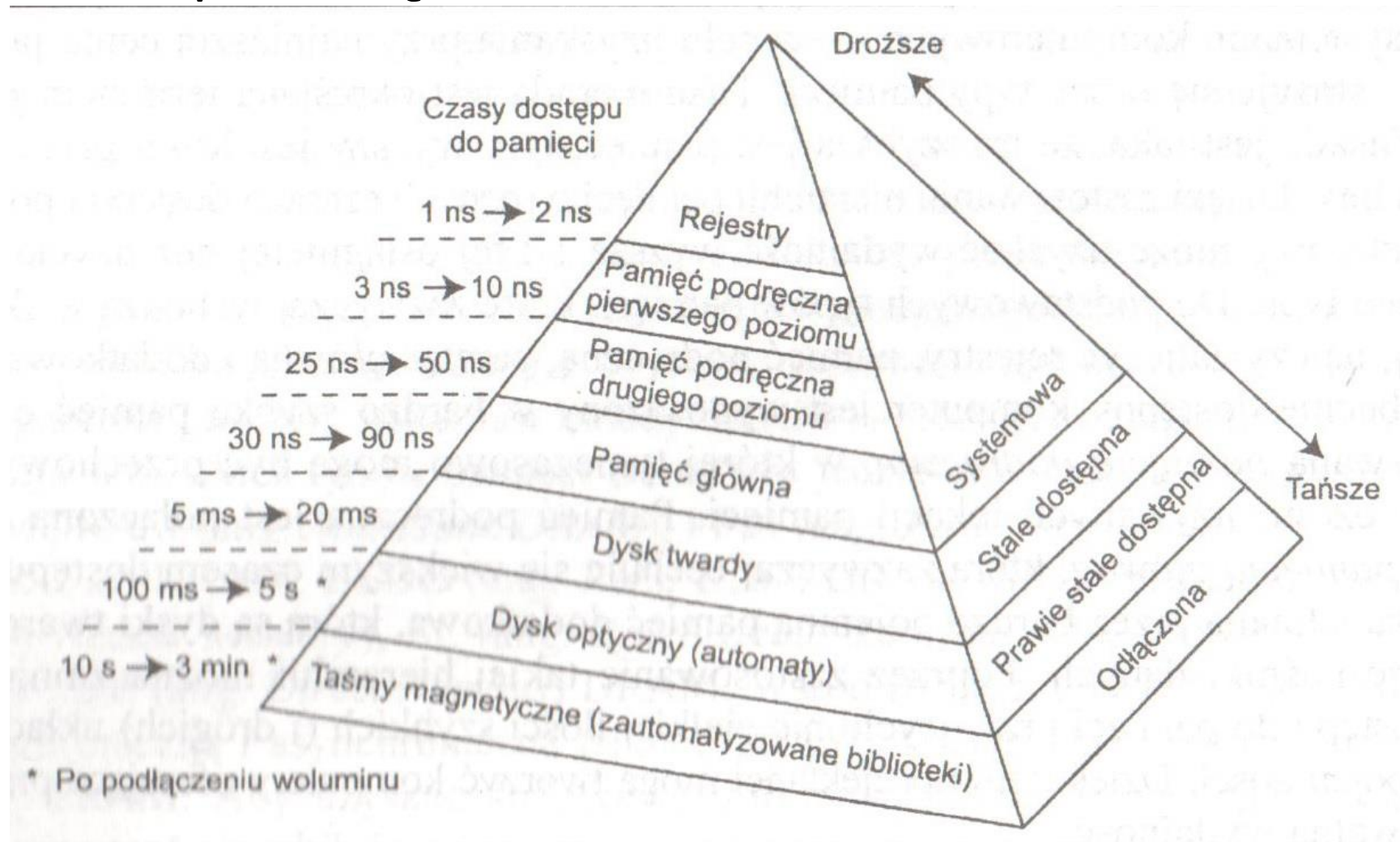
- *Pamięć o pojemności 640 kB powinna wystarczyć każdemu*

*- Bill Gates*

# Hierarchia pamięci

- W systemach komputerowych w celu uzyskania jak największej wydajności jak najmniejszym kosztem stosuje się różne pamięci
- ZASADA: im szybsza pamięć, tym wyższy koszt przechowywania w przeliczeniu na bity
- Stosujemy pamięci o różnych pojemnościach i czasach dostępu, aby zoptymalizować wydajność
- Podstawowe typy pamięci:
  - Rejestry,
  - Pamięć podręczna,
  - Pamięć Główna,
  - Pamięć Dodatkowa

# Hierarchia pamięci



# Hierarchia Pamięci

- W odniesieniu do hierarchii pamięci używana jest terminologia:
  - Trafienie – żądane dane zostały odnalezione w pamięci określonego poziomu (zazwyczaj współczynnik trafień jest brany pod uwagę w przypadku pamięci wyższego poziomu)
  - Chybiecie – żądane dane zostały odnalezione w pamięci określonego poziomu
  - Współczynnik trafień – procentowa wartość operacji dostępu do pamięci znajdującej się na określonym poziomie zakończonych pobraniem danych
  - Współczynnik chybiecie =  $1 - \text{współczynnik trafień}$
  - Czas trafienia – czas potrzebny na pobranie danych z pamięci określonego poziomu
  - Czas chybiecie – czas wymagany na przetworzenie chybionego żądania, jest sumą czasu potrzebnego na zastąpienie bloku pamięci znajdującej się na wyższym poziomie i dodatkowego czasu potrzebnego na dostarczenie danych do procesora

# Pamięć podręczna (Cache)

- Procesor komputera jest bardzo szybki i cały czas pobiera dane z pamięci, która ma czas dostępu znacznie dłuższy, aniżeli szybkość procesora
- Do buforowania danych, które najprawdopodobniej będą w najbliższym czasie potrzebne stosuje się szybką pamięć o niewielkiej pojemności – Cache
- Aby uniknąć odwoływania się do pamięci głównej w celu pobrania danych kopiowane są często używane dane.
- Komputer „nie wie z góry” jakich danych będzie potrzebował – przenosi cały blok danych z pamięci głównej do podręcznej.
- To w jakim miejscu pamięci podręcznej znajdują się poszukiwane dane zależy od:
  - Zasad odwzorowania pamięci podręcznej
  - Jej pojemności

# Pamięć podręczna (Cache)

- Pojemność pamięci Cache drugiego stopnia (L2) wynosi (zazwyczaj) 256 lub 512 KB;
- Pojemność pamięci Cache pierwszego stopnia (L1) wynosi (zazwyczaj) 8 lub 16 KB;
- Pamięć L1 znajduje się w procesorze,
- Pamięć L2 pośredniczy pomiędzy pamięcią główną a procesorem
- Pamięć L1 jest szybsza niż pamięć L2

# Pamięć podręczna (Cache)

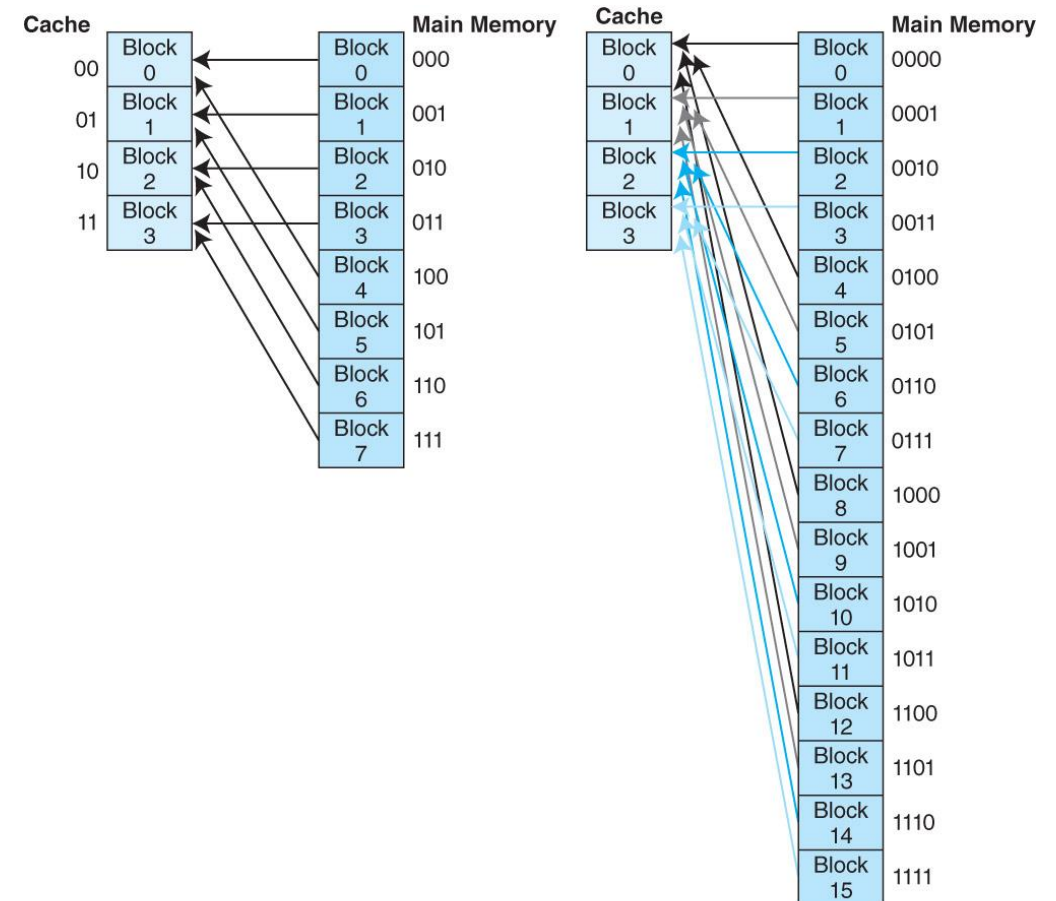
- Do pamięci podręcznej procesor NIE odwołuje się przy użyciu adresu, lecz zawartości
- W celu sprawdzenia, czy żądane dane są przechowywane w pamięci lokalnej konieczne jest jej sprawdzenie lub przeszukanie
- Aby uprościć proces lokalizacji wykorzystywane są różne algorytmy odwzorowania pamięci

# Schematy odwzorowania

- Procesor, aby pobrać żądane dane lub rozkaz generuje ich adres w pamięci głównej
  - Jeśli dane są skopiowane do pamięci podręcznej ich adres uległ zmianie!
- Procesor korzysta ze schematu, który konwertuje adres pamięci głównej na lokalizację w pamięci podręcznej
- Konwersja jest realizowana przez nadanie specjalnego znaczenia bitom adresu pamięci głównej
  - Bity są dzielone na grupy zwane *polami*
  - W zależności od schematu mogą być 2 lub 3 pola

# Mapowanie bezpośrednio bloków pamięci głównej

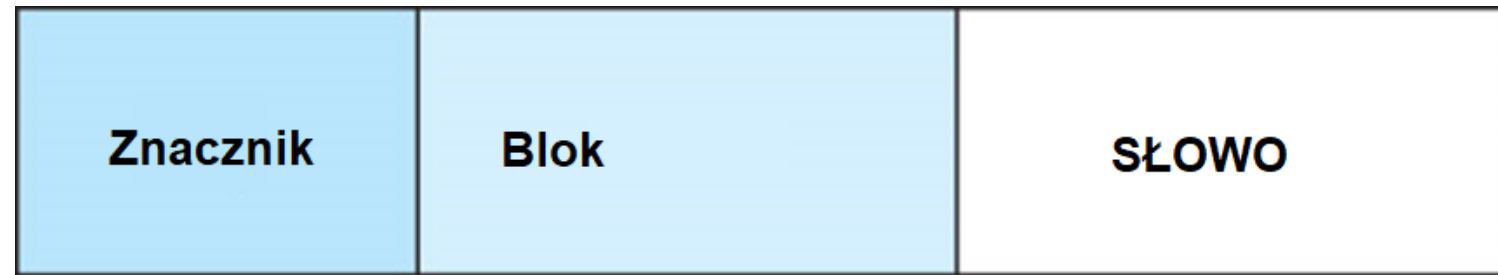
- Odwzorowanie na zasadzie modulo  $N$
- Bloki pamięci głównej „rywalizują” o dostęp do pamięci podręcznej
- Procesor rozpoznaje, który blok pamięci jest zmapowany do Cachu na podstawie znaczników



a) 8 Memory Blocks to 4 Cache Blocks

b) 16 Memory Blocks to 4 Cache Blocks

# Format adresu pamięci głównej przy mapowaniu bezpośrednim



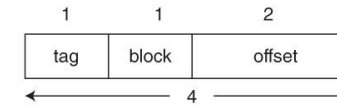
← BITY ADRESU PAMIĘCI GŁÓWNEJ →



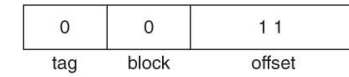
← Bits in Main Memory Address →

# Przykład 1

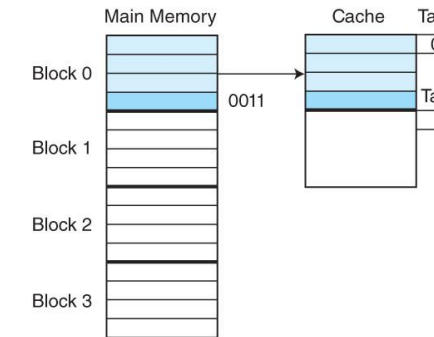
- Pamięć główna zawiera  $2^{14}$  słów
- Cache: 16 bloków po 8 słów
  - 1 blok = 8 słów
- Pamięć główna zawiera  $2^{11}$  bloków
  - $(2^{14}/2^3)$



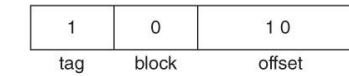
a) Main Memory Format



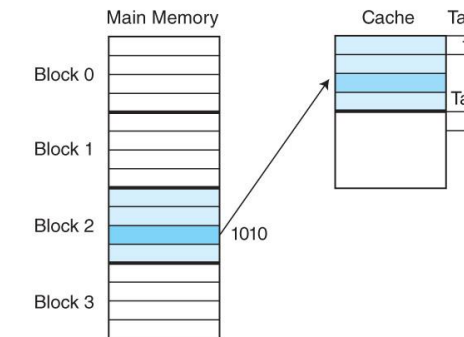
b) The Address 0011 Partitioned into Fields



c) Mapping of Block Containing Address 0011 = 0x3

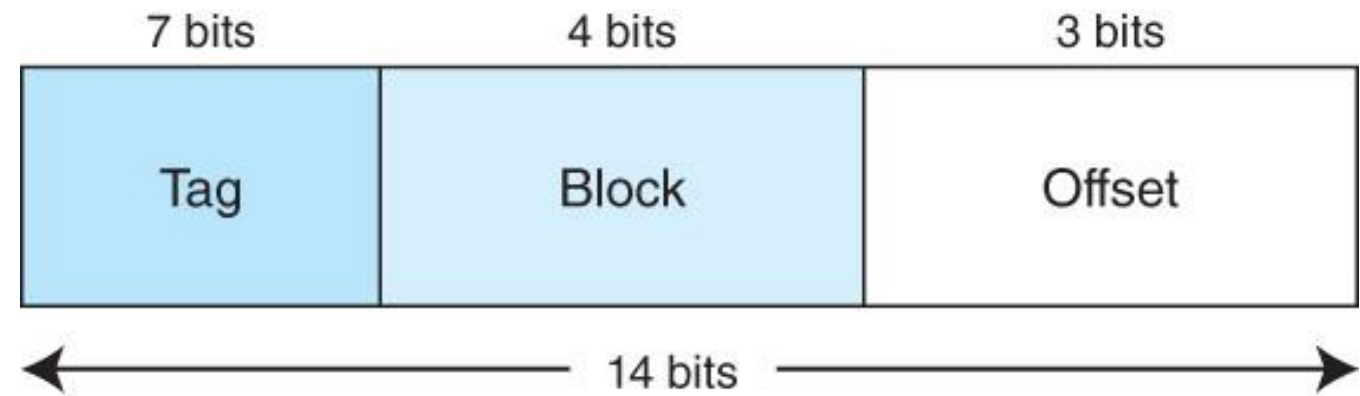


d) The Address 1010 Partitioned into Fields



e) Mapping of Block Containing Address 1010 = 0xA

# Przykład 1

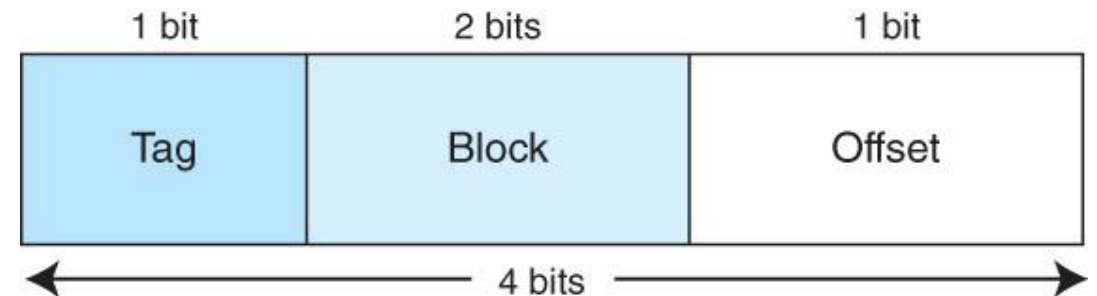


- $2^{14}$  słów = 14 bitowe adresy
- 3 bity najbardziej na prawo – identyfikują **Słowo (Offset)**
- Do wybrania bloku pamięci podręcznej potrzebne są 4 bity ( jest 16 bloków)
- Pozostałe 7 bitów przypada na pole **znacznika (Tag)**
  - Ponieważ blok 0 oraz blok 16 pamięci głównej odwzorowywane są w bloku 0 pamięci podręcznej pole znacznika umożliwia identyfikację, który aktualnie blok znajduje się w Cach'u

# Przykład 2

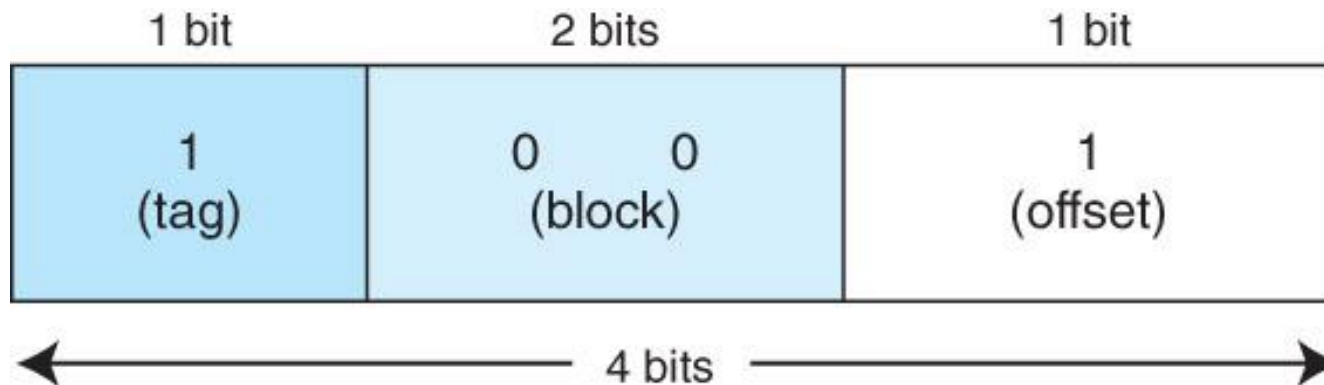
- Pamięć główna:
  - 16 słów pamięci głównej
  - Podział na 8 bloków
- Cache:
  - 4 bloki po 2 słowa
- W tabeli pokazano mapowanie bezpośrednie
- 4 bitowy adres pamięci głównej jest dzielony jak na rysunku obok

| Main Memory                        | Maps To | Cache        |
|------------------------------------|---------|--------------|
| (000) Block 0 (addresses 0x0, 0x1) | →       | Block 0 (00) |
| (001) Block 1 (addresses 0x2, 0x3) | →       | Block 1 (01) |
| (010) Block 2 (addresses 0x4, 0x5) | →       | Block 2 (10) |
| (011) Block 3 (addresses 0x6, 0x7) | →       | Block 3 (11) |
| (100) Block 4 (addresses 0x8, 0x9) | →       | Block 0 (00) |
| (101) Block 5 (addresses 0xA, 0xB) | →       | Block 1 (01) |
| (110) Block 6 (addresses 0xC, 0xD) | →       | Block 2 (10) |
| (111) Block 7 (addresses 0xE, 0xF) | →       | Block 3 (11) |



# Przykład 2

- Generujemy adres pamięci głównej 9
  - Jest on powiązany z blokiem 4 pamięci głównej
  - Powinien być mapowany na blok 0 pamięci podręcznej
- W celu określenia bloku pamięci podręcznej na który wykonano mapowanie komputer posługuje się adresem pamięci głównej:



# Mapowanie pamięci

- Jeżeli wystąpi rywalizacja o blok pamięci podręcznej aktualny blok jest usuwany z Cach'u. Proces ten jest nazywany **zastępowaniem**
- Przy mapowaniu bezpośrednim reguła zastępowania jest z góry określona
- Istnieją inne rodzaje mapowania:
  - Mapowanie asocjacyjne
  - Zespołowe mapowanie asocjacyjne
  - Wymagany jest algorytm wyboru bloku który zostanie zastąpiony nazywany **ZASADĄ ZASTĘPOWANIA**

# ECD – efektywny czas dostępu

- Wydajność pamięci hierarchicznej określa ECD
  - Średnia ważona współczynnika trafień oraz względnych czasów dostępu do kolejnych poziomów pamięci hierarchicznej
- Przykład: czas dostępu do Cache: 10ns, do pamięci głównej: 200ns, współczynnik trafień w pamięci cache: 99%,
  - Średni czas dostępu do elementu w pamięci dwupoziomowej:  
$$\text{ECD} = 0,99(10\text{ns}) + 0,01(200\text{ns}) = 9,9\text{ns} + 2\text{ns} = 11\text{ns}$$
    - Trafienie w pamięci podręcznej
    - Chybiecie w pamięci podręcznej

# Pamięć wirtualna

- Jej zadaniem jest użycie dysku twardego jako rozszerzenie RAM
- Komputer jest w stanie zaadresować więcej pamięci głównej niż w rzeczywistości posiada
- Najczęściej do implementacji pamięci wirtualnej stosuje się **stronicowanie** (*paging*)
  - Pamięć główna i zawarte w niej programy dzielone są na bloki o stałej wielkości
  - Segmenty programu są umieszczane w pamięci gdy jest taka potrzeba
  - Segmenty nie muszą być ciągłe (przechowywane obok siebie), mogą być przechowywane w sposób nieuporządkowany
  - Konieczność „panowania” nad adresami

# Pamięć wirtualna, stronicowanie

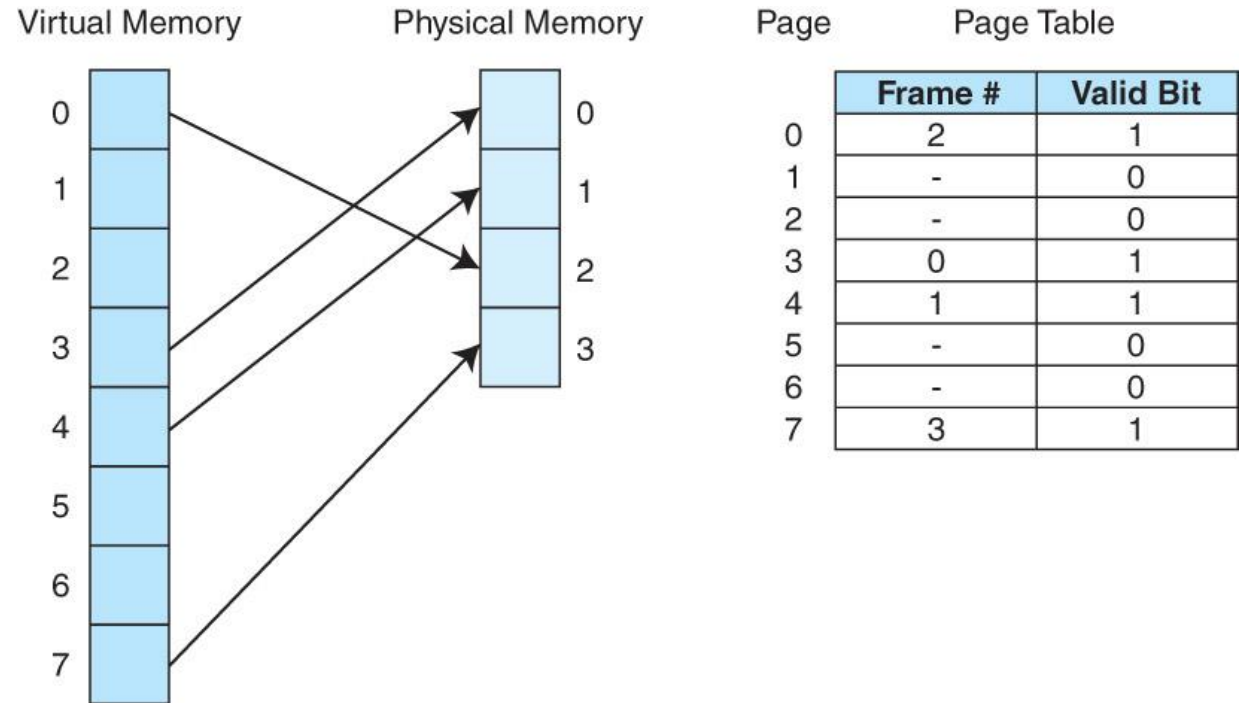
- *Adres Wirtualny* – adres logiczny lub programowy używany przez procesor
- *Adres fizyczny* – rzeczywisty adres pamięci fizycznej
- *Mapowanie* – mechanizm translacji adresów wirtualnych na fizyczne (podobny do mapowania pamięci podręcznej)
- *Ramki strony* – segmenty lub bloki (stałej wielkości), na które dzielona jest fizyczna pamięć główna
- *Strony* - Segmenty lub bloki (stałej wielkości), na które dzielona jest pamięć wirtualna (logiczna pamięć adresowa)
- *Stronicowanie* – proces kopiowania wirtualnej strony z dysku do ramki strony w pamięci głównej
- *Fragmentacja* – pamięć, która stała się bezużyteczna
- *Błąd strony* – zdarzenie, gdy żądana strona NIE znajduje się w pamięci głównej i musi zostać skopiowana z dysku do pamięci

# Pamięć wirtualna, stronicowanie (Przykład)

- Dysponujemy wirtualną przestrzenią adresową liczącą  $2^8$  słów przydzieloną pewnemu procesowi
  - adresy w zakresie 0-255 dec (00-FF hex)
- Pamięć fizyczna: 4 ramki stron
  - długość strony wynosi 32 słowa ( $4 \times 32 = 128 = 2^7$ )
- Adresy wirtualne mają 8 bitów, adresy fizyczne 7 bitów
- Niektóre strony umieszczono w pamięci głównej
- Inne w pamięci wirtualnej

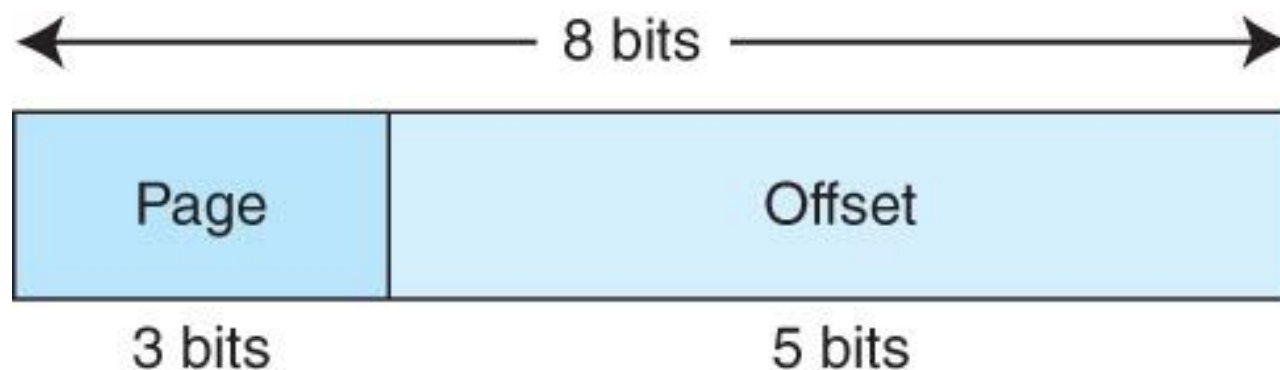
# Pamięć wirtualna, stronicowanie (Przykład, cd)

- Konfiguracja systemu:



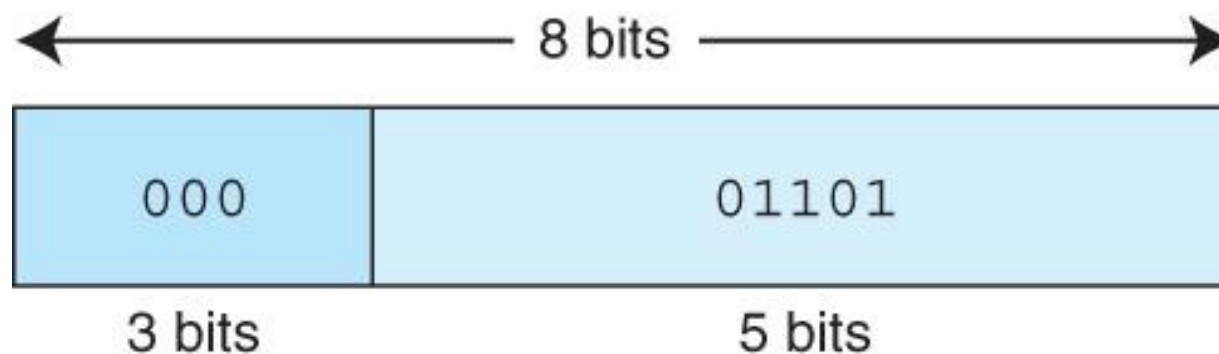
# Pamięć wirtualna, stronicowanie (Przykład, cd)

- Każdy wirtualny adres posiada 8 bitów
- Jest podzielony na 2 pola:
  - Pole strony – 3 bity (dostępne  $2^3$  stron)
  - Długość każdej strony wynosi 32 słowa, czyli  $2^5$
- 8-bitowy wirtualny adres ma format:



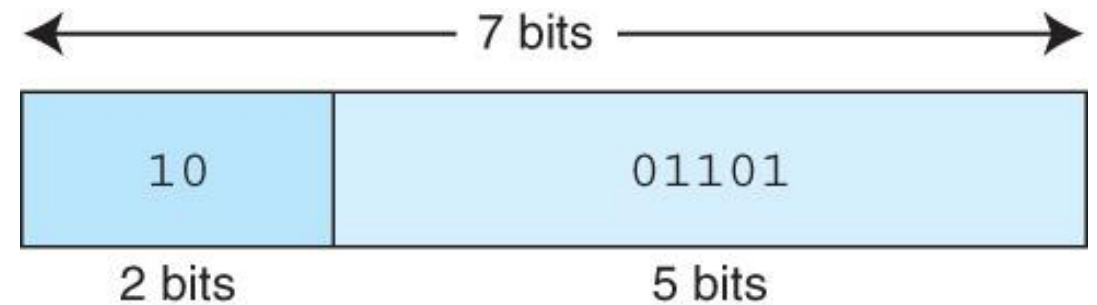
# Pamięć wirtualna, stronicowanie (Przykład, cd)

- System wygenerował wirtualny adres 13 dec = 0D hex = 00001101
- Pole strony: 000
- Pole offsetu: 01101



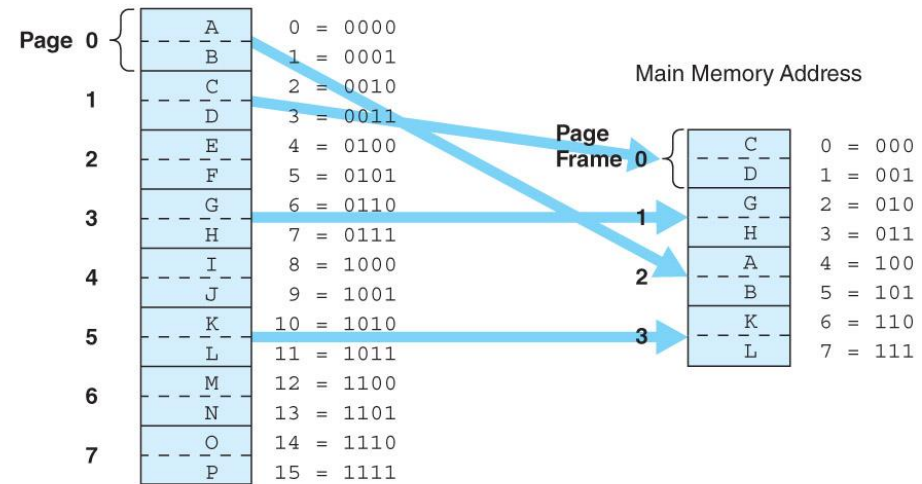
# Pamięć wirtualna, stronicowanie (Przykład, cd)

- Wpis zerowy w tabeli stron:
  - Strona 0 jest odwzorowana przez ramkę 2 fizycznej strony = 10
- Adres fizyczny będzie identyfikowany przez ramkę 2 i przesunięcie 13
  - Adres fizyczny posiada 7 bitów!
  - 4D hex = 77 dec
- Lub inaczej:
  - Początek fizycznej ramki nr 2 = adres 64 dec
  - Przesunięcie = 13
  - Adres=77 dec



# Przykład pamięci o niewielkiej pojemności z wykorzystaniem stronicowania

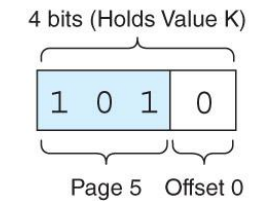
a) Program Address Space



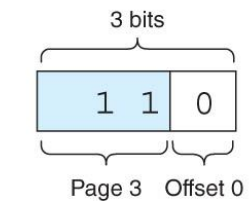
b) Page Table

| Page | Frame | Valid Bit |
|------|-------|-----------|
| 0    | 2     | 1         |
| 1    | 0     | 1         |
| 2    | -     | 0         |
| 3    | 1     | 1         |
| 4    | -     | 0         |
| 5    | 3     | 1         |
| 6    | -     | 0         |
| 7    | -     | 0         |

c) Virtual Address 0xA = 1010<sub>2</sub>



d) Physical Address



# Wskaźniki ECD (efektywny czas dostępu)

- Ma zastosowanie również w przypadku pamięci wirtualnej
- Pamięć wirtualna = dodatkowe opóźnienie
  - Każda operacja dostępu do pamięci = dwie operacje dostępu do pamięci fizycznej:
    - I. Odwołanie się do tabeli stron
    - II. Odwołanie się do danych

# Wskaźniki ECD (efektywny czas dostępu)

- Przykład:
  - czas dostępu do pamięci głównej: 200ns,
  - błąd strony 1% (przez 99% czasu żądana strona jest szukana w pamięci),
  - czas dostępu do strony, która nie znajduje się w pamięci dodatkowo 10ms (przesłanie strony do pamięci, uaktualnienie tabeli stron, pobranie danych)
  - $ECD = 0,99(200ns + 200ns) + 0,01(10ms) = 100,396 \text{ us}$
- 100% stron znajduje się w pamięci głównej:
  - $ECD = 1,000,99(200ns + 200ns) = 400ns$

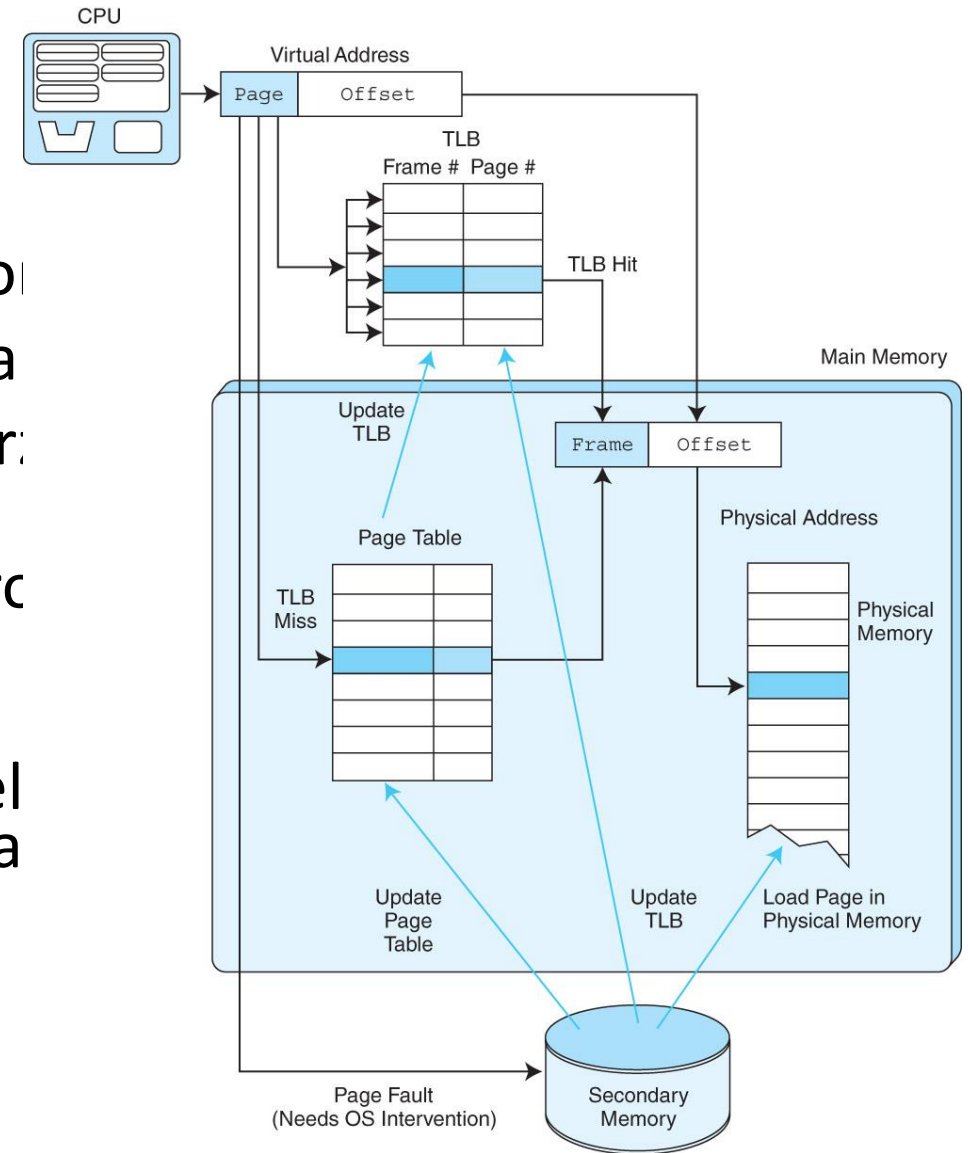
# Zwiększenie szybkości przeszukiwania tabeli stron

- Bufor TLB: bufor translacji adresowej (*translation look-aside buffer*)
- Wpis w buforze TLB zawiera numer wirtualnej strony i powiązany z nią numer strony
- TLB – najczęściej asocjacyjna **pamięć podręczna**

| Virtual Page Number | Physical Page Number |
|---------------------|----------------------|
| -                   | -                    |
| 5                   | 1                    |
| 2                   | 0                    |
| -                   | -                    |
| -                   | -                    |
| 1                   | 3                    |
| 6                   | 2                    |

# Zastosowanie bufora TLB

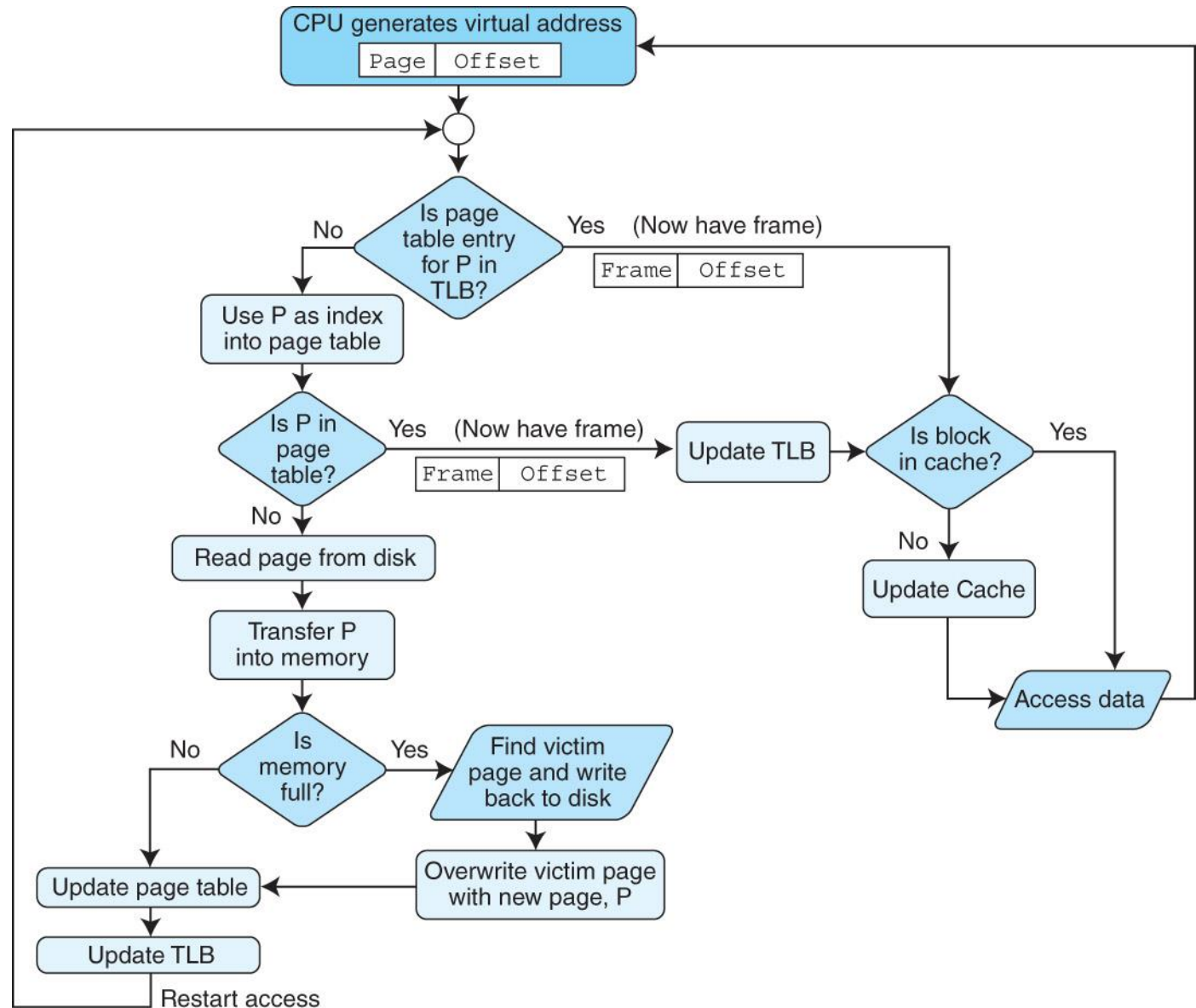
- 1. pobranie z adresu wirtualnego numeru strony
- 2. pobranie z adresu wirtualnego przesunięcia
- Odszukanie numeru strony wirtualnej w bufor: TLB
- Jeżeli znaleziono w TLB parę: nr wirtualnej strony nr ramki -> dodać offset i uzyskać alokację pamięci
- Jeżeli chybienie: przejście do tabeli stron, jeżeli strona znajduje się w pamięci: użyć numeru ramki i dodać przesunięcie
- Jeżeli strona NIE znajduje się w pamięci -> ponowna operacja dostępu



# POŁĄCZENIE ELEMENTÓW: Pamięć podręczna + bufory TLB + stronicowanie

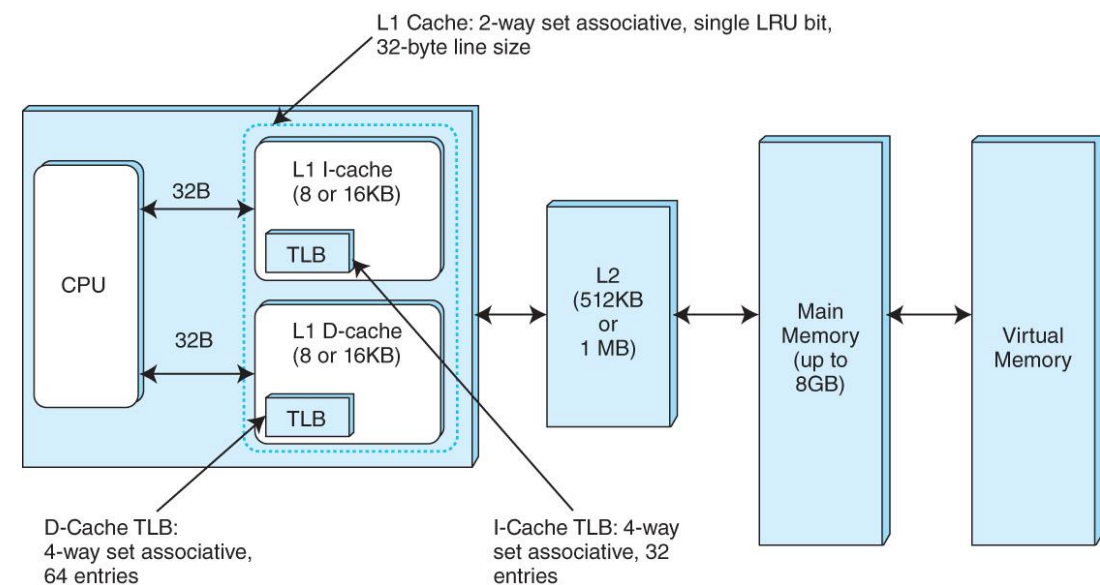
- Gdy Procesor generuje adres jest to adres wirtualny (lub względny dla programu) MUSI być zamieniony na adres fizyczny!
  - Dwie metody wykonania zadania:
    - I – wykorzystanie bufora TLB (trafienie)
    - II – chybiecie w TLB – szukanie w pamięci głównej przy użyciu tabeli stron
  - Utworzenie adresu fizycznego (jeszcze nie pobrano danych!)
  - Dwie metody pobierania danych:
    - I – przeszukiwanie pamięci podręcznej (trafienie)
    - II – chybiecie – pobranie zawartości z pamięci głównej (zazwyczaj uaktualnienie zawartości cachu)

# POŁĄCZENIE ELEMENTÓW: Pamięć podręczna + bufory TLB + stronicowanie



# Hierarchia pamięci procesora pentium

- 32-bitowe adresy wirtualne i 32 bitowe adresy fizyczne
- Stronicowanie: strony 4kB lub 4 MB
- L1, L2 – 32 bajtowe bloki
- L1 – składa się z 2 podpamięci:
  - l – przechowuje instrukcje (I-Cache)
  - D – przechowuje dane (D-Cache)
- L1- 2-blokowe mapowanie asocjacyjne
- Bufory TLB: 4-blokowe mapowanie asocjacyjne
- TLB I-cache: 32 wpisy
- TLB D-cache: 64 wpisy
- TO TYLKO OGÓLNY ZARYS



- Teraz wystarczy napisać egzamin